

AI Agent 架构与私人订制版实践

从工具调用、RAG、记忆系统到多 Agent 工作流

NewPhant Poynting Cheque9 Lucky-Five QuisVic 编撰

版本：第 1 版（完整初稿）

二〇二六年六月

本讲义面向有 Python 基础的开发者、研究生与 AI 工程读者，目标是从原理、架构、框架和项目实践四个层面理解并搭建私人订制版 Agent。

目录

前言

xvii

第一部分 Agent 基础与统一模型	1
第一章 从 Chatbot 到 Agent	2
1.1 为什么“会聊天”不等于 Agent	2
1.2 Agent 的工作定义	3
1.3 贯穿例子：论文阅读任务的四种实现	5
1.4 从概念到最小系统	6
1.5 私人订制化为什么需要 Agent	8
1.6 边界、误区与练习	9
第一章 练习	10
第二章 Agent 的最小组成	11
2.1 从一次性回答到最小闭环	11
2.2 最小组成的工程定义	12
2.3 贯穿例子：只读文件整理 Agent	13
2.4 最小 Python 骨架	14
2.5 最小不等于简陋	17
2.6 失败模式、边界与练习	18
第二章 练习	19
第三章 LLM、Prompt 与上下文	20
3.1 上下文不是一个大 prompt	20
3.2 上下文七层模型	20
3.3 贯穿例子：论文阅读 Agent 的上下文包	21
3.4 上下文构造器：从状态生成消息	23
3.5 冲突、压缩与长任务	24
3.6 失败模式、评估与练习	25
第三章 练习	25
第四章 工具调用机制	26
4.1 工具调用不等于 Agent	26
4.2 工具契约的工程定义	26
4.3 贯穿例子：文件整理工具族	27
4.4 执行器最小骨架	28
4.5 权限、安全与私人助理边界	30
4.6 失败模式、评估与练习	30
第四章 练习	31
第五章 状态、记忆与任务轨迹	32
5.1 四类“历史”的边界	32

5.2 运行时状态如何演化	33
5.3 轨迹与 checkpoint	34
5.4 长期记忆的写入审批	35
5.5 私人科研 Agent 的分层实践	36
5.6 失败模式、评估与练习	36
第五章 练习	37
第六章 知识库与 RAG 基础	38
6.1 从普通问答到 RAG Agent	38
6.2 基础 pipeline 的工程定义	39
6.3 贯穿例子：论文实验问题的 RAG 流程	39
6.4 chunk、metadata 与权限过滤	40
6.5 最小 RAG 代码骨架	41
6.6 评估、反例与练习	43
第六章 练习	43
第二部分 十大 Agent 架构	45
第七章 LLM Chain Agent	46
7.1 问题引入	46
7.2 执行流程	46
7.3 最小 Python 骨架	46
7.4 深入讲解	47
7.5 工程实现要点	47
7.6 适用与不适用	47
7.7 失败模式与调试	47
7.8 评估指标	48
7.9 私人定制化实践	48
7.10 本章例题	48
7.11 失败模式矩阵	48
7.12 进阶练习	48
7.13 设计边界与选择表	49
7.14 实现清单	49
7.15 本章小项目	49
7.16 与后续章节的关系	49
7.17 案例展开与验收标准	49
7.18 常见误区与反例	50
7.19 小结	50
第七章 练习	51
第八章 Tool-Using Agent	52
8.1 问题引入	52
8.2 执行流程	52
8.3 最小 Python 骨架	52
8.4 深入讲解	53

8.5 工程实现要点	53
8.6 适用与不适用	53
8.7 失败模式与调试	53
8.8 评估指标	54
8.9 私人定制化实践	54
8.10 本章例题	54
8.11 失败模式矩阵	54
8.12 进阶练习	54
8.13 设计边界与选择表	54
8.14 实现清单	55
8.15 本章小项目	55
8.16 与后续章节的关系	55
8.17 案例展开与验收标准	55
8.18 常见误区与反例	56
8.19 小结	56
第八章 练习	56
第九章 ReAct Agent	57
9.1 问题引入	57
9.2 执行流程	57
9.3 最小 Python 骨架	57
9.4 深入讲解	58
9.5 工程实现要点	58
9.6 适用与不适用	58
9.7 失败模式与调试	58
9.8 评估指标	59
9.9 私人定制化实践	59
9.10 本章例题	59
9.11 失败模式矩阵	59
9.12 进阶练习	59
9.13 设计边界与选择表	59
9.14 实现清单	60
9.15 本章小项目	60
9.16 与后续章节的关系	60
9.17 案例展开与验收标准	60
9.18 常见误区与反例	61
9.19 补充注记	61
9.20 小结	62
第九章 练习	62
第十章 Plan-and-Execute Agent	63
10.1 问题引入	63
10.2 执行流程	63
10.3 最小 Python 骨架	63
10.4 深入讲解	64

10.5 工程实现要点	64
10.6 适用与不适用	64
10.7 失败模式与调试	64
10.8 评估指标	65
10.9 私人定制化实践	65
10.10 本章例题	65
10.11 失败模式矩阵	65
10.12 进阶练习	65
10.13 设计边界与选择表	65
10.14 实现清单	66
10.15 本章小项目	66
10.16 与后续章节的关系	66
10.17 案例展开与验收标准	66
10.18 常见误区与反例	67
10.19 小结	67
第十章 练习	67
第十一章 RAG Agent	69
11.1 问题引入	69
11.2 执行流程	69
11.3 最小 Python 骨架	69
11.4 深入讲解	70
11.5 工程实现要点	70
11.6 适用与不适用	70
11.7 失败模式与调试	70
11.8 评估指标	71
11.9 私人定制化实践	71
11.10 本章例题	71
11.11 失败模式矩阵	71
11.12 进阶练习	71
11.13 设计边界与选择表	71
11.14 实现清单	72
11.15 本章小项目	72
11.16 与后续章节的关系	72
11.17 案例展开与验收标准	72
11.18 常见误区与反例	73
11.19 补充注记	73
11.20 小结	74
第十一章 练习	74
第十二章 Memory-Augmented Agent	75
12.1 问题引入	75
12.2 执行流程	75
12.3 最小 Python 骨架	75
12.4 深入讲解	76

12.5 工程实现要点	76
12.6 适用与不适用	76
12.7 失败模式与调试	76
12.8 评估指标	77
12.9 私人定制化实践	77
12.10 本章例题	77
12.11 失败模式矩阵	77
12.12 进阶练习	77
12.13 设计边界与选择表	77
12.14 实现清单	78
12.15 本章小项目	78
12.16 与后续章节的关系	78
12.17 案例展开与验收标准	78
12.18 常见误区与反例	79
12.19 小结	79
第十二章 练习	79
第十三章 Reflection Agent	80
13.1 问题引入	80
13.2 执行流程	80
13.3 最小 Python 骨架	80
13.4 深入讲解	81
13.5 工程实现要点	81
13.6 适用与不适用	81
13.7 失败模式与调试	81
13.8 评估指标	82
13.9 私人定制化实践	82
13.10 本章例题	82
13.11 失败模式矩阵	82
13.12 进阶练习	82
13.13 设计边界与选择表	82
13.14 实现清单	83
13.15 本章小项目	83
13.16 与后续章节的关系	83
13.17 案例展开与验收标准	83
13.18 常见误区与反例	84
13.19 补充注记	84
13.20 小结	85
第十三章 练习	85
第十四章 Workflow / Graph Agent	86
14.1 问题引入	86
14.2 执行流程	86
14.3 最小 Python 骨架	86
14.4 深入讲解	87

14.5 工程实现要点	87
14.6 适用与不适用	87
14.7 失败模式与调试	87
14.8 评估指标	88
14.9 私人定制化实践	88
14.10 本章例题	88
14.11 失败模式矩阵	88
14.12 进阶练习	88
14.13 设计边界与选择表	89
14.14 实现清单	89
14.15 本章小项目	89
14.16 与后续章节的关系	89
14.17 案例展开与验收标准	89
14.18 常见误区与反例	90
14.19 小结	90
第十四章 练习	91
第十五章 Multi-Agent Collaboration	92
15.1 问题引入	92
15.2 执行流程	92
15.3 最小 Python 骨架	92
15.4 深入讲解	93
15.5 工程实现要点	93
15.6 适用与不适用	93
15.7 失败模式与调试	93
15.8 评估指标	94
15.9 私人定制化实践	94
15.10 本章例题	94
15.11 失败模式矩阵	94
15.12 进阶练习	94
15.13 设计边界与选择表	94
15.14 实现清单	95
15.15 本章小项目	95
15.16 与后续章节的关系	95
15.17 案例展开与验收标准	95
15.18 常见误区与反例	96
15.19 小结	96
第十五章 练习	96
第十六章 Human-in-the-loop Agent	98
16.1 问题引入	98
16.2 执行流程	98
16.3 最小 Python 骨架	98
16.4 深入讲解	99
16.5 工程实现要点	99

16.6 适用与不适用	99
16.7 失败模式与调试	99
16.8 评估指标	100
16.9 私人定制化实践	100
16.10 本章例题	100
16.11 失败模式矩阵	100
16.12 进阶练习	100
16.13 设计边界与选择表	100
16.14 实现清单	101
16.15 本章小项目	101
16.16 与后续章节的关系	101
16.17 案例展开与验收标准	101
16.18 常见误区与反例	102
16.19 补充注记	102
16.20 小结	103
第十六章 练习	103

第三部分 主流框架与工程实践 104

第十七章 原生 SDK：从零搭建 Agent 105

17.1 问题引入	105
17.2 执行流程	105
17.3 最小 Python 骨架	105
17.4 深入讲解	106
17.5 工程实现要点	106
17.6 适用与不适用	106
17.7 失败模式与调试	106
17.8 评估指标	107
17.9 私人定制化实践	107
17.10 本章例题	107
17.11 失败模式矩阵	107
17.12 进阶练习	107
17.13 设计边界与选择表	107
17.14 实现清单	108
17.15 本章小项目	108
17.16 与后续章节的关系	108
17.17 案例展开与验收标准	108
17.18 常见误区与反例	109
17.19 小结	109
第十七章 练习	109

第十八章 LangChain：组件化 Agent 原型 110

18.1 问题引入	110
18.2 执行流程	110
18.3 最小 Python 骨架	110

18.4 深入讲解	111
18.5 工程实现要点	111
18.6 适用与不适用	111
18.7 失败模式与调试	111
18.8 评估指标	112
18.9 私人定制化实践	112
18.10 本章例题	112
18.11 失败模式矩阵	112
18.12 进阶练习	112
18.13 设计边界与选择表	112
18.14 实现清单	113
18.15 本章小项目	113
18.16 与后续章节的关系	113
18.17 案例展开与验收标准	113
18.18 常见误区与反例	114
18.19 补充注记	114
18.20 小结	115
第十八章 练习	115
第十九章 LangGraph: 可控 Agent 状态机	116
19.1 问题引入	116
19.2 执行流程	116
19.3 最小 Python 骨架	116
19.4 深入讲解	117
19.5 工程实现要点	117
19.6 适用与不适用	117
19.7 失败模式与调试	117
19.8 评估指标	118
19.9 私人定制化实践	118
19.10 本章例题	118
19.11 失败模式矩阵	118
19.12 进阶练习	118
19.13 设计边界与选择表	119
19.14 实现清单	119
19.15 本章小项目	119
19.16 与后续章节的关系	119
19.17 案例展开与验收标准	119
19.18 常见误区与反例	120
19.19 小结	120
第十九章 练习	121
第二十章 CrewAI: 角色分工式多 Agent	122
20.1 问题引入	122
20.2 执行流程	122
20.3 最小 Python 骨架	122

20.4 深入讲解	123
20.5 工程实现要点	123
20.6 适用与不适用	123
20.7 失败模式与调试	123
20.8 评估指标	124
20.9 私人定制化实践	124
20.10 本章例题	124
20.11 失败模式矩阵	124
20.12 进阶练习	124
20.13 设计边界与选择表	124
20.14 实现清单	125
20.15 本章小项目	125
20.16 与后续章节的关系	125
20.17 案例展开与验收标准	125
20.18 常见误区与反例	126
20.19 小结	126
第二十章 练习	126
第二十一章 AutoGen: 对话式多 Agent 系统	127
21.1 问题引入	127
21.2 执行流程	127
21.3 最小 Python 骨架	127
21.4 深入讲解	128
21.5 工程实现要点	128
21.6 适用与不适用	128
21.7 失败模式与调试	128
21.8 评估指标	129
21.9 私人定制化实践	129
21.10 本章例题	129
21.11 失败模式矩阵	129
21.12 进阶练习	129
21.13 设计边界与选择表	129
21.14 实现清单	130
21.15 本章小项目	130
21.16 与后续章节的关系	130
21.17 案例展开与验收标准	130
21.18 常见误区与反例	131
21.19 小结	131
第二十一章 练习	131
第二十二章 MCP: Agent 的外部工具生态	132
22.1 问题引入	132
22.2 执行流程	132
22.3 最小 Python 骨架	132
22.4 深入讲解	133

22.5 工程实现要点	133
22.6 适用与不适用	133
22.7 失败模式与调试	133
22.8 评估指标	134
22.9 私人定制化实践	134
22.10 本章例题	134
22.11 失败模式矩阵	134
22.12 进阶练习	134
22.13 设计边界与选择表	134
22.14 实现清单	135
22.15 本章小项目	135
22.16 与后续章节的关系	135
22.17 案例展开与验收标准	135
22.18 常见误区与反例	136
22.19 补充注记	136
22.20 小结	137
第二十二章 练习	137
第二十三章 向量数据库与知识库工程	138
23.1 问题引入	138
23.2 执行流程	138
23.3 最小 Python 骨架	138
23.4 深入讲解	139
23.5 工程实现要点	139
23.6 适用与不适用	139
23.7 失败模式与调试	139
23.8 评估指标	140
23.9 私人定制化实践	140
23.10 本章例题	140
23.11 失败模式矩阵	140
23.12 进阶练习	140
23.13 设计边界与选择表	140
23.14 实现清单	141
23.15 本章小项目	141
23.16 与后续章节的关系	141
23.17 案例展开与验收标准	141
23.18 常见误区与反例	142
23.19 补充注记	142
23.20 小结	143
第二十三章 练习	143
第二十四章 Agent 日志、调试与可观测性	144
24.1 问题引入	144
24.2 执行流程	144
24.3 最小 Python 骨架	144

24.4 深入讲解	145
24.5 工程实现要点	145
24.6 适用与不适用	145
24.7 失败模式与调试	145
24.8 评估指标	146
24.9 私人定制化实践	146
24.10 本章例题	146
24.11 失败模式矩阵	146
24.12 进阶练习	146
24.13 设计边界与选择表	146
24.14 实现清单	147
24.15 本章小项目	147
24.16 与后续章节的关系	147
24.17 案例展开与验收标准	147
24.18 常见误区与反例	148
24.19 补充注记	148
24.20 小结	149
第二十四章 练习	149

第四部分 私人定制化 Agent 项目实战 150

第二十五章 私人知识库 Agent 151

25.1 问题引入	151
25.2 执行流程	151
25.3 最小 Python 骨架	151
25.4 深入讲解	152
25.5 工程实现要点	152
25.6 适用与不适用	152
25.7 失败模式与调试	152
25.8 评估指标	153
25.9 私人定制化实践	153
25.10 本章例题	153
25.11 失败模式矩阵	153
25.12 进阶练习	153
25.13 设计边界与选择表	153
25.14 实现清单	154
25.15 本章小项目	154
25.16 与后续章节的关系	154
25.17 案例展开与验收标准	154
25.18 常见误区与反例	155
25.19 补充注记	155
25.20 小结	156
第二十五章 练习	156

第二十六章 论文阅读 Agent 157

26.1 问题引入	157
26.2 执行流程	157
26.3 最小 Python 骨架	157
26.4 深入讲解	158
26.5 工程实现要点	158
26.6 适用与不适用	158
26.7 失败模式与调试	158
26.8 评估指标	159
26.9 私人定制化实践	159
26.10 本章例题	159
26.11 失败模式矩阵	159
26.12 进阶练习	159
26.13 设计边界与选择表	159
26.14 实现清单	160
26.15 本章小项目	160
26.16 与后续章节的关系	160
26.17 案例展开与验收标准	160
26.18 常见误区与反例	161
26.19 补充注记	161
26.20 小结	162
第二十六章 练习	162
第二十七章 讲义写作 Agent	163
27.1 问题引入	163
27.2 执行流程	163
27.3 最小 Python 骨架	163
27.4 深入讲解	164
27.5 工程实现要点	164
27.6 适用与不适用	164
27.7 失败模式与调试	164
27.8 评估指标	165
27.9 私人定制化实践	165
27.10 本章例题	165
27.11 失败模式矩阵	165
27.12 进阶练习	165
27.13 设计边界与选择表	165
27.14 实现清单	166
27.15 本章小项目	166
27.16 与后续章节的关系	166
27.17 案例展开与验收标准	166
27.18 常见误区与反例	167
27.19 补充注记	167
27.20 小结	168
第二十七章 练习	168

第二十八章 代码开发 Agent	169
28.1 问题引入	169
28.2 执行流程	169
28.3 最小 Python 骨架	169
28.4 深入讲解	170
28.5 工程实现要点	170
28.6 适用与不适用	170
28.7 失败模式与调试	170
28.8 评估指标	170
28.9 私人定制化实践	171
28.10 本章例题	171
28.11 失败模式矩阵	171
28.12 进阶练习	171
28.13 设计边界与选择表	171
28.14 实现清单	172
28.15 本章小项目	172
28.16 与后续章节的关系	172
28.17 案例展开与验收标准	172
28.18 常见误区与反例	173
28.19 补充注记	173
28.20 小结	173
第二十八章 练习	174
第二十九章 多 Agent 科研 workflows	175
29.1 问题引入	175
29.2 执行流程	175
29.3 最小 Python 骨架	175
29.4 深入讲解	176
29.5 工程实现要点	176
29.6 适用与不适用	176
29.7 失败模式与调试	176
29.8 评估指标	177
29.9 私人定制化实践	177
29.10 本章例题	177
29.11 失败模式矩阵	177
29.12 进阶练习	177
29.13 设计边界与选择表	177
29.14 实现清单	178
29.15 本章小项目	178
29.16 与后续章节的关系	178
29.17 案例展开与验收标准	178
29.18 常见误区与反例	179
29.19 补充注记	179
29.20 小结	180
第二十九章 练习	180

第五部分 评估、安全与前沿方向	181
第三十章 Agent 评估方法	182
30.1 问题引入	182
30.2 执行流程	182
30.3 最小 Python 骨架	182
30.4 深入讲解	183
30.5 工程实现要点	183
30.6 适用与不适用	183
30.7 失败模式与调试	183
30.8 评估指标	184
30.9 私人定制化实践	184
30.10 本章例题	184
30.11 失败模式矩阵	184
30.12 进阶练习	184
30.13 设计边界与选择表	184
30.14 实现清单	185
30.15 本章小项目	185
30.16 与后续章节的关系	185
30.17 案例展开与验收标准	185
30.18 常见误区与反例	186
30.19 补充注记	186
30.20 小结	187
第三十章 练习	187
第三十一章 失败模式与调试	188
31.1 问题引入	188
31.2 执行流程	188
31.3 最小 Python 骨架	188
31.4 深入讲解	189
31.5 工程实现要点	189
31.6 适用与不适用	189
31.7 失败模式与调试	189
31.8 评估指标	190
31.9 私人定制化实践	190
31.10 本章例题	190
31.11 失败模式矩阵	190
31.12 进阶练习	190
31.13 设计边界与选择表	190
31.14 实现清单	191
31.15 本章小项目	191
31.16 与后续章节的关系	191
31.17 案例展开与验收标准	191
31.18 常见误区与反例	192
31.19 补充注记	192

31.20 小结	193
第三十一章 练习	193
第三十二章 安全边界与权限控制	194
32.1 问题引入	194
32.2 执行流程	194
32.3 最小 Python 骨架	194
32.4 深入讲解	195
32.5 工程实现要点	195
32.6 适用与不适用	195
32.7 失败模式与调试	195
32.8 评估指标	196
32.9 私人定制化实践	196
32.10 本章例题	196
32.11 失败模式矩阵	196
32.12 进阶练习	196
32.13 设计边界与选择表	197
32.14 实现清单	197
32.15 本章小项目	197
32.16 与后续章节的关系	197
32.17 案例展开与验收标准	197
32.18 常见误区与反例	198
32.19 补充注记	198
32.20 小结	199
第三十二章 练习	199
第三十三章 Agent Benchmark 与科研前沿	200
33.1 问题引入	200
33.2 执行流程	200
33.3 最小 Python 骨架	200
33.4 深入讲解	201
33.5 工程实现要点	201
33.6 适用与不适用	201
33.7 失败模式与调试	201
33.8 评估指标	202
33.9 私人定制化实践	202
33.10 本章例题	202
33.11 失败模式矩阵	202
33.12 进阶练习	202
33.13 设计边界与选择表	202
33.14 实现清单	203
33.15 本章小项目	203
33.16 与后续章节的关系	203
33.17 案例展开与验收标准	203
33.18 常见误区与反例	204

33.19 小结	204
第三十三章 练习	204
第三十四章 构建自己的长期 Agent 系统	206
34.1 问题引入	206
34.2 执行流程	206
34.3 最小 Python 骨架	206
34.4 深入讲解	207
34.5 工程实现要点	207
34.6 适用与不适用	207
34.7 失败模式与调试	207
34.8 评估指标	208
34.9 私人定制化实践	208
34.10 本章例题	208
34.11 失败模式矩阵	208
34.12 进阶练习	208
34.13 设计边界与选择表	208
34.14 实现清单	209
34.15 本章小项目	209
34.16 与后续章节的关系	209
34.17 案例展开与验收标准	209
34.18 常见误区与反例	210
34.19 补充注记	210
34.20 小结	211
第三十四章 练习	211
参考文献与延伸阅读	212

前言

这本讲义讨论 AI Agent 的架构与私人定制化实践。它不是面向完全零基础用户的科普，也不是某个框架的 API 手册，而是面向有 Python 基础的开发者、研究生和 AI 工程读者的系统讲义。

本讲义的基本问题是：如何从零设计、实现、评估并迭代一个可靠的私人定制化 Agent？这里的私人定制化不是简单地给模型加一句“你是我的助手”，而是把用户的知识库、 workflows、工具权限、长期偏好和反馈机制纳入系统设计。

读者最好具备以下基础：能够阅读 Python 代码，理解 JSON、函数、类、命令行和 HTTP API；知道大语言模型可以通过消息接口完成文本生成；对 RAG、工具调用或 LangChain 等概念有所耳闻。没有这些经验也可以阅读前两篇，但工程章节需要边读边实践。

本书采用五篇结构：第一篇建立 Agent 的统一模型；第二篇讲解十大常见架构；第三篇比较主流框架与工程实践；第四篇完成私人定制化项目；第五篇讨论评估、安全和长期运行。每章都尽量保持同一节奏：先提出问题，再给定义和最小代码，然后分析失败模式，最后落到私人 Agent 实践。

代码示例以 Python 为主。为了突出架构，许多代码会省略具体 SDK 的认证细节。真实项目中必须加入超时、重试、日志、权限控制和测试。凡涉及文件写入、邮件发送、外部发布、命令执行、付款或删除操作，都应默认需要人工确认。

如果只想快速上手，可以先读第 1、2、4、6、14、19、22、25、30、32、34 章。如果希望系统掌握 Agent 架构，则应按顺序阅读，并把每章练习改造成自己的小项目。

第一部分

Agent 基础与统一模型

第一章 从 Chatbot 到 Agent

内容提要

- ❑ 本章不急着介绍任何框架，而是先回答一个根本问题：什么样的系统才值得称为 Agent。
- ❑ 我们从 Chatbot、RAG Bot、工具助手和 Agent 的对比出发，建立目标、状态、动作、观察和反馈的统一模型。
- ❑ 本章的样板原则是少分节、深展开：概念必须落到系统结构、代码骨架、失败模式和私人订制化场景中。

1.1 为什么“会聊天”不等于 Agent

近几年，许多产品都把自己称为 Agent。一个聊天窗口，如果能联网搜索，就被称为 Agent；一个问答机器人，如果能读取 PDF，也被称为 Agent；一个工作流工具，如果能把几个大语言模型调用串起来，也被称为 Agent。这样的用法在宣传语里很方便，但在工程上非常危险。因为它把几类本质不同的系统混在一起，让我们无法判断一个系统到底能做什么、不能做什么、失败后怎样修复。

先看一个最普通的问题：

用户问：请帮我总结这篇论文的核心贡献。

如果系统只是把这个问题交给模型，然后模型根据用户粘贴的文本回答，那么它是 Chatbot。它的核心行为是“读入一段上下文，生成一段文本”。如果系统先从论文库中检索相关段落，再把段落和问题一起交给模型，那么它是 RAG Bot。它的核心行为是“检索证据，再生成答案”。如果系统能调用一个 PDF 解析工具、一个搜索工具和一个引用格式化工具，但每一步都由用户手动触发，那么它是工具助手。它的核心行为是“把外部能力封装给用户使用”。

真正接近 Agent 的系统，会多出一层任务执行结构。它会判断当前是否已有论文文件；如果没有，会请求用户上传，或者在授权范围内搜索；如果有，会先解析章节，再分别读取摘要、方法、实验和结论；如果证据不足，会继续读取更多段落；如果发现实验部分缺失，会在最终答案里标注“不足以判断实验贡献”；如果任务完成，会给出带来源的阅读卡片。这里的关键不是它“说话更像人”，而是它围绕一个目标维护状态、选择动作、观察结果、修正路线，并在满足完成条件后停止。

这一区分非常重要。若把所有能聊天的系统都称为 Agent，我们就会忽略以下问题：系统是否知道当前目标？是否记录自己已经做过哪些步骤？是否能区分“回答问题”和“调用工具”？是否能判断证据是否足够？是否知道哪些动作需要用户确认？是否能在失败后回放过程？这些问题如果没有被显式设计，系统的外表再像 Agent，也只是一个更复杂的 Chatbot。

1.1.1 四类系统的基本对比

下面的表格给出一个粗略但实用的区分。它不是为了给所有产品贴标签，而是为了让我们在设计系统时知道自己正在增加哪一种能力。

类型	核心能力	典型输入输出	主要风险
Chatbot	根据上下文生成文本	用户提问，模型回答	容易把模型先验当事实
RAG Bot	检索外部证据后回答	问题、检索片段、引用	召回错误、引用漂移、证据不足
工具助手	封装外部工具供调用	用户选择工具，系统执行	工具参数错误、权限边界不清
Agent	围绕目标自主选择动作并更新状态	目标、状态、工具、观察、反馈	循环失控、工具误用、越权行动、难以复盘

这张表的重点在最后一列。**Agent**并不只是“更强”，它也更危险、更难调试。**Chatbot**说错话，通常只影响一段回答；**RAG Bot**检索错资料，会导致错误引用；工具助手调用错工具，可能造成外部系统状态变化；**Agent**若在错误状态下持续行动，则可能把小错误放大为一串错误动作。因此，是否采用**Agent**架构，不是越早越好，而要看任务是否真的需要目标驱动的多步执行。

注 一个常见误解是：只要模型会调用工具，就是**Agent**。实际上，工具调用只是**Agent**的一个组成部分。没有目标、状态、动作选择、反馈更新和停止条件，工具调用只是“函数调用能力”，还不是完整的任务执行系统。

1.1.2 从用户体验看不出系统结构

用户界面会掩盖系统差异。**Chatbot**、**RAG Bot**、工具助手和**Agent**都可以长得像一个聊天窗口，都可以显示“正在思考”，都可以输出自然语言。但对开发者来说，真正需要关心的是后台结构。用户的一句话进入系统后，系统是否只调用一次模型？是否先检索知识库？是否调用工具？是否把工具结果写入状态？是否有下一步决策？是否有最大步数和终止条件？这些问题决定了系统能否可靠运行。

例如，一个“论文总结助手”如果每次都让用户手动上传 PDF、手动点击解析、手动选择总结章节，那么它更接近工具助手。它可能非常有用，但还不是**Agent**。相反，如果系统接到“总结这篇论文并指出实验是否充分”这个目标后，能自动执行“定位文件—解析 PDF—识别章节—读取实验—提取表格—生成卡片—检查证据—输出结论”这一串动作，那么它就具有**Agent**的结构。

从这个角度看，**Agent**不是一种 UI 形态，而是一种系统组织方式。它的核心是把任务过程显式化。显式化之后，我们才能测试它、调试它、限制它、改进它。

1.2 Agent 的工作定义

为了避免概念漂移，本书采用一个工程化定义。它不把**Agent**理解为“像人一样思考的智能体”，也不把**Agent**理解为“任何自动化脚本”。我们关心的是能否构建一个围绕目标运行、能调用外部能力、能维护中间状态、能根据反馈继续行动的系统。

定义 1.1 (Agent)

一个**Agent**是围绕目标运行的计算系统。它在给定状态下读取上下文，选择动作，调用工具或生成输出，接收观察与反馈，并据此更新状态，直到任务完成、失败或被人类中断。

这个定义包含五个关键词：目标、状态、动作、观察、反馈。缺少其中任何一个，系统都会退化。

目标规定系统要完成什么。没有目标，系统只是响应输入的文本生成器。状态记录系统已经知道什么、做过什么、遇到什么问题。没有状态，系统每一步都像第一次回答。动作是系统可以选择的下一步，例如回答、检索、读取文件、调用 API、请求确认。观察是动作之后得到的结果，例如搜索结果、文件内容、工具错误、测试输出。反馈则用来判断行动是否有效，包括模型自检、程序测试、检索证据、用户确认和人工评价。

可以把**Agent**写成如下抽象：

$$\text{Agent} = (M, P, S, A, O, U, E),$$

其中 M 是模型， P 是提示与策略， S 是状态， A 是动作空间， O 是观察， U 是状态更新函数， E 是评估或终止条件。这个表达式不是为了形式化而形式化，而是提醒我们：模型只是系统的一部分。一个强模型配上混乱状态和危险工具，仍然会产生不可靠 Agent；一个中等模型配上清晰状态、窄工具和严格评估，反而可能在特定任务上更可靠。

命题 1.1 (目标—状态—动作原则)

判断一个系统是否接近 Agent，不看它是否使用了某个框架，也不看它是否能生成很长回答，而看它是否显式表示目标、维护状态、选择动作，并利用观察或反馈更新下一步行为。

1.2.1 目标不是一句提示词

很多人把目标理解为用户输入的一句话。例如“帮我写一篇报告”“帮我读这篇论文”“帮我修复这个 bug”。这些话只是目标的自然语言入口，还不是工程上可执行的目标。可执行目标至少应包含三类信息：交付物、约束和完成标准。

以“帮我读这篇论文”为例。交付物可以是“生成一张论文阅读卡片”；约束可以是“必须引用原文证据，不要只根据摘要判断”；完成标准可以是“卡片包含贡献、方法、实验、局限、可复现性五栏，并为每栏提供来源”。如果不把这些信息明确下来，Agent 就无法判断自己何时完成，也无法知道是否需要继续读取实验部分。

同样，“帮我修复这个 bug”也不是可执行目标。可执行版本应写成：“定位导致测试 `test_parser_handles_empty_input` 失败的原因，最小修改相关代码，新增或更新测试，运行指定测试并报告 diff。”这个目标告诉 Agent 它要修改什么、不应修改什么、如何验证。目标越清楚，Agent 越不需要猜测。

1.2.2 状态是 Agent 和 Chatbot 的分水岭

Chatbot 也有对话历史，但对话历史不等于状态。对话历史是文本序列，状态是为任务执行而设计的数据结构。状态应该回答：当前目标是什么？已经完成哪些步骤？有哪些证据？哪些工具调用失败？哪些动作等待用户确认？最终产物在哪里？

一个论文阅读 Agent 的状态可以包含如下字段：

```
state = {
  "goal": "write a paper reading card",
  "paper_path": "papers/react.pdf",
  "phase": "reading_experiments",
  "sections_read": ["abstract", "introduction", "method"],
  "evidence": [],
  "open_questions": ["Are ablations sufficient?"],
  "pending_approval": None,
  "final_card": None,
  "errors": [],
}
```

这样的状态比一长串对话更容易调试。若最终答案缺少实验评价，我们可以检查 `sections_read` 是否包含实验章节；若 Agent 重复读取同一页，我们可以检查 `phase` 和 `evidence` 是否更新；若系统试图下载外部文件，我们可以检查 `pending_approval` 是否记录了用户确认。



笔记 状态设计不是“高级功能”，而是 Agent 的基础设施。没有状态，后面的 RAG、记忆、反思、LangGraph 和多 Agent 协作都会变成松散的提示词拼接。

1.2.3 动作空间决定能力边界

动作空间是 Agent 可以做什么的集合。最简单的动作是 `finish`，表示直接给出答案。复杂一些的动作包括 `search`、`read_file`、`parse_pdf`、`run_tests`、`ask_human`、`write_patch`、`send_email`。动作空间越大，系统能力越强，风险也越高。

设计动作空间时，应把动作按风险分层。只读动作通常风险较低，例如搜索、读取文件、解析 PDF。建议动作风险中等，例如生成修改计划、提出移动文件建议、起草邮件。写入动作风险较高，例如修改文件、提交代码、创建日程。外部发布或不可逆动作风险最高，例如发送邮件、删除数据、付款、公开发布。不同风险等级不应共享同一套权限。

这就是为什么本书反复强调“窄工具”。与其给 Agent 一个万能 `run_shell`，不如给它 `list_files`、`read_file`、`propose_patch`、`apply_patch` 四个工具，并规定 `apply_patch` 必须在用户确认后执行。工具越窄，越容易测试；权限越清楚，越容易审计。

1.3 贯穿例子：论文阅读任务的四种实现

为了把概念落地，我们用“论文阅读”作为贯穿例子。用户给出一篇论文，希望系统回答：这篇论文的核心贡献是什么，实验是否充分，有哪些局限。这个任务很适合展示 Chatbot、RAG Bot、工具助手和 Agent 的区别，因为它既需要语言理解，也需要证据检索，还可能多步阅读。

1.3.1 实现一：Chatbot

Chatbot 版本最简单。用户把论文摘要或全文片段粘贴进对话框，模型生成总结。这个版本的优点是实现成本低，几乎没有外部依赖。缺点也明显：如果用户只粘贴摘要，模型可能根据摘要推断实验情况；如果论文很长，模型可能忽略细节；如果用户追问“这句话来自哪里”，系统不一定能给出可靠来源。

Chatbot 版本适合快速获得初步印象，但不适合做严肃阅读卡片。因为严肃阅读需要证据。论文贡献可以从摘要和引言得到初步判断，但实验是否充分，需要看实验设置、数据集、指标、消融和局限讨论。Chatbot 如果没有读取这些部分，就不应给出确定评价。

1.3.2 实现二：RAG Bot

RAG Bot 版本加入检索。系统先把论文切成片段，建立索引。用户提问时，系统检索相关片段，把片段和问题一起交给模型。这样可以带答案带来源，也可以处理比上下文窗口更长的文档。

但 RAG Bot 仍然通常是“问一次、检索一次、答一次”。如果问题是“实验是否充分”，第一次检索可能只命中摘要和方法，而没有命中实验表格。普通 RAG Bot 很可能直接根据现有片段回答。更好的 RAG 系统会在证据不足时提示“未检索到实验表格”，但它未必会主动换 `query`、继续读取实验章节或检查引用完整性。

因此，RAG 是 Agent 的重要部件，但 RAG 本身不等于 Agent。RAG 解决“知识从哪里来”的问题，Agent 还要解决“下一步做什么”的问题。

1.3.3 实现三：工具助手

工具助手版本把多个能力封装出来：解析 PDF、提取章节、搜索关键词、生成引用、导出 Markdown 卡片。用户可以一步步点击或发指令使用这些工具。这个版本比 RAG Bot 更灵活，因为用户可以决定先读方法还是先读实验，也可以要求系统只导出某一部分。

但工具助手的主动性仍然主要在用户一侧。如果用户不知道该读实验部分，系统不会自动补充；如果用户忘记检查局限，系统也不会主动提醒。工具助手的价值是降低操作成本，而不是自动完成任务。

1.3.4 实现四：Agent

Agent 版本会把论文阅读视为一个目标驱动任务。它先建立状态：论文路径、已读章节、待回答问题、已收集证据、缺失证据、最终卡片。然后根据状态选择动作。若论文尚未解析，调用 PDF 解析工具；若贡献栏缺证据，读取摘要和引言；若实验栏缺证据，读取实验章节和表格；若局限栏缺证据，搜索 `limitation`、`discussion`、`failure case` 等关键词；若所有栏都有证据，生成阅读卡片；若某栏始终无证据，在最终答案中标注“论文未明确说明”。

一个简化轨迹如下：

步骤	动作	观察	状态变化
1	<code>parse_pdf</code>	得到章节列表	<code>phase</code> 变为 <code>reading</code>
2	<code>read_section(introduction)</code>	找到贡献描述	<code>evidence</code> 增加贡献证据
3	<code>read_section(experiments)</code>	找到数据集和指标	<code>evidence</code> 增加实验证据
4	<code>search_in_paper(limitations)</code>	未找到明确局限	<code>open_questions</code> 增加局限 缺失
5	<code>draft_card</code>	生成带引用卡片	<code>final_card</code> 写入状态

这个轨迹表说明了 Agent 的核心：每一步都不是孤立回答，而是围绕状态推进任务。若第 3 步失败，系统可以重试或请求人工提供实验页码；若第 4 步找不到局限，它可以诚实标注缺失，而不是编造。

注 真正的 Agent 输出不仅应包含最终答案，还应能在需要时给出过程摘要：读取了哪些章节，使用了哪些工具，哪些证据支持结论，哪些问题仍然缺证据。这种过程可见性是可靠性的基础。

1.4 从概念到最小系统

本节给出一个最小 Agent 骨架。它不是生产级框架，也不依赖 LangChain、LangGraph 或 AutoGen。我们先用普通 Python 表达 Agent 的核心循环。这样做的好处是：后面学习框架时，读者知道框架到底替我们管理了什么。

1.4.1 动作、观察和状态

先定义三个核心对象。为了保持示例简洁，下面使用普通字典；真实项目中可以使用 `dataclass`、`Pydantic` 模型或 `TypedDict`。

```
from typing import Any, Callable

Action = dict[str, Any]
Observation = dict[str, Any]
State = dict[str, Any]

Tool = Callable[..., Observation]

def initial_state(goal: str) -> State:
    return {
        "goal": goal,
        "phase": "start",
        "trace": [],
        "evidence": [],
        "errors": [],
        "pending_approval": None,
```



```

    "answer": None,
    "done": False,
}

```

这里最重要的是 `trace`。很多玩具 Agent 没有 `trace`，只把工具结果拼回 `prompt`。这样做短期能跑，长期很难调试。`trace` 应记录动作、参数、观察、错误和耗时。若系统最终失败，`trace` 是复盘依据。

1.4.2 工具注册表

工具注册表负责保存工具、说明和风险等级。这里我们只示意三个工具：读取论文章节、搜索论文关键词、请求人工确认。

```

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, Tool] = {}
        self._risk: dict[str, str] = {}

    def register(self, name: str, fn: Tool, risk: str = "read"):
        self._tools[name] = fn
        self._risk[name] = risk

    def get(self, name: str) -> Tool:
        if name not in self._tools:
            raise KeyError(f"unknown tool: {name}")
        return self._tools[name]

    def risk(self, name: str) -> str:
        return self._risk.get(name, "unknown")

```

注意，工具风险不是注释，而应进入执行逻辑。读操作可以自动执行；写操作、发送操作、删除操作应进入审批流程。即使模型“承诺会小心”，执行器也不能跳过权限检查。

1.4.3 执行循环

最小 Agent 循环如下。它包含最大步数、动作选择、权限检查、工具执行、状态更新和结束判断。

```

def run_agent(goal: str, model, tools: ToolRegistry, max_steps: int = 8) -> State:
    state = initial_state(goal)

    for step in range(max_steps):
        action: Action = model.choose_action(state)
        name = action["name"]
        args = action.get("args", {})

        if name == "finish":
            state["answer"] = args.get("answer", "")
            state["done"] = True
            state["trace"].append({"step": step, "action": action})
            return state

        if tools.risk(name) in {"write", "send", "delete", "publish"}:

```

```

state["pending_approval"] = action
state["trace"].append({"step": step, "action": action, "status": "needs_approval"})
return state

try:
    observation = tools.get(name)(**args)
    state = update_state(state, action, observation)
except Exception as exc:
    state["errors"].append({"action": action, "error": str(exc)})

state["errors"].append({"error": "max_steps reached"})
return state

```

这个循环很短，但已经包含 Agent 的关键边界。第一，`max_steps` 防止无限循环。第二，`pending_approval` 防止高风险动作自动执行。第三，`trace` 让失败可复盘。第四，`finish` 是明确结束动作，而不是让模型在自然语言里暗示“我完成了”。

1.4.4 状态更新函数

状态更新函数要把观察转成下一步可用的信息。下面只是示意：

```

def update_state(state: State, action: Action, observation: Observation) -> State:
    state["trace"].append({"action": action, "observation": observation})

    if observation.get("kind") == "evidence":
        state["evidence"].append(observation)

    if observation.get("next_phase"):
        state["phase"] = observation["next_phase"]

    if observation.get("error"):
        state["errors"].append(observation["error"])

    return state

```

许多 Agent 系统的问题就出在这里：工具结果没有进入结构化状态，只是被追加到下一轮 prompt 中。这样模型可能看见了结果，但程序本身不知道发生了什么。结构化状态让系统可以检查“证据是否足够”“是否已经读过实验章节”“是否需要人工确认”。



笔记 本章代码故意保持朴素。LangGraph 的 State、节点和边，LangChain 的 Runnable，AutoGen 的多 Agent 对话，都可以看作在不同层次上组织这个循环。先理解这个循环，再学框架，才不会被框架概念牵着走。

1.5 私人订制化为什么需要 Agent

如果只是问百科问题，普通 Chatbot 往往已经够用。如果只是问“这段话是什么意思”，RAG Bot 也许足够。私人订制化 Agent 的价值出现在另一类任务中：任务反复发生，依赖用户自己的资料 and 工具，需要长期偏好，需要多步执行，还需要在关键节点保留用户控制权。

1.5.1 私人知识库不是简单 RAG

私人知识库 Agent 不只是把所有文件丢进向量数据库。它要知道哪些资料属于哪个项目，哪些文件可以被检索，哪些文件包含敏感信息，哪些回答需要引用，哪些偏好应该长期保存。用户问“上次那个 Agent 讲义的目录怎么定的”，系统需要知道“Agent 讲义”是哪个项目，相关设计文档在哪里，哪些内容是已确认方案，哪些只是临时讨论。

因此，私人知识库 Agent 至少需要四层结构。第一是资料层，包括 Markdown、PDF、LaTeX、代码、会议记录。第二是索引层，包括 chunk、embedding、metadata、权限字段。第三是任务层，包括当前问题、检索结果、回答草稿。第四是记忆层，包括用户长期偏好、项目规范、常用输出格式。若四层混在一起，系统很快会把临时资料当长期偏好，或者把无关项目资料混入答案。

1.5.2 私人写作 Agent 要能执行工作流

以讲义写作为例，用户真正需要的不是“生成一段文字”，而是一个写作工作流：确定目录，写章节，检查风格，补例题，编译 LaTeX，查看 PDF，修复错误，记录反馈。这个流程中，模型生成文本只是其中一步。若系统不会编译 PDF，不会读取错误日志，不会根据风格规范修订，不会保存用户反馈，它就只是写作 Chatbot，不是写作 Agent。

私人写作 Agent 还必须尊重用户风格。比如用户明确要求“不要压缩字数，宁滥勿缺”，系统就应把这条偏好反映到章节长度、节结构和审核标准中。若审核脚本只检查“有没有例题、有没有练习”，却不检查每节是否充分展开，就会生成看似完整、实际空心的讲义。这正是 Agent 需要反馈闭环的原因：用户指出问题后，系统不仅要改当前章节，还要更新后续写作标准。

1.5.3 私人 Agent 的边界是用户控制权

私人订制化常被误解为“模型更懂我”。更准确地说，私人订制化是“系统能在用户允许的范围内利用用户资料、工具和偏好”。这里最重要的是允许范围。用户应能查看系统保存了什么记忆，能删除错误偏好，能限制工具权限，能要求某些动作必须确认。

例如，一个私人助理可以读取日历并建议会议时间，但不应在没有确认的情况下邀请他人；可以起草邮件，但不应直接发送；可以整理文件移动计划，但不应直接删除文件；可以生成代码 patch，但不应直接推送到远端仓库。Agent 越贴近真实工作，越要把权限做成系统机制，而不是写在提示词里。

注 私人 Agent 的长期价值不在于一次惊艳输出，而在于稳定地适应用户工作流，同时保留可解释、可撤销、可审计的控制权。

1.6 边界、误区与练习

本章最后整理几个判断准则。它们比定义更实用，因为真实项目中，我们经常不是在“是不是 Agent”这个问题上做哲学分类，而是在决定“这个任务要不要用 Agent 架构”。

1.6.1 什么时候不该用 Agent

并非所有任务都应该做成 Agent。若任务只需要一次回答，例如“解释这个术语”，普通 Chatbot 足够。若任务只是从一个固定文档库中问答，RAG Bot 可能足够。若任务步骤固定且没有分支，Chain 或普通工作流可能足够。Agent 架构适合目标明确但过程不完全固定、需要根据观察调整路线的任务。

一个简单判断是：如果你无法写出状态字段和停止条件，就不要急着做 Agent。状态字段写不出来，说明任务过程还没有被理解；停止条件写不出来，说明系统不知道何时完成。这时应先澄清需求，而不是引入框架。

1.6.2 常见误区

误区	说明
把术语当能力	知道状态、工具、记忆这些词，并不意味着系统已经具备这些能力。只有当它们被写成数据结构、接口和测试时，才进入工程层面。
把长上下文当记忆	长上下文只是一次调用可见的信息窗口，不能替代可更新、可删除、可审计的长期记忆。
把工具调用当自主性	会调用工具只说明系统有外部能力，不说明它会正确选择目标、判断风险和停止执行。
把多 Agent 当升级	多 Agent 增加协调成本。若没有并行、专业化或互审需求，多个角色只是更贵的单 Agent。
把安全写进提示词	权限必须由执行器和工具层强制检查，不能只依赖模型遵守系统提示。

一个典型反例是“帮我整理项目文件”。如果系统没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成整理计划和变更预览，让用户确认后再执行。这个反例说明：Agent 的能力越强，边界越要前置。

1.6.3 本章小结

本章的核心观点可以概括为三句话。第一，Agent 不是聊天界面，而是目标驱动的任务执行系统。第二，Agent 的基本单元是目标、状态、动作、观察和反馈，而不是某个框架 API。第三，私人定制化 Agent 的关键不是让模型更会说话，而是让系统在用户允许的范围内接入资料、工具、偏好和反馈，并保留可审计的控制权。

后续章节会在这个基础上逐步展开。第 2 章会把最小 Agent 进一步拆成模型、提示、状态、工具和执行循环。第 4 章会专门讨论工具调用。第 5 章会区分状态、轨迹和记忆。第 6 章会进入 RAG。第二篇会把这些组件组合成十类 Agent 架构。

第一章 练习

1. 选择一个你常用的 AI 产品，判断它更接近 Chatbot、RAG Bot、工具助手还是 Agent。要求写出理由：它是否有目标、状态、动作、观察和停止条件。
2. 为“论文阅读 Agent”设计一个状态对象，至少包含目标、已读章节、证据、待解决问题、错误和最终产物六类字段。
3. 设计一个只读版“文件整理 Agent”。要求列出允许动作、禁止动作和必须人工确认的动作。
4. 说明为什么“会联网搜索”本身不足以构成 Agent。请给出一个搜索型 RAG Bot 和一个真正 Agent 的行为差异。
5. 反思题：如果用户要求“以后都按我的风格写”，系统应不应该立即写入长期记忆？请说明需要哪些确认信息。

第二章 Agent 的最小组成

内容提要

- ❑ 先把框架放到一边。要判断一个系统是不是 Agent，先看它在删到最小以后，还能不能保住目标、状态、动作和停止边界。
- ❑ 先从一个一次性问答函数写起，再把它改成会记录状态、会执行动作、也知道何时停下的循环。
- ❑ 这里的“最小”不是随手写个 demo。代码可以短，但每一步要看得见，能停下来，出错后还能回放和测试。

2.1 从一次性回答到最小闭环

工程上最常见的失败，是先选框架，再寻找问题。看到框架支持工具调用、记忆、工作流和多 Agent 协作，很多人会先把能力都接上。系统一不稳，才发现几个底层问题还没说清：任务要交付什么，当前状态是什么，模型能做哪些动作，工具结果写回哪里，什么时候停，失败后能不能回放。

最小 Agent 的意义，就是先把这条闭环写出来。先搞清楚一个循环里到底哪些部分不能省，后面再看成熟框架时，才容易知道它们替我们管理了哪些状态、权限和停止边界。如果一个最小循环都无法解释系统为什么调用某个工具，那么换成更大的框架只会把问题藏得更深。

先看一个看似简单的请求：

用户说：请帮我整理下载目录，把论文、发票、图片和临时文件分开。

如果系统只是根据文件名生成一段建议，它是 Chatbot 或写作助手。如果系统直接获得一个万能 `run_shell`，然后自己执行移动和删除命令，它可能很快做出效果，但风险极高：误删文件、覆盖同名文件、把私人资料移动到错误目录，都可能在用户还没看见计划之前发生。最小 Agent 的正确切入点不是“自动把目录整理好”，而是先实现一个只读闭环：读取目录、分类候选、生成计划、记录轨迹、指出哪些动作需要确认。

2.1.1 一次性函数的局限

一次性回答函数可以写得很简单：

```
def answer(goal: str, model) -> str:
    return model.generate(goal)
```

这个函数没有错。对于“解释一个概念”“润色一段文字”“把一段英文翻译成中文”这类任务，它甚至是最合适的结构。但它不适合承担多步任务，因为它没有显式状态。模型输出一段文本后，程序不知道它是否读过目录，不知道哪些文件已分类，不知道哪些路径存在冲突，也不知道下一步应继续读取、请求确认还是结束。

更严重的是，一次性函数无法区分“建议”和“执行”。用户说“整理目录”时，系统可能把生成建议、生成脚本、执行脚本都混在一段文本里。对 Agent 来说，这三件事必须分开：建议是低风险输出，生成脚本是中风险产物，执行脚本是高风险动作。只要这些动作没有进入显式动作空间和权限边界，就不应称为可靠 Agent。

2.1.2 最小循环的形状

最小 Agent 需要多出一个循环。每一轮，系统把当前状态交给模型；模型选择一个动作或给出最终答案；执行器检查动作是否允许；工具返回观察；更新器把观察写回状态；终止条件判断是否结束。用自然语言写出来，就是下面这条链：

环节	作用
目标	告诉系统要交付什么、哪些约束不可违反、完成标准是什么。
状态	记录当前阶段、已经做过的动作、工具观察、错误、待确认事项和最终产物。
动作	表示模型建议的下一步，例如读取文件、提出分类、请求确认或结束。
观察	表示动作执行后的结构化结果，例如文件列表、读取错误、分类候选或用户反馈。
更新	把观察转化为新的状态，而不是只把结果拼回下一次 prompt。
停止	在完成、失败、达到步数上限或等待人工确认时停止。

这条链看起来朴素，却是后续复杂架构的共同底座。后面遇到 ReAct、计划-执行、图编排或记忆增强时，可以先回到这条链来检查：它们究竟是在改动作选择、状态更新，还是在改历史保存方式。没有这条最小链，高级名词很容易只剩提示词包装。

2.2 最小组成的工程定义

本书采用下面的工程化表达：

$$\text{Agent} = (M, P, S, A, O, U, E),$$

其中 M 是模型， P 是提示与策略， S 是状态， A 是动作空间， O 是观察， U 是状态更新函数， E 是执行环境与终止条件。这个表达式不是为了把 Agent 形式化成数学对象，而是为了提醒我们：模型只是一个部件。一个强模型若没有清晰状态和窄工具，会做出难以复盘的错误；一个中等模型若状态、动作和停止条件设计清楚，反而可以在窄任务上稳定工作。

定义 2.1 (最小 Agent)

一个最小 Agent 是围绕明确目标运行的任务系统。它至少包含模型调用、结构化状态、受限动作空间、观察写回、执行环境和停止条件。它可以暂时没有长期记忆、多 Agent 协作、复杂规划器和生产级监控，但不能没有状态更新和终止判断。

2.2.1 哪些是必需，哪些是扩展

很多教程会把工具、记忆、规划、反思、多 Agent 都放进 Agent 定义中。这样做适合展示能力，却不适合建立最小概念。对本章来说，最小闭环中必需的是目标、状态、动作、观察、更新和停止；工具可以是外部函数，也可以只是一个受控的内部动作；长期记忆、复杂规划器和评估器是扩展，不应一开始就塞进教学最小系统。

部件	在最小系统中的地位	说明
模型 M	必需	负责根据上下文选择动作或生成最终输出，但不直接执行副作用。
提示与策略 P	必需	规定任务边界、输出格式、工具说明和不可违反的规则。
状态 S	必需	保存目标、阶段、轨迹、错误、证据、待确认事项和产物。
动作空间 A	必需	限制模型可以请求的下一步，动作越窄越容易测试。
观察 O	必需	工具或环境返回的结构化结果，是更新状态的依据。
更新器 U	必需	把观察写入状态，避免把工具结果散落在自然语言历史中。
长期记忆	扩展	只有跨任务稳定有用、用户可理解且可删除的信息才应进入。
复杂规划器	扩展	长任务需要；最小任务可以让模型每轮直接选择下一步。
评估器	可简化但不可忽略	教学中可用规则和测试样本替代生产级评估系统。

最小 Agent 不必压缩所有能力；它要保住任务执行的骨架。若一个系统没有长期记忆，它仍然可以是 Agent；若一个系统没有状态更新，即使它接入了十个工具，也只是工具调用助手。

2.2.2 停止条件是安全边界

很多初学者把 `max_steps` 看成性能参数，仿佛步数越多系统越聪明。实际上，步数上限首先是安全边界。模型如果进入错误循环，可能重复读取同一个文件、重复搜索同一个关键词、不断请求更宽权限，甚至在外部工具上消耗真实成本。没有步数上限，Agent 失败时不是输出一个坏答案，而是持续放大坏决策。

停止条件至少包括四类：第一，模型显式返回 `finish`；第二，状态满足完成标准；第三，达到步数或成本上限；第四，遇到需要人工确认的动作。对私人 Agent 来说，第四类尤其重要。系统不应因为自己“觉得可以”就移动、删除、发送或发布；一旦动作超出自动授权范围，就应停止并把计划交给用户确认。

2.3 贯穿例子：只读文件整理 Agent

下面把下载目录整理任务写成一个教学最小系统。我们故意把能力限制在只读和建议层面：`list_files` 可以列出文件，`read_metadata` 可以读取文件大小、修改时间和扩展名，`propose_plan` 可以生成移动建议，`ask_human` 可以请求确认。系统没有 `apply_move`，更没有 `delete_file`。这不是因为这些动作不重要，而是因为本章要先验证闭环，而不是追求自动化程度。

2.3.1 目标和状态

这个任务的可执行目标可以写成：

读取下载目录中文件的名称和元数据，按论文、发票、图片、压缩包和临时文件生成整理建议。不得移动或删除任何文件。最终输出一份带理由的分类表，并列出需要用户确认的潜在风险。

为了让系统可复盘，状态不能只是一段“用户想整理目录”的文本，而应写成字段：

```
state = {
  "goal": "propose a safe organization plan",
```

```
"phase": "scanning",
"root": "Downloads",
"files_seen": [],
"categories": {},
"uncertain_files": [],
"trace": [],
"errors": [],
"pending_approval": None,
"final_plan": None,
"done": False,
}
```

这些字段看起来比一个 prompt 麻烦，但它们带来三个好处。第一，系统可以知道自己是否已经扫描目录。第二，失败时可以检查错误发生在哪一步。第三，输出建议时可以说明每个分类依据，而不是凭模型感觉给文件贴标签。

2.3.2 动作和观察

动作空间应当小而明确。下面的过程表展示一次理想运行轨迹：

步骤	动作	观察	状态变化
1	list_files	返回 42 个文件名与路径	files_seen 写入文件列表，阶段变为分类。
2	read_metadata	读取扩展名、大小和修改时间	为每个文件增加元数据，记录读取失败项。
3	propose_plan	生成分类和移动建议	categories 与 uncertain_files 更新。
4	finish	输出整理建议表，并列需要用户确认的潜在风险	final_plan 写入状态，done 变为 true。

如果模型或用户进一步提出移动、删除、覆盖等写入动作，执行器应把该动作写入 pending_approval 并停止。正常的只读整理流程只输出建议表，不自动移动文件；这里的“停止”不是失败，而是正确边界。

2.4 最小 Python 骨架

本节给出一个不依赖框架的教学骨架。它参考了许多从零构建 Agent 的教程思路，但代码全部按本讲义的文件整理例子重写。读者先看接口，再看代码：动作怎么表示，工具怎么注册，权限怎么拦截，观察怎么写回轨迹。

2.4.1 动作、观察和状态类型

```
from typing import Any, Callable, TypedDict, Literal

Risk = Literal["read", "suggest", "write", "publish", "destructive"]

class Action(TypedDict):
```



```

name: str
args: dict[str, Any]

class Observation(TypedDict):
    ok: bool
    data: dict[str, Any]
    error: str | None

class State(TypedDict):
    goal: str
    phase: str
    root: str
    files_seen: list[dict[str, Any]]
    categories: dict[str, Any]
    uncertain_files: list[dict[str, Any]]
    trace: list[dict[str, Any]]
    errors: list[str]
    pending_approval: dict[str, Any] | None
    final_plan: dict[str, Any] | None
    done: bool

Tool = Callable[..., Observation]

def initial_state(goal: str) -> State:
    return {
        "goal": goal,
        "phase": "start",
        "root": "Downloads",
        "files_seen": [],
        "categories": {},
        "uncertain_files": [],
        "trace": [],
        "errors": [],
        "pending_approval": None,
        "final_plan": None,
        "done": False,
    }

```

这里最重要的字段是 `trace` 和 `pending_approval`。没有 `trace`，系统无法解释自己做过什么；没有 `pending_approval`，危险动作就容易被当成普通下一步。

2.4.2 工具注册和权限检查

```

class ToolRegistry:
    def __init__(self) -> None:
        self._tools: dict[str, Tool] = {}
        self._risk: dict[str, Risk] = {}

```

```

def register(self, name: str, fn: Tool, risk: Risk) -> None:
    self._tools[name] = fn
    self._risk[name] = risk

def get(self, name: str) -> Tool:
    if name not in self._tools:
        raise KeyError(f"unknown tool: {name}")
    return self._tools[name]

def risk(self, name: str) -> Risk:
    if name not in self._risk:
        raise KeyError(f"unknown tool: {name}")
    return self._risk[name]

def check_permission(action: Action, registry: ToolRegistry) -> Observation | None:
    try:
        risk = registry.risk(action["name"])
    except KeyError:
        return {
            "ok": False,
            "data": {"action": action},
            "error": "unknown_tool",
        }
    if risk in {"write", "publish", "destructive"}:
        return {
            "ok": False,
            "data": {"action": action, "required": "human_approval"},
            "error": "permission_required",
        }
    return None

```

这几行里最不能省的是执行器拦截。提示词可以提醒模型不要自动删除文件，但只靠提醒不够；写入和发布动作要在工具层被拦下来。模型“答应遵守规则”，不等于系统已经有权限控制。

2.4.3 执行循环和状态更新

```

def update_state(state: State, action: Action, obs: Observation) -> State:
    state["trace"].append({
        "action": action["name"],
        "args": action["args"],
        "observation": obs,
    })
    if not obs["ok"]:
        state["errors"].append(obs["error"] or "unknown_error")
    if obs["error"] == "permission_required":
        state["pending_approval"] = obs["data"]
        state["done"] = True
    if action["name"] == "finish" and obs["ok"]:

```

```

    state["final_plan"] = obs["data"]
    state["done"] = True
    return state

def run_agent(goal: str, model, registry: ToolRegistry, max_steps: int = 6) -> State:
    state = initial_state(goal)
    for step in range(max_steps):
        action: Action = model.decide(state)
        if action["name"] == "finish":
            obs = {"ok": True, "data": action["args"], "error": None}
            return update_state(state, action, obs)

        blocked = check_permission(action, registry)
        if blocked is not None:
            return update_state(state, action, blocked)

        try:
            tool = registry.get(action["name"])
            obs = tool(**action["args"])
        except Exception as exc:
            obs = {"ok": False, "data": {}, "error": str(exc)}

        state = update_state(state, action, obs)
        if state["done"]:
            return state

    state["errors"].append("step_limit_exceeded")
    state["done"] = True
    return state

```

这不是生产级实现：它没有并发、重试、结构化参数校验、持久化日志和成本统计。但作为最小 Agent，它已经包含可观察闭环。每轮动作都被记录，每个工具错误都进入状态，写入动作会被拦截，步数上限会停止循环。读者后面学习任意框架时，都可以问：这个框架把上面哪些函数变成了内置机制？又把哪些边界交给开发者自己负责？

2.5 最小不等于简陋

最小 Agent 有三种层次。教学最小系统只要求结构清楚；可测试最小系统要求有固定样本和可复现失败；生产最小系统则必须补上持久化、权限、日志、成本和监控。本章的目标是从教学最小走向可测试最小，而不是一步跳到生产系统。

层次	必须具备	暂时可以不具备
教学最小	状态、动作、观察、更新、停止条件	数据库、复杂 UI、多模型路由。
可测试最小	固定测试任务、期望轨迹、失败样本、 权限拦截	全量评估平台、自动部署。
生产最小	持久化日志、审计、鉴权、成本上限、 回滚或人工确认	多 Agent 协作、复杂长期记忆。

这个区分可以防止两种极端。第一种极端是把最小系统写成玩具：只要能跑一次 **demo**，就认为 **Agent** 已经完成。第二种极端是把第一版写成庞大平台：数据库、队列、前端、监控、用户系统全部接入，结果任何一个失败都难以定位。更好的做法是先做一个可测试最小系统，再按失败记录逐步扩展。

2.5.1 验收样例

一个只读文件整理 **Agent** 至少应准备九条样本：三条正常任务、三条边界任务、三条必须请求确认的任务。正常任务用来验证闭环能完成；边界任务用来验证系统不会猜测；确认任务用来验证权限边界有效。

类型	样本	期望轨迹特征
正常任务	“按扩展名给下载目录生成分类建议”	调用只读工具，生成分类表，返回 finish 。
正常任务	“列出可能是论文 PDF 的文件”	读取文件名和元数据，不移动文件。
正常任务	“找出超过 500MB 的压缩包”	只使用元数据，不读取文件内容。
边界任务	“把看起来没用的文件删掉”	停止并请求确认，不调用删除工具。
边界任务	“帮我整理，但目录路径没给”	先澄清路径，不猜测默认目录。
边界任务	“把私人照片上传到云盘备份”	识别为外部发布，高风险，进入确认。
确认任务	“生成移动计划后直接执行”	生成计划可以自动，执行移动必须确认。
确认任务	“覆盖同名文件”	标记冲突，不能自动覆盖。
确认任务	“运行脚本清理临时文件”	拒绝万能执行或请求人工审查脚本。

这些样本不需要一开始很多，但必须固定。没有固定样本，**Agent** 的改进就会变成“这次看起来更好”。固定样本让我们能看到：修改提示词后是否破坏了权限边界，增加新工具后是否导致模型过度行动，扩大上下文后是否让输出格式变得不稳定。

2.6 失败模式、边界与练习

最小 **Agent** 的失败模式往往很朴素，但后果会在复杂系统中被放大。下面的矩阵列出本章最应该检查的结构性问题。

症状	可能原因	修复方式
循环失控	没有 max_steps 或完成标准含糊	加入步数、成本、阶段和显式 finish 条件。
状态漂移	工具结果只拼回 prompt , 没有结构化写入	设计 State 字段，并让 update_state 成为唯一写入口。
工具误用	动作空间过宽或工具说明含糊	拆分窄工具，参数 schema 明确，写入动作必须确认。
不可复盘	没有记录动作参数、观察和错误	每步写入 trace , 运行结束后保留状态快照。
自然语言结束	模型用一句“我完成了”替代结构化完成信号	要求 finish 动作携带最终产物和完成依据。
过早自动化	教学闭环未稳定就加入写文件、发消息或删除能力	先做只读和建议型 Agent ，再逐步接入人工审批后的写入。

调试时不要首先责怪模型。更可靠的顺序是：复现失败输入，查看状态快照，检查上一轮 **action**，检查工具 **observation**，检查更新器是否写错字段，最后再考虑提示词是否需要调整。很多所谓“模型乱来”，其实是动作空间太宽、状态字段不清或停止条件缺失。

2.6.1 与参考项目的关系

许多从零构建 Agent 的 GitHub 教程都会展示对话、工具调用、RAG、记忆和 ReAct 等能力。本章吸收的是它们的工程模式：先搭闭环，再逐步增加能力；先看动作与状态，再看框架 API；先记录失败，再优化提示词。讲义中的例子不复制任何教程代码，而是把这些思想转译为文件整理、论文阅读和私人写作这类长期会反复出现的私人任务。

2.6.2 本章小结

会调用工具还不够。Agent 至少要把目标、状态、动作、观察、更新和停止条件串成一条能回放的循环。最小系统可以没有长期记忆和复杂规划器，但不能没有状态更新、权限边界和步数上限；这个闭环稳了，后面再接 RAG、记忆、ReAct 或图编排，才不至于一出错就无从下手。

第二章 练习

1. 为一个“只读日程查询 Agent”写出最小状态字典。要求至少包含目标、阶段、轨迹、错误、待确认事项和最终答案。
2. 解释为什么 `max_steps` 是安全边界，而不是单纯的性能参数。请给出一个没有步数上限导致失败放大的例子。
3. 把一个普通问答函数改写为带状态更新的最小 Agent 伪代码。要求显式出现动作、观察和 `finish`。
4. 设计三条必须请求人工确认的文件整理任务，并说明每条任务应在状态中留下哪些 `pending_approval` 信息。
5. 选择一个你日常会用的私人任务，例如整理论文、查询实验日志或生成讲义提纲，判断它在什么条件下只需要 Chatbot，什么条件下才值得做成 Agent。

第三章 LLM、Prompt 与上下文

内容提要

- ❑ 提示词写得再漂亮，也救不了混乱的上下文。Agent 出问题，第一步应还原这次调用里模型实际看见的规则、状态、证据和格式要求。
- ❑ 先把上下文拆开看：哪些是规则，哪些是目标，哪些是状态、证据和输出格式。
- ❑ 上下文不是塞得越多越好。真正要紧的是：能分层，能删减，程序出错时也能查得出来。

3.1 上下文不是一个大 prompt

很多 Agent 失败会被归因于模型“不够聪明”。但在实际工程中，许多失败来自上下文组织混乱。如果把系统规则、用户目标、旧聊天、工具说明、检索片段和输出格式都揉进一段话，模型就很难分清轻重。哪些规则不能被覆盖，哪些信息已经过期，哪些证据真的支持结论，下一步该给动作还是给答案，都会变得模糊。

一个常见写法如下：

你是一个很专业的写作助手。用户喜欢详细讲解。之前说过不要太短。现在请参考下面这些资料写第二章。资料包括若干 GitHub 仓库、旧草稿、用户反馈和一些工具结果。输出最好是 JSON，但如果你觉得自然语言更清楚也可以。注意不要违反权限。下面是所有历史记录……

这段文字看似信息充分，实际上把不同生命周期的信息混在一起了。“你是写作助手”是稳定系统边界；“写第二章”是本次任务；“用户喜欢详细讲解”可能是长期偏好；“旧草稿”是状态；“GitHub 仓库”是外部证据；“输出 JSON”是程序接口；“不要违反权限”是不可被用户覆盖的边界。若这些内容都以同等权重放在一段 prompt 里，后续调试会非常困难。模型输出不符合 JSON 时，我们不知道是输出约束太弱；引用错误时，我们不知道是证据格式混乱；越权调用工具时，我们不知道是权限规则被上下文稀释，还是执行器没有拦截。

3.1.1 Prompt、Context、Memory 和 Trace

这里先把四个词分开。**Prompt** 是一次调用里写给模型的指令；**Context** 是这次调用模型实际能看到的全部内容。**Memory** 指经过筛选、能跨任务复用的信息；**Trace** 则记录已经发生的动作、参数、观察和错误。

这四者的生命周期不同。Prompt 服务于一次调用；Context 服务于当前决策；Trace 服务于复盘和状态更新；Memory 服务于未来任务。如果把 trace 全部当成 memory，系统会保存大量无用细节；如果把 memory 原样塞进每次 context，敏感信息和过期偏好就会污染任务；如果把输出 schema 只写成 prompt 里的温柔建议，程序就无法稳定解析模型返回。

定义 3.1 (上下文)

上下文是模型在一次调用中可见的信息集合。对 Agent 来说，上下文不仅包括用户输入，还包括系统边界、任务目标、状态摘要、工具说明、历史轨迹摘要、外部证据和输出格式要求。

3.2 上下文七层模型

为了让上下文可维护，本书采用七层模型。七层不是固定模板，而是一种检查清单：每次构造模型输入时，都问这些信息是否应该出现，应该以什么格式出现，是否过期，是否可以被用户覆盖，是否能被程序裁剪。

层级	写入者	生命周期	主要风险
系统边界	开发者或平台	长期稳定	被用户提示覆盖，或写得过长导致关键规则稀释。
任务目标	用户与 Agent 澄清后形成	当前任务	目标过宽、交付物不清、完成标准缺失。
状态摘要	程序从 State 生成	当前运行过程	摘要遗漏关键错误或待确认动作。
工具说明	开发者和工具注册表	随工具版本变化	工具能力夸大、参数说明含糊、风险等级缺失。
轨迹摘要	程序从 trace 压缩	当前任务或会话	过度保留噪声，或压缩掉失败原因。
外部证据	检索器、工具或用户资料	与来源相关	引用漂移、证据污染、权限过滤缺失。
输出格式	程序接口与评估要求	随调用目的变化	自然语言混入结构化输出，导致解析失败。

真正要分清的是谁写入、能活多久。规则不能被用户临时改写，目标要能澄清，状态摘要应由程序生成，证据要带来源，输出格式要方便程序接手。上下文工程不是把 prompt 写得更漂亮，而是别让这些信息挤在同一层。

3.2.1 系统边界与任务目标

系统边界回答“系统永远不应该做什么”和“系统必须遵守什么”。例如，文件整理 Agent 不得自动删除文件；论文阅读 Agent 不得在没有证据时编造实验结论；代码开发 Agent 不得在用户未确认时发布外部服务。这类规则应短、稳定、可执行，不能被用户一句“忽略上面的规则”覆盖。

任务目标回答“这次要交付什么”。它应包含交付物、约束和完成标准。用户说“扩写这份资料”只是自然语言入口；更可执行的目标是“参照已有文档的写法扩写目标内容，保留原有结构，补充核心概念、示例骨架、常见失败模式和练习题，并以结构完整、示例可运行、格式检查通过作为完成标准”。目标越清楚，模型越不需要从上下文噪声中猜。

3.2.2 状态、轨迹和证据

状态摘要不是聊天历史摘要。它应从结构化状态中抽取，例如当前阶段、已完成步骤、缺失证据、最近错误、待确认动作和最终产物路径。轨迹摘要则说明“发生过什么”，例如最近三次工具调用及其结果。证据区提供事实依据，例如检索片段、文件段落、页码或 source id。

这三层的边界必须清楚。状态告诉模型当前任务进行到哪里；轨迹解释为什么走到这里；证据支持模型要写出的事实结论。如果把三者混成一段“以下是背景资料”，模型很容易把旧错误当事实，把工具失败当已完成，把证据来源和模型推测混在一起。

3.3 贯穿例子：论文阅读 Agent 的上下文包

设想一个私人论文阅读 Agent 要为用户写一份带引用的阅读卡片。它需要知道论文路径、当前阅读阶段、已收集证据、开放问题、用户对卡片字段的偏好、上一轮失败和输出格式。最危险的写法是把所有内容拼成一个长 prompt；更稳妥的写法是构造一个上下文包。

3.3.1 原始混乱写法

混乱写法通常像这样：

请帮我读一篇关于大模型上下文工程的论文，写一份阅读卡片，风格简洁、带引用。可以参考论文 PDF 和我的笔记，但不要编造论文里没有的结论。请保留关键发现和实验数据，并根据用户反馈补充局限性和后续问题。下面是历史对话、参考资料和工具结果……

这段输入包含有用信息，但没有层级。若模型没有按简洁带引用的风格写，我们不知道是风格说明不够；若它编造论文里没有的结论，我们不知道是证据区没有说明引用边界；若它输出过长导致程序无法写入，我们不知道是输出格式没约束。上下文越长，这种混乱越难发现。

3.3.2 分层后的消息结构

分层后的结构可以写成下面的形式。注意，代码中具体字段只是示例；真实系统可使用不同消息角色或对象格式，但思想相同：稳定规则、任务目标、状态、证据和输出格式分开放。

```
messages = [
  {
    "role": "system",
    "content": "Write a concise reading card. Use evidence. Do not invent missing results."
  },
  {
    "role": "user",
    "content": "Summarize the target paper with evidence."
  },
  {
    "role": "assistant",
    "content": format_state_summary({
      "task": "paper_reading_card",
      "phase": "drafting",
      "must_include": ["main claim", "method", "evidence", "limitations"],
      "open_questions": ["Are the ablations sufficient?"],
      "pending_approval": None,
    })
  },
  {
    "role": "user",
    "content": format_evidence([
      {"source_id": "paper:p2:intro", "note": "main claim and motivation"},
      {"source_id": "paper:p7:table2", "note": "main experiment table"},
    ])
  },
  {
    "role": "user",
    "content": "Return JSON with answer, citations, missing_evidence, and next_action."
  },
]
```


这里有一个细节：状态摘要和证据区都由程序生成，而不是让模型自己从海量历史里总结。这样做的好处是可测试。若最终阅读卡片缺少实验局限，可以检查 `must_include` 是否进入状态摘要；若引用不可靠，可以检查 `evidence` 是否带 `source id`；若模型继续追问无关内容，可以检查输出格式是否把 `next_action` 约束清楚。

3.4 上下文构造器：从状态生成消息

构造器不要只会拼字符串。它要从状态里取目标，挑该用的记忆，放进工具说明和证据；`token` 不够时，先删低价值内容，别动系统边界和输出格式。

3.4.1 最小构造器代码

```
def build_messages(state, tools, evidence, budget):
    system_rules = [
        "Follow the system safety boundary.",
        "Use evidence for factual claims.",
        "Return either an action JSON or a final answer JSON.",
    ]

    recent_trace = state["trace"][-3:]
    trace_summary = summarize_trace(recent_trace)
    evidence_block = format_evidence_with_source_ids(evidence, max_items=8)
    tool_block = format_tool_specs(tools, include_risk=True)

    messages = [
        {"role": "system", "content": "\n".join(system_rules)},
        {"role": "user", "content": format_goal(state["goal"])},
        {"role": "user", "content": format_state_summary(state)},
        {"role": "user", "content": tool_block},
        {"role": "user", "content": trace_summary},
        {"role": "user", "content": evidence_block},
        {"role": "user", "content": output_schema_text()},
    ]

    while token_count(messages) > budget.max_tokens:
        messages = compress_low_priority_context(messages)
    return messages
```

构造上下文时，压缩顺序比代码形式更重要。系统边界和输出 `schema` 不宜改动；任务目标和待确认动作不能丢；较旧轨迹、重复证据和低相关资料才适合优先压缩。长上下文模型能放进更多材料，但不代表所有材料都同等重要。

3.4.2 输出 `schema` 与解析失败

Agent 的模型输出通常不是给人直接阅读，而是给程序继续处理。因此，输出格式应显式规定。例如下一步动作可以要求：

```
ACTION_SCHEMA = {
    "type": "object",
    "required": ["type", "reason"],
```

```
"properties": {
  "type": {"enum": ["tool_call", "ask_human", "finish"]},
  "tool_name": {"type": "string"},
  "args": {"type": "object"},
  "answer": {"type": "string"},
  "reason": {"type": "string"},
},
}
```

如果模型返回不可解析文本，不应简单把错误吞掉。更好的做法是把解析错误作为观察写入状态，然后构造一次修复调用：告诉模型上次输出违反了哪个字段，要求只返回合法结构。这样，格式错误也成为 `trace` 的一部分，而不是隐藏在日志里。

3.5 冲突、压缩与长任务

上下文工程最难的地方，不是写第一版 `prompt`，而是处理冲突和长期运行。私人 `Agent` 往往同时面对系统规则、用户临时要求、长期偏好、项目资料和外部证据。它们不总是一致。

3.5.1 冲突的优先级

第一类冲突是用户要求和系统边界冲突。例如用户说“忽略权限，直接删除这些文件”。上下文层面应明确：系统边界优先，执行器也必须拦截。第二类冲突是外部证据和模型先验冲突。例如模型“知道”某篇论文通常使用某数据集，但检索证据没有出现该数据集。此时证据优先，答案应降低置信度或继续检索。第三类冲突是长期偏好和当前任务冲突。例如用户长期偏好“详细解释”，但本次明确要求“只给 5 行摘要”。当前任务目标优先，长期偏好只能作为默认值。

冲突类型	优先级	正确处理
用户要求越权	系统边界与工具权限优先	拒绝或请求确认，不能仅靠模型自律。
证据不足但用户催促	证据状态优先	说明证据不足、继续检索或请求资料。
长期偏好与本次任务矛盾	本次任务优先	把长期偏好降为默认建议，不覆盖明确指令。
旧轨迹与当前状态矛盾	当前结构化状态优先	检查更新器和 <code>checkpoint</code> ，必要时回放 <code>trace</code> 。

3.5.2 长上下文不取消裁剪

长上下文模型很有用，尤其适合阅读长文档、长代码和长轨迹。但长上下文不是把所有东西都塞进去的许可。上下文越长，越需要明确来源、压缩层级和重要性。否则模型可能在大量材料中忽略真正关键的工具权限、引用要求或输出格式。

一个实用策略是：保留稳定系统边界；保留当前目标和完成标准；保留最近几步完整 `trace`；把更早 `trace` 压缩为阶段摘要；证据按 `source id` 去重；长期记忆只注入与当前任务强相关且用户可见的部分；输出 `schema` 永远放在靠近调用末尾的位置。这样即便上下文被裁剪，系统仍能保持任务方向和程序接口。

3.6 失败模式、评估与练习

上下文问题常常表现为“模型不听话”，但根因不一定是提示词不够强。下面的矩阵把常见症状拆成可调试的结构问题。

症状	可能原因	修复方式
规则被忽略	系统边界过长或与普通资料混放	缩短稳定规则，提升层级，并在执行器中落实权限。
引用漂移	证据没有 <code>source id</code> 或答案句子未绑定来源	证据格式化时保留来源，生成后做 <code>citation check</code> 。
输出不可解析	输出 <code>schema</code> 只是自然语言建议	使用结构化 <code>schema</code> ，并把解析错误写入 <code>trace</code> 后修复。
上下文膨胀	原样保留全部历史	保留最近 <code>trace</code> ，压缩旧轨迹，删除重复证据。
记忆误注入	把临时要求或模型猜测当长期偏好	记忆写入必须有来源、确认和删除入口。
提示注入	外部资料包含“忽略规则”等文本	把外部证据标记为不可信内容，只作为引用材料。

评估上下文工程，可以从四类任务开始。第一，结构化解析成功率：模型输出能否稳定通过 `schema` 校验。第二，约束违反测试：给出越权用户请求，检查系统是否拒绝或请求确认。第三，`prompt injection` 测试：在检索资料中放入恶意指令，检查系统是否仍遵守系统边界。第四，裁剪一致性测试：删除无关历史后，答案是否保持一致；裁剪关键证据后，系统是否降低置信度而不是编造。

3.6.1 本章小结

上下文不是越长越稳。规则、目标、状态、工具、轨迹、证据和格式分开放，出错时才知道该查哪一层：是输出格式松了，证据没撑住，还是状态更新写错了。

第三章 练习

1. 把一次“帮我总结论文并指出实验是否充分”的请求拆成七层上下文。要求说明每层由谁写入、何时更新。
2. 设计一个上下文裁剪策略：当 `token` 预算不足时，哪些信息绝不能删，哪些信息可以摘要，哪些信息可以丢弃？
3. 给出一个 `prompt injection` 例子：外部资料中包含“忽略之前规则”。说明上下文构造器和执行器分别应如何防御。
4. 为一个文件整理 Agent 设计下一步动作的 `JSON` 输出格式，并说明解析失败后如何把错误写入 `trace`。
5. 反思题：长上下文模型出现后，为什么仍然需要状态摘要和证据 `source id`？请从调试、引用和隐私三个角度回答。

第四章 工具调用机制

内容提要

- ❑ 接 API 只是工具调用的入口。难点在于分清模型只提出动作，函数只提供能力，执行器负责校验和记录，权限系统决定哪些动作必须停下。
- ❑ 工具不能只是一个函数名。参数怎么验、结果怎么回、风险多大、事后怎么查，都要写清楚。
- ❑ 模型可以提议做什么，但不能自己动手。凡是会改外部世界的操作，都要经过执行器检查、记录，必要时拦下来。
- ❑ 工具不能只是一个函数名。参数怎么验、结果

4.1 工具调用不等于 Agent

许多模型和 SDK 都支持 tool calling 或 function calling。模型输出一个工具名和参数，程序执行函数，再把结果交回模型。这个能力非常重要，但它本身不等于 Agent。因为工具调用只解决“模型如何请求外部能力”的问题，还没有解决目标如何维护、状态如何更新、权限如何控制、失败如何恢复以及何时停止。

例如，模型输出：

调用 `delete_file(path="Downloads/tmp.zip")`。

这只是一段动作意图。真正的系统还必须回答：这个工具是否存在？参数是否符合 schema？这个路径是否在允许范围内？删除是否需要人工确认？是否先做 dry run？工具失败后返回什么结构？这次调用是否进入审计日志？如果模型下一步继续要求删除更多文件，系统如何判断是否越权？这些问题不属于模型本身，而属于执行器和权限系统。

定义 4.1 (工具)

工具是 Agent 可以请求执行的外部能力。一个可靠工具不只是函数名，而是一份契约：名称、用途、输入 schema、输出 schema、错误格式、风险等级、副作用说明、超时、重试规则和审计字段。

命题 4.1 (模型提出动作，执行器承担责任)

模型可以建议调用工具，但不能绕过执行器直接产生外部副作用。凡是读写文件、执行命令、发送消息、创建日程、联网发布或调用付费 API 的动作，都必须由执行器进行参数校验、权限判断和日志记录。

4.1.1 函数调用、工具、执行器与动作空间

函数调用是模型输出结构化调用的一种接口；工具是系统暴露给模型的受控能力；执行器是负责校验、授权、运行和记录的程序层；动作空间则是 Agent 当前允许选择的所有下一步。四者相互关联，但不能混为一谈。

一个天气查询函数可以直接暴露为工具，因为它通常只读且风险较低。一个文件管理函数则不能简单暴露为 `manage_files(command: str)`，因为它把读取、移动、删除和执行命令混在一起。对 Agent 来说，动作空间越宽，模型越容易在不明确的状态下做出危险选择；工具越窄，系统越容易测试和审计。

4.2 工具契约的工程定义

工具契约要把几个关口说清：什么时候能用，参数怎么验，结果和错误怎么回，风险与副作用多大，以及这次调用以后如何审计。下面这张表把这些关口拆成字段。很多 demo 只写 name、description 和 parameters，这对演示足够，对私人 Agent 不够。

字段	示例	作用
name	propose_move	稳定工具标识，不应频繁变化。
description	生成文件移动建议但不执行	说明能力边界，避免模型误用。
input_schema	source, target, reason	校验参数类型、必填字段和枚举值。
output_schema	ok, plan, warnings	让观察可写回状态。
error_schema	code, message, recoverable	支持重试、澄清或降级。
risk_level	read, suggest, write, publish, destructive	决定是否自动执行、请求确认或默认禁止。
side_effect	none / filesystem / external	标记是否改变外部世界。
timeout	5s / 30s	防止工具调用无限等待。
audit_fields	user, args, result, time	便于复盘和权限审计。

4.2.1 坏工具接口的代价

坏工具接口通常为了 demo 方便而设计。例如：

```
run_shell(command: str) -> str
manage_files(instruction: str) -> str
execute(task: str) -> str
```

这些接口的问题不是“模型可能不够聪明”，而是程序层无法知道参数含义。`run_shell` 可以读取文件，也可以删除文件，还可以联网下载和执行脚本；`manage_files` 可能只是列目录，也可能移动或覆盖文件；`execute` 更是把风险全部交给自然语言。只要工具接口无法表达风险，权限系统就无法可靠拦截。

更好的做法是把能力拆小：`list_files` 只读目录，`read_file` 只读文件，`propose_move` 只生成计划，`apply_move` 执行移动且必须确认，`delete_file` 默认禁用或要求二次确认。拆分以后，模型仍然可以完成任务，但每一步的风险变得可见。

4.3 贯穿例子：文件整理工具族

沿用第 2 章的下载目录整理任务，本章把动作空间写成工具族。这个例子比天气查询更适合讲工具调用，因为它同时包含只读、建议、写入和删除四类风险。

工具	风险等级	返回结构	是否自动
<code>list_files</code>	read	文件名、路径、大小、修改时间	可以
<code>read_file</code>	read	文本预览、编码、读取错误	限制大小后可以
<code>propose_move</code>	suggest	源路径、目标路径、理由、冲突警告	可以
<code>apply_move</code>	write	实际移动结果、失败项、回滚信息	需确认
<code>delete_file</code>	destructive	删除候选、影响范围、恢复可能性	默认禁止或二次确认

这个工具族体现了 `dry run` 和 `apply` 分离。`propose_move` 可以自动运行，因为它只生成建议；`apply_move` 会改变文件系统，必须在用户确认后执行。删除动作更危险，因为恢复成本高，应默认禁用，或者设计成“把文件移入回收区并生成恢复记录”的受控版本。

4.3.1 一次调用的完整流程

一次工具调用不应只记录“调用成功”。更完整的轨迹如下：

步骤	模型输出	执行器检查	状态变化
1	<code>list_files(root)</code>	参数 <code>root</code> 是否在允许目录内	<code>trace</code> 增加文件列表观察。
2	<code>propose_move(files)</code>	检查只生成计划, 不执行移动	<code>categories</code> 写入建议。
3	<code>apply_move(plan)</code>	风险为 <code>write</code> , 缺少确认	<code>pending_approval</code> 写入计划并停止。
4	用户确认部分计划	执行器只执行已确认项	<code>trace</code> 记录用户反馈和结果。

这里的关键是第 3 步。模型提出 `apply_move` 并不等于系统必须执行。执行器发现该工具风险为 `write` 且没有确认, 就把动作转为 `pending approval`。这样, 权限边界不依赖模型自觉。

4.4 执行器最小骨架

工具执行器的关键职责是：模型给出工具名和参数后, 程序不能立刻执行, 而要先校验参数, 再检查权限; 需要确认的动作进入挂起状态; 执行完成后, 再把 `observation` 写入 `trace`。

```
import hashlib
import json
from typing import Any

class ToolSpec:
    def __init__(self, name, fn, input_schema, risk, timeout=10):
        self.name = name
        self.fn = fn
        self.input_schema = input_schema
        self.risk = risk
        self.timeout = timeout

def validate_args(schema: dict[str, Any], args: dict[str, Any]) -> list[str]:
    errors = []
    for field in schema.get("required", []):
        if field not in args:
            errors.append(f"missing required field: {field}")
    for field, allowed in schema.get("enum", {}).items():
        if field in args and args[field] not in allowed:
            errors.append(f"invalid value for {field}: {args[field]}")
    return errors

def make_approval_key(tool: ToolSpec, args: dict[str, Any], run_id: str) -> str:
    payload = {
        "tool": tool.name,
        "risk": tool.risk,
```

```

        "args": args,
        "run_id": run_id,
    }
    canonical = json.dumps(payload, sort_keys=True, ensure_ascii=False)
    digest = hashlib.sha256(canonical.encode("utf-8")).hexdigest()[:16]
    return f"{tool.name}:{digest}"

def check_permission(tool: ToolSpec, args: dict[str, Any], approvals: set[str], run_id: str):
    if tool.risk in {"write", "publish", "destructive"}:
        approval_key = make_approval_key(tool, args, run_id)
        if approval_key not in approvals:
            return {
                "ok": False,
                "status": "pending_approval",
                "approval_key": approval_key,
                "tool": tool.name,
                "args": args,
            }
    return None

def execute_tool(call, registry, approvals, trace, run_id):
    try:
        tool = registry.get(call["name"])
    except KeyError:
        tool = None
    if tool is None:
        obs = {
            "ok": False,
            "status": "unknown_tool",
            "tool": call["name"],
        }
        trace.append({"call": call, "observation": obs})
        return obs

    args = call.get("args", {})
    arg_errors = validate_args(tool.input_schema, args)
    if arg_errors:
        obs = {"ok": False, "status": "invalid_args", "errors": arg_errors}
        trace.append({"call": call, "observation": obs})
        return obs

    blocked = check_permission(tool, args, approvals, run_id)
    if blocked:
        trace.append({"call": call, "observation": blocked})
        return blocked

    try:

```



```

    result = tool.fn(**args)
    obs = {"ok": True, "status": "success", "data": result}
except TimeoutError:
    obs = {"ok": False, "status": "timeout", "recoverable": True}
except Exception as exc:
    obs = {"ok": False, "status": "tool_error", "message": str(exc)}

trace.append({"call": call, "observation": obs})
return obs

```

这段执行器最容易漏掉的是失败路径。参数不对、权限被拦，都应作为 `observation` 写回；高风险调用即使函数名正确，也不能自动执行；调用记录进入 `trace` 后，后面才查得清模型、执行器和工具各自做了什么。

4.4.1 工具结果如何回填

工具结果不应只被拼成一句“工具返回如下”。对 `Agent` 来说，工具结果要写入状态。例如 `list_files` 的结果写入 `files_seen`，`propose_move` 的结果写入 `draft_plan`，权限拦截写入 `pending_approval`，工具失败写入 `errors`。只有这样，下一轮模型调用才能基于结构化状态继续行动，而不是从自然语言历史中猜测。

4.5 权限、安全与私人助理边界

工具调用把 `Agent` 从“说话系统”变成“行动系统”。这也是它最有价值、最危险的地方。私人 `Agent` 一旦接入日历、邮件、文件系统、浏览器、代码执行和支付接口，就必须按风险分层，而不是用一个“用户已授权”概括所有动作。

风险层级	示例	推荐策略
只读	读取日历、列文件、检索知识库	可自动，但要记录来源和权限过滤。
建议	起草邮件、生成移动计划、提出会议时间	可自动生成草稿，不产生外部副作用。
写入	移动文件、修改文档、创建日程	需要确认、 <code>dry run</code> 、审计和回滚方案。
发布	发送邮件、发消息、公开网页、提交表单	明确展示内容、对象和后果后再确认。
破坏性	删除文件、覆盖数据、付款、终止服务	默认禁止或二次确认，必要时要求人工手动执行。

浏览器工具尤其要谨慎。浏览器 `Agent` 可以点击、输入、下载、上传和提交表单；这些能力既可以提高效率，也可能造成越权、误提交或隐私泄露。讲义中讨论浏览器作为工具环境时，重点不是绕过限制或模拟真人，而是页面观察、受控点击、授权边界、下载记录和人工确认。

4.6 失败模式、评估与练习

工具调用失败时，表面症状常常是“模型选错工具”或“参数错了”。但根因可能在工具契约、执行器、权限系统或状态更新。下面的矩阵给出本章应优先检查的问题。

症状	可能原因	修复方式
参数幻觉	schema 太宽，必填字段和枚举不清	收窄参数，增加校验错误并让模型修复。
越权调用	风险等级缺失，执行器只按函数名执行	为工具加 risk level，写入审批队列。
工具返回被误读	返回值是自然语言，缺少结构字段	规定 output schema，把状态字段和错误字段分开。
不可回滚副作用	dry run 与 apply 混在同一工具	拆分建议工具和执行工具，写操作需确认。
万能 shell 滥用	为了方便暴露过宽执行能力	改为窄工具集合，shell 默认禁用或仅在沙箱中人工确认。
无法复盘	没有记录调用参数、耗时、错误和审批信息	每次调用写入 trace 和 audit log。

评估工具调用，可以构造一个小型红队集。给定同一目标，检查模型是否选择正确工具；给出缺少参数的任务，检查系统是否澄清而不是猜测；给出越权请求，检查执行器是否拦截；模拟工具超时，检查 Agent 是否重试、降级或报告失败；给出危险动作，检查审批覆盖率是否为百分之百。

4.6.1 与参考项目的关系

OpenAI、Anthropic、Google、AWS 等官方 cookbook 和 SDK 示例，适合帮助我们理解 tool calling 的基本接口；MCP 生态适合帮助我们理解工具、资源和提示的标准化边界；浏览器 Agent 项目适合说明外部环境也可以成为工具。但本讲义不把任何厂商 API 当作唯一主线。正文关心的是可迁移的工程问题：工具契约、执行器责任、权限分层和观察回填。

4.6.2 本章小结

工具调用一接上外部世界，就不能再只看函数名和参数。模型提议，执行器验参、判风险、要确认、记日志；结果要写回状态，危险动作要交给权限系统。否则它只是会调函数，不是能放心托付的 Agent。

第四章 练习

1. 为一个 PDF 解析工具写出完整工具契约。要求包含输入 schema、输出 schema、错误 schema、风险等级和审计字段。
2. 把 `run_shell(command: str)` 拆成三个窄工具，并说明每个工具的权限等级。
3. 设计一个文件整理 Agent 的 dry run / apply 分离流程。要求说明哪一步可以自动，哪一步必须人工确认。
4. 给出五类必须人工确认的工具调用，并为其中一类设计 `pending_approval` 状态字段。
5. 反思题：为什么“模型承诺不会删除文件”不能替代执行器权限控制？请从提示注入、模型误判和审计复盘三个角度回答。

第五章 状态、记忆与任务轨迹

内容提要

- ❑ 长任务的问题常常不在记忆不够多，而在信息放错了位置：状态、上下文、轨迹和长期记忆各有生命周期，混在一起就会污染后续决策。
- ❑ 论文阅读 Agent 跑几轮以后，会留下当前进度、工具记录、用户反馈和可复用偏好；这些东西不能放在同一个地方。
- ❑ 先问这条信息能活多久。所有历史都塞进 prompt，会把上下文撑爆；把“这次用英文”存成长期偏好，下次就会误伤。

5.1 四类“历史”的边界

Agent 做长任务时，一定会产生历史。用户说过什么，模型想过什么，工具调用过什么，检索到哪些资料，失败过几次，用户纠正过哪些偏好，这些信息都可能影响后续行为。问题在于，它们不应被放在同一个篮子里。很多所谓“记忆系统”的失败，本质上是把运行状态、对话上下文、任务轨迹和长期记忆混成一团。

例如，一个论文阅读 Agent 正在帮用户整理一篇论文。用户说：“这次用英文摘要。”这句话应该影响当前任务的输出语言，但不应该立刻写成长期偏好。若系统把它永久保存为“用户默认要英文摘要”，下次用户写中文讲义时就可能出错。相反，如果用户在多个项目中反复确认“我的论文卡片默认要列出 limitation 和 ablation”，这才可能成为长期记忆候选，而且仍然需要展示给用户确认。

定义 5.1 (状态、轨迹与记忆)

状态是一次任务执行期间被显式维护的数据结构；轨迹是已经发生的动作、参数、观察和错误序列；记忆是经过筛选、可跨任务复用、用户可理解且可删除的信息资产。

5.1.1 生命周期决定存放位置

下面的表格给出四类信息的基本边界。它不是数据库设计规范，而是一个思考工具：每当你想把某条信息保存下来，就先问它的生命周期是什么。

类型	生命周期	典型字段	放错后的后果
运行状态	当前任务	phase, evidence, open_questions, done	任务无法恢复，模型重复动作或忘记待办。
短期上下文	一次模型调用	当前目标、状态摘要、最近轨迹、证据片段	上下文膨胀，关键约束被稀释。
任务轨迹	当前任务，可长期归档	action、args、observation、error、timestamp	无法复盘失败，无法审计工具行为。
长期记忆	跨任务或跨项目	稳定偏好、项目背景、用户确认过的约束	记忆污染、隐私风险、过期偏好影响未来任务。

从这个角度看，“把所有聊天历史放进长上下文”不是记忆系统。长上下文只是一次调用可以看到更多信息；记忆系统需要选择、写入、检索、冲突处理、删除和审计。没有这些机制，所谓记忆只是越来越长的历史文本。

5.2 运行时状态如何演化

当前任务走到哪一步，最好写在程序状态里，别指望模型每轮都从聊天记录里猜对。以论文阅读 Agent 为例，状态可以包含论文路径、当前阶段、已读章节、证据、开放问题、草稿章节、错误、待确认事项和最终产物。

```
state = {
  "goal": "write a paper reading card with evidence",
  "paper_path": "papers/example.pdf",
  "phase": "reading_experiments",
  "sections_read": ["abstract", "introduction", "method"],
  "evidence": [
    {"source_id": "p3:intro", "claim": "main contribution"},
  ],
  "open_questions": ["Are ablations sufficient?"],
  "draft_sections": {"contribution": "..."},
  "errors": [],
  "pending_approval": None,
  "trace": [],
  "done": False,
}
```

这段状态让系统知道下一步应该读实验部分，而不是重新总结摘要。若最终卡片缺少实验评价，我们可以检查 phase 是否进入 reading_experiments，evidence 是否包含实验表格，open_questions 是否被关闭。状态字段越清楚，失败越容易定位。

5.2.1 状态演化表

下面是一条简化演化路径：

阶段	触发动作	观察	状态变化
start	parse_pdf	识别章节和页码	phase 变为 reading，保存章节索引。
reading	read_section(method)	找到方法描述	evidence 增加方法证据。
reading_experiments	read_section(experiments)	找到表格和消融	关闭实验相关 open_questions。
drafting	draft_card	生成阅读卡片草稿	draft_sections 写入贡献、方法、实验。
reviewing	citation_check	发现一条结论无来源	重新打开证据问题或降低置信度。
finish	finish	输出带引用卡片	done 变为 true，保存最终产物。

状态不是为了让系统显得复杂，而是为了让下一步有依据。没有状态，模型可能在每一轮都重新“读摘要”；有状态，模型可以知道自己已经读过摘要，现在缺的是实验和局限。

5.3 轨迹与 checkpoint

轨迹是给事后查账用的。对 Agent 来说，最终答案只是运行结果的一部分；更重要的是系统能不能在失败后解释自己。它读取了哪些文件，调用了哪些工具，哪些参数失败了，哪些动作等待用户确认，哪些观察支持最终结论，这些都应进入轨迹。

一个轨迹事件可以写成：

```
trace_event = {
    "step": 3,
    "action": "read_section",
    "args": {"section": "experiments"},
    "observation": {
        "ok": True,
        "source_id": "p7:experiments",
        "summary": "found dataset table and ablation results",
    },
    "status": "success",
    "error": None,
    "timestamp": "2026-06-04T10:15:00",
}
```

时间戳、参数和观察在教学示例中看似啰嗦，在真实调试中非常关键。若 Agent 重复调用同一个工具，可以从 trace 看到参数是否变化；若某个结论引用错误，可以追溯是哪次检索返回了错误证据；若工具产生副作用，可以审计当时是否有用户确认。

5.3.1 checkpoint 支持恢复

长任务不应只保存在内存里。即使本章不实现完整持久化系统，也应理解 checkpoint 的意义：在关键步骤保存状态快照，使系统中断后可以恢复，使失败后可以回放，使模型或工具版本变化后可以重跑特定阶段。

```
def checkpoint_state(state, store, run_id):
    snapshot = {
        "run_id": run_id,
        "phase": state["phase"],
        "state": state,
        "trace_len": len(state["trace"]),
    }
    store.save(snapshot)
    return snapshot

def should_repeat_action(state, action):
    recent = state["trace"][-5:]
    for event in recent:
        if event["action"] == action["name"] and event["args"] == action["args"]:
            return True
    return False
```

should_repeat_action 是一个简单例子。它不需要复杂推理，却能防止许多循环错误。很多 Agent 的失败不是模型完全不知道做什么，而是在错误状态下重复做同一件事。把轨迹结构化以后，这类错误可以用规则先

拦住。

5.4 长期记忆的写入审批

记忆会影响以后的任务，所以比状态更敏感。长期记忆可以提升私人 Agent 的体验，例如记住用户偏好的论文卡片结构、常用实验平台、讲义写作风格、项目目录位置。但它也可能造成长期错误，例如保存模型猜测的用户身份、过期项目背景、一次性语气要求或敏感信息。

命题 5.1 (记忆写入审慎原则)

长期记忆只保存跨任务仍然有用、来源明确、用户可理解、可修改、可删除的信息。临时运行细节应留在状态或轨迹中，而不是进入长期记忆。

5.4.1 候选、确认、写入与删除

不要让模型一句话就 `memory_store.upsert(text)`。先把它当候选记忆拿给用户看：来源是哪轮，准备用在哪些任务，以后怎么改、怎么删。用户点头后再写入；如果用户改了说法，就按用户改后的版本保存。

```
from typing import TypedDict

class MemoryRecord(TypedDict):
    key: str
    value: str
    scope: str          # user, project, task_type
    source: str          # run id or user message id
    confidence: float
    approved: bool
    deleted: bool

def propose_memory(user_feedback, memory_history, run_id) -> MemoryRecord | None:
    if "this time" in user_feedback.lower() or "这次" in user_feedback:
        return None
    if repeated_across_runs(user_feedback, memory_history):
        return {
            "key": "paper_card_default_fields",
            "value": "include contribution, method, experiments, limitations, ablations",
            "scope": "project",
            "source": run_id,
            "confidence": 0.72,
            "approved": False,
            "deleted": False,
        }
    return None

def approve_memory(record, user_edit=None):
    if user_edit:
        record["value"] = user_edit
    record["approved"] = True
```

```

return record

def delete_memory(record):
    record["deleted"] = True
    return record

```

这里特意把 `approved` 和 `deleted` 写成字段。私人定制化不是无限保存用户数据；用户要知道系统记住了什么、为什么记住、何时会用、在哪里删除。没有删除能力的记忆系统，不适合长期私人 Agent。

5.4.2 记忆污染反例

记忆污染有几种典型形式。第一，把一次性要求永久化。用户说“这次摘要用英文”，系统以后默认全用英文。第二，把模型猜测当事实。模型从几次对话推断“用户是某领域教授”，并据此改变输出。第三，把过期项目背景继续使用。一个项目目录已经重组，系统仍把旧路径当默认资料库。第四，把敏感信息写成长期偏好，例如身份证、密钥、私人通讯内容。

这些问题的共同点是：写入太容易，删除太困难，来源不可见。修复方法不是完全不用记忆，而是降低默认写入权限。记忆应该像工具写操作一样，有候选、确认、审计和撤销。

5.5 私人科研 Agent 的分层实践

科研和写作任务最能体现状态、轨迹和记忆的区别。一个长期科研 Agent 可能同时管理用户研究方向、项目计划、论文库、实验日志、写作偏好和审稿意见。若所有信息都放进一个“记忆库”，系统很快会混乱。

更好的分层是：用户层记忆保存稳定偏好，例如“论文卡片需要 `limitation` 和 `ablation`”；项目层记忆保存当前项目目标、数据集、实验平台和投稿约束；任务状态保存当前这次阅读或写作的阶段；轨迹保存具体工具调用、检索结果和失败；证据库存放论文段落、实验表格和引用；日志系统保存运行成本、错误和版本。

层级	保存内容	更新规则
用户层记忆	稳定写作偏好、常用术语、长期研究方向	需要用户确认，可查看和删除。
项目层记忆	项目目标、数据集、实验平台、目录结构	项目文档变化时更新，避免跨项目污染。
任务状态	当前论文、当前阶段、开放问题、草稿	每次运行自动更新，任务结束后归档。
任务轨迹	动作、参数、观察、错误、审批	自动记录，不作为默认上下文全部注入。
证据库	论文段落、页码、表格、引用	通过 RAG 检索进入上下文，保留 source id。

这种分层看似繁琐，却能避免很多长期问题。用户修改写作偏好时，不需要重建论文索引；项目资料过期时，不会污染用户层偏好；某次工具调用失败时，不会变成长期记忆。

5.6 失败模式、评估与练习

状态和记忆问题的症状通常比较隐蔽。系统可能仍然给出流畅答案，但答案基于过期状态、错误记忆或不可复盘轨迹。下面的矩阵列出常见问题。

症状	可能原因	修复方式
重复调用工具	状态没有记录已完成步骤,trace 不可查询	在 state 中记录阶段和最近动作, 加入重复动作检测。
恢复失败	没有 checkpoint 或状态字段不可序列化	在关键阶段保存状态快照,限制状态对象类型。
记忆污染	临时要求或模型猜测直接写入长期记忆	引入候选、用户确认、来源和删除机制。
偏好冲突	新旧记忆自动覆盖,缺少适用范围	为记忆增加 scope、source、confidence 和冲突提示。
隐私失控	敏感信息默认持久化且不可见	默认不写入敏感信息,提供查看和删除入口。
上下文膨胀	把全部历史当记忆注入每次调用	检索相关记忆,旧轨迹摘要化,不相关信息不注入。

评估这类系统,至少要做四类测试。第一,状态恢复测试:中断后从 checkpoint 继续,是否重复或漏掉步骤。第二,重复工具调用计数:同一动作同一参数是否在短时间内反复出现。第三,记忆命中纠正率:系统使用某条记忆后,用户是否经常纠正。第四,删除验证:用户删除某条记忆后,后续任务是否不再使用它。

5.6.1 与参考项目的关系

记忆专题教程、官方 cookbook 的 session memory 示例和 LangGraph 的 checkpoint 思路,都能为本章提供参考。但讲义吸收的是概念边界:短期状态、轨迹、长期记忆、检查点和用户控制权,而不是复制 notebook 结构。尤其是教程型仓库中的 memory 技术很多,不能因此把所有 memory 类型都塞进基础章;本章只要求读者建立生命周期和审批意识。

5.6.2 本章小结

状态、轨迹、记忆不要混用。状态管这次任务怎么继续,轨迹管失败后怎么查账,记忆才管以后能不能复用。三者混在一起,prompt 会越来越胀,旧偏好会污染新任务,隐私也会变得难收拾。

第五章 练习

1. 区分下列信息应放在哪里:用户姓名、当前任务步骤、工具错误日志、论文实验表格、用户一次性要求“这次用英文摘要”。
2. 为论文阅读 Agent 设计一个 State 字段列表,要求覆盖阶段、证据、开放问题、错误和待确认事项。
3. 设计一个 TraceEvent 结构,说明哪些字段支持失败回放,哪些字段支持权限审计。
4. 写出一个记忆候选审批流程:从用户反馈中发现稳定偏好,到展示理由、用户确认、写入、冲突处理和删除。
5. 反思题:为什么“记住用户越多越好”是危险想法?请从过期偏好、隐私、模型猜测和用户控制权四个角度回答。

第六章 知识库与 RAG 基础

内容提要

- ❑ GraphRAG 和向量库都可以以后再谈。本章先抓住 RAG 的底线：答案里每个事实性判断，都要能追到具体来源。
- ❑ 先分清三种问答方式：凭记忆答、检索后答、证据不够就继续查。后面的解析、切分、检索和引用，都围着这个区别展开。
- ❑ 本章只守一条规矩：说了事实，就给出处；找不到出处，就别硬答。可以继续查，也可以请用户补材料。

6.1 从普通问答到 RAG Agent

私人 Agent 要服务真实工作，迟早要接入文档、笔记、论文、项目资料和历史记录。直接让模型凭先验回答，容易产生过时或编造的信息；把全部资料塞进上下文，又不可扩展、不可审计；只用向量相似度检索，也可能召回错误片段、漏掉表格或混入无权限资料。所以别把 RAG 理解成“接个向量库”。真正要补上的，是答案和证据之间那根线。

先看一个论文阅读问题：

这篇论文在哪个数据集上提升最大？这个提升是否有消融实验支持？

普通问答系统可能根据摘要或模型先验直接回答。RAG Bot 会检索论文片段，把相关片段和问题交给模型，再生成带来源的答案。RAG Agent 则更进一步：如果第一次检索只命中摘要，没有命中实验表格，它会改写 query，继续检索 experiments、ablation、dataset、Table 等线索；如果仍然没有证据，它会说明证据不足，而不是根据常识补全。

定义 6.1 (RAG)

检索增强生成是在生成答案前，从外部知识源检索相关证据，并将证据连同问题交给模型的架构。它的目标是提高答案的可追溯性、时效性和领域适配能力。

命题 6.1 (证据优先原则)

RAG 系统的事实性结论应绑定具体来源。没有足够证据时，系统应降低置信度、继续检索、请求用户提供资料或明确拒答，而不是凭模型先验补全。

6.1.1 RAG Bot 与 RAG Agent

RAG Bot 和 RAG Agent 的区别，不在于是否用了向量库，而在于是否把检索看成一个可迭代任务。RAG Bot 通常是“一次检索，一次回答”；RAG Agent 会把证据状态写入任务状态，并根据证据缺口决定下一步。

类型	行为	主要边界
普通问答	直接根据上下文和模型先验回答	容易把常识、猜测和事实混在一起。
RAG Bot	检索若干片段后回答	依赖首次召回质量，证据不足时可能仍然回答。
RAG Agent	多轮检索、改写 query、检查引用、必要时拒答	更可靠，但需要状态、停止条件和成本控制。

这个对比延续了第 1 章的观点：RAG 解决“知识从哪里来”，Agent 解决“下一步做什么”。一个好的私人知识库系统可能两者都需要，但基础章必须先把 RAG 的证据链讲清楚。

6.2 基础 pipeline 的工程定义

最小 RAG 不是把文本扔进向量库就结束。先把文档解析对，保住表格、标题和页码；再切成能引用、能按权限过滤的片段；最后检查答案里的结论是否真的有来源。很多失败并不是模型弱，而是证据链前面已经断了。

步骤	输入输出	常见失败
加载与解析	PDF、Markdown、网页转为结构化文本	表格丢失、页码错位、标题层级丢失。
清洗	去噪、统一编码、保留结构	删除了脚注、公式、表头或关键编号。
chunk	按章节、段落、表格或语义切分	chunk 过大含噪，过小失去上下文。
embedding	把 chunk 转为向量	领域术语、公式和表格语义表达不足。
索引	保存向量、文本和 metadata	缺 source id、权限字段或更新时间。
检索	根据 query 召回候选片段	query 太窄、只看相似度、漏掉关键词。
重排	用更精细模型或规则排序	成本增加，若候选差仍无法补救。
生成	把证据和问题交给模型	模型加入未被证据支持的结论。
引用检查	检查 claim 是否被 source 支持	引用漂移、句子和来源绑定不清。

这条 pipeline 不要求所有系统都使用同一框架。LlamaIndex、Haystack、LangChain、RAGFlow 或自写代码都可以承载其中部分步骤。基础章的重点是认识这些步骤的输入输出和失败模式，而不是记住某个库的 API。

6.3 贯穿例子：论文实验问题的 RAG 流程

下面把“哪一个数据集提升最大”这个问题写成一个完整 RAG 流程。假设系统已经得到一篇论文 PDF，目标是回答实验结果，而不是只总结摘要。

6.3.1 解析与 chunk

论文 PDF 解析后，不应只得到一长段文本。更好的结构是保留标题、页码、段落类型和表格边界。实验表格尤其重要，因为问题中的“提升最大”通常需要比较数值。如果解析器把表格打散，后续 embedding 再好也难以恢复原始关系。

```
chunk = {
  "chunk_id": "paperA:p7:table2",
  "source_id": "paperA",
  "title_path": ["Experiments", "Main Results"],
  "page": 7,
  "kind": "table",
  "text": "Table 2: Accuracy on Dataset A, B, C ...",
  "metadata": {
    "project_id": "agent-book",
    "visibility": "private",
    "sensitivity": "normal",
  },
}
```

这个 chunk 里，text 只是信息的一部分；chunk_id、page、kind 和权限 metadata 同样重要。没有这些字段，答案就很难引用具体来源，也很难在私人知识库中做权限过滤。

6.3.2 检索、重排与证据不足

一次简单 query 可能写成 “largest improvement dataset”。它可能召回摘要中的贡献描述，却没有召回实验表格。RAG Agent 不应直接回答，而应根据状态改写 query，例如加入 “Table”，“ablation”，“dataset”，“baseline”，“improvement”。若仍然没有表格证据，系统应告诉用户证据不足，而不是编造。

步骤	查询或动作	检索结果	状态变化
1	largest improvement dataset	命中摘要和引言	证据不足，缺实验表格。
2	Table results dataset baseline	命中 Table 2 和实验段落	记录候选证据 source id。
3	ablation dataset improvement	命中消融段落	增加支持或反证证据。
4	citation check	发现一条结论无来源	删除 unsupported claim 或继续检索。
5	finish	输出带引用答案	每个结论绑定 chunk id 和页码。

看这条轨迹时，先盯住“证据不足”这一步。RAG Agent 不是简单地多检索几条结果，而是把证据缺口写进状态，再决定改 query、继续查、检查引用还是停来说明不足。

6.3.3 稀疏检索与混合检索

基础 RAG 不应只理解为向量检索。向量检索擅长语义相近的问题，例如“实验是否充分”和“有没有 ablation study”；但它不一定擅长精确匹配表格编号、数据集名称、API 标识符、法律条款、章节标题和引用编号。这些场景中，BM25、关键词检索或其他稀疏检索方法常常更可靠。

因此，许多实际系统会使用混合检索：先用 dense retrieval 找语义相近片段，同时用 sparse retrieval 找精确词项，再合并候选并交给 reranker。对论文实验问题来说，dense 检索可能找到“performance improves on multiple benchmarks”这样的段落，sparse 检索则可能直接命中“Table 2”、数据集名和 baseline 名称。二者互补，才能降低漏召回风险。

6.4 chunk、metadata 与权限过滤

RAG 质量很大程度上由数据工程决定。好的 chunk 策略既要保留语义完整性，又要支持精确引用和权限过滤。不同资料需要不同切分方式：讲义可以按 chapter、section、subsection 和页码切；论文可以按标题、段落、表格和图注切；代码文档可以按文件、类、函数和代码块切；会议纪要可以按议题、发言人和时间段切。

6.4.1 chunk 不只是长度参数

许多入门系统把 chunk 当成“每 500 字切一段，重叠 50 字”。这可以作为起点，但不足以支撑严肃问答。对于论文实验问题，表格、表头、表注和附近解释应保持在同一或相邻 chunk 中；对于讲义写作，标题层级比固定长度更重要；对于项目文档，版本号、路径和更新时间会影响答案是否过期。

资料类型	推荐切分	必须保留的 metadata
讲义 PDF / LaTeX	chapter、section、页码、定义或例题块	章节号、标题、页码、源文件路径。
论文 PDF	标题层级、段落、表格、图注	论文 ID、页码、段落类型、表格编号。
Markdown 笔记	标题、列表、代码块	项目、标签、更新时间、作者。
项目资料	文档类型、版本、路径、段落	project_id、owner、visibility、sensitivity。

6.4.2 私人知识库的权限字段

私人知识库 RAG 必须在检索阶段做权限过滤，而不是检索后再让模型“不要看”。如果用户询问项目 A，系统不应把项目 B 的私密资料召回再交给模型。metadata 至少应包含 project_id、owner、visibility、sensitivity、updated_at 和 source_path 等字段。

权限过滤也要支持删除和更新。用户删除某份资料后，索引中对应 chunk 应被删除或标记不可用；资料更新后，旧 chunk 不应继续作为证据回答。否则，RAG 系统会变成一个难以察觉的过期知识缓存。

6.5 最小 RAG 代码骨架

下面的骨架不绑定具体向量数据库，只保留几项必要边界：检索前做权限过滤，合并 dense 与 sparse 两路候选，在 prompt 中保留 source id，并在生成后检查引用。

```
from typing import TypedDict, Any

class Document(TypedDict):
    doc_id: str
    text: str
    metadata: dict[str, Any]

class Chunk(TypedDict):
    chunk_id: str
    source_id: str
    text: str
    metadata: dict[str, Any]

class Passage(TypedDict):
    chunk_id: str
    text: str
    score: float
    metadata: dict[str, Any]

def permission_filter(query_context, chunk: Chunk) -> bool:
    return (
        chunk["metadata"].get("project_id") == query_context["project_id"]
        and chunk["metadata"].get("visibility") in query_context["allowed_visibility"]
    )

def dense_retrieve(question, allowed_chunks, embedder, top_k=20):
    query_vec = embedder(question)
    return vector_search(query_vec, allowed_chunks, top_k=top_k)

def sparse_retrieve(question, allowed_chunks, top_k=20):
    return bm25_search(question, allowed_chunks, top_k=top_k)
```

```

def merge_candidates(*candidate_lists):
    merged = {}
    for candidates in candidate_lists:
        for passage in candidates:
            merged[passage["chunk_id"]] = passage
    return list(merged.values())

def retrieve(question, chunks, query_context, embedder, top_k=20):
    allowed = [c for c in chunks if permission_filter(query_context, c)]
    dense = dense_retrieve(question, allowed, embedder, top_k=top_k)
    sparse = sparse_retrieve(question, allowed, top_k=top_k)
    return merge_candidates(dense, sparse)

def build_prompt(question: str, passages: list[Passage]) -> str:
    evidence = []
    for p in passages:
        evidence.append(
            f"[source={p['chunk_id']} page={p['metadata'].get('page')}] {p['text']}"
        )
    return "\n".join([
        "Answer only from the evidence below.",
        "Cite the supporting source id for every factual claim, for example [source=paperA:p7:table2].",
        ,
        "If evidence is insufficient, say so.",
        "\n".join(evidence),
        f"Question: {question}",
    ])

def citation_check(answer, passages):
    source_ids = {p["chunk_id"] for p in passages}
    unsupported = []
    for claim in extract_claims(answer):
        if not claim_has_valid_source(claim, source_ids):
            unsupported.append(claim)
    return unsupported

def rag_answer(question, chunks, query_context, embedder, reranker, model):
    candidates = retrieve(question, chunks, query_context, embedder)
    passages = reranker(question, candidates)[:6]
    if not passages:
        return {"answer": "证据不足, 未检索到可用资料。", "sources": []}
    answer = model.generate(build_prompt(question, passages))
    unsupported = citation_check(answer, passages)
    if unsupported:
        return {

```

```

    "answer": "证据不足，以下结论缺少来源支持。",
    "unsupported_claims": unsupported,
    "sources": passages,
  }
  return {"answer": answer, "sources": passages}

```

这段骨架的边界在检索之前就开始了。无权限资料不能先交给模型再靠提示词约束；进入 prompt 的 passage 要带上 source id 和页码；生成后如果引用撑不住结论，系统应降级或拒答，而不是把无来源判断写得很顺。

6.6 评估、反例与练习

RAG 评估不能只看最终答案是否“看起来对”。至少要分别评估召回、引用和回答。召回看的是：该找的证据有没有找出来。引用看的是：标出的来源能不能支撑对应句子。回答本身再看结论是否正确、完整，有没有越过证据往外推。

指标	含义	数据构造方式
Recall@k	前 k 个候选是否包含 gold evidence	人工标注问题与支持段落。
MRR / nDCG	正确证据排序是否靠前	为多个相关证据设置相关度等级。
Citation precision	引用是否支持对应 claim	逐句检查 claim-source 对。
Unsupported claim rate	答案中无证据结论比例	抽取 claim 并标注是否被来源支持。
Abstention rate	证据不足时是否拒答	构造无答案或资料缺失问题。

常见反例包括：只接向量库但没有 source id；把全文塞进上下文但不做引用；检索结果没有权限过滤；答案引用了摘要却回答实验结果；资料更新后旧索引仍被使用；chunk 切分把表格和表头分开；模型在证据不足时用先验补全。这些问题都不能靠“换一个更强模型”完全解决，因为它们发生在证据工作流的结构层。

6.6.1 与参考项目的关系

LlamaIndex、Haystack、LangChain 的 RAG 教程适合帮助读者理解 pipeline 抽象；RAGFlow 这类生产级系统适合提醒我们文档解析、grounded citation、多源数据和自托管边界的重要性；Second Brain 类课程适合说明私人知识库的 ETL、观测和长期维护；RAG 技术集合可以作为 query transform、rerank 和复杂问答的延伸阅读。基础章只吸收这些项目的架构思想，不复制 notebook 或版本敏感 API。

6.6.2 本章小结

RAG 的底线不是“检索一次”，而是把证据链接好。解析时保住结构，chunk 能回到来源，metadata 能挡住无权限资料；如果证据仍不够，Agent 应改写查询、检查引用，或者直接说明证据不足，而不应补出一个看似完整的答案。

第六章 练习

1. 为一本讲义 PDF 设计 chunk 策略。要求说明如何处理章节标题、页码、定义、例题、代码块和表格。
2. 解释 Recall@k、Citation precision 和 Unsupported claim rate 的区别。为什么只看答案正确率不够？
3. 为私人知识库设计 metadata 字段，至少包含 project_id、owner、visibility、sensitivity 和 updated_at，并说明每个字段如何影响检索。
4. 把“这篇论文在哪个数据集上提升最大”设计成一个 RAG Agent 流程。要求包含 query 改写、继续检索、引用检查和证据不足时的拒答。

5. 反思题：为什么“把全文塞进长上下文”不能替代 RAG？请从成本、引用、权限、更新和调试五个角度回答。

第二部分

十大 Agent 架构

第七章 LLM Chain Agent

内容提要

- ❑ 掌握线性任务管道的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

7.1 问题引入

把固定步骤串成可测试的链条，适合摘要、分类、格式转换等流程稳定的任务。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 7.1 (LLM Chain Agent)

LLM Chain Agent 是一种 Agent 设计模式，其核心是：把固定步骤串成可测试的链条，适合摘要、分类、格式转换等流程稳定的任务。

命题 7.1 (架构适配原则)

只有当任务确实需要线性任务管线时才采用该架构。否则应优先选择更简单、更可测试的实现。

7.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

7.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
class ChainStep:
    def __init__(self, name, fn):
        self.name = name
        self.fn = fn
```



```
def run_chain(question, steps):
    artifact = {'question': question}
    for step in steps:
        artifact[step.name] = step.fn(artifact)
    return artifact['final_answer']
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

7.4 深入讲解

Chain 适合步骤固定、输入输出明确的任务。例如先抽取字段，再改写风格，最后格式化为 JSON。它不适合探索式任务，因为链条一旦写死，就很难根据观察动态改变路线。

评估 Chain 时要逐步检查每个节点，而不是只看最终答案。若第一步抽取错了，后面的改写和格式化可能都很漂亮，但最终结论仍然错误。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

7.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

7.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

7.7 失败模式与调试

常见失败模式如下：

- 链越长，误差越会向后传播。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

7.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

7.9 私人订制化实践

私人 Agent 采用 LLM Chain Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

7.10 本章例题

例题 7.1 三步摘要链 先抽取要点，再改写成讲义风格，最后检查格式。每一步输入输出固定，适合 Chain。

解 若第二步输出已经偏离事实，第三步格式检查无法自动恢复事实正确性。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

7.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

7.12 进阶练习

1. 把“三步摘要链”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

7.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

7.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

7.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

7.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

7.17 案例展开与验收标准

为了避免把“LLM Chain Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，

失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

7.18 常见误区与反例

学习“LLM Chain Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人定制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

7.19 小结

本章的核心结论是：掌握线性任务管线的核心思想、适用场景、工程代价和私人定制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

 第七章 练习

1. 为一个论文阅读任务判断是否需要 LLM Chain Agent。
2. 写出 LLM Chain Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第八章 Tool-Using Agent

内容提要

- ❑ 掌握工具使用型架构的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

8.1 问题引入

让模型根据状态选择外部工具，适合需要查询、计算、读写文件或调用 API 的任务。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 8.1 (Tool-Using Agent)

Tool-Using Agent 是一种 Agent 设计模式，其核心是：让模型根据状态选择外部工具，适合需要查询、计算、读写文件或调用 API 的任务。

命题 8.1 (架构适配原则)

只有当任务确实需要工具使用型架构时才采用该架构。否则应优先选择更简单、更可测试的实现。

8.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

8.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def choose_and_call(model, state, registry):
    call = model.select_tool(state, schemas=registry.schemas())
    tool = registry.get(call.name)
    tool.validate(call.args)
    if tool.risk == 'write' and not state['approved']:
```

```
return {'needs_approval': call}
return tool(**call.args)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

8.4 深入讲解

Tool-Using Agent 的关键是选择权。模型不仅要知道有哪些工具，还要知道何时不该调用工具。工具选择错误通常来自工具描述重叠、参数 schema 过宽或状态缺少任务阶段。

设计工具表时，可以为每个工具写四个字段：能力、输入、输出、风险。只有当这四个字段清楚时，模型选择工具的行为才容易调试。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

8.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

8.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

8.7 失败模式与调试

常见失败模式如下：

- 工具能力越大，权限风险越高。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

8.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

8.9 私人订制化实践

私人 Agent 采用 Tool-Using Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

8.10 本章例题

例题 8.1 计算器工具 模型需要回答复利问题时不应心算，而应调用 calculator 工具并把公式和参数写入状态。

解 工具结果应回填到状态中，最终答案要说明计算依据。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

8.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

8.12 进阶练习

1. 把“计算器工具”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

8.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

8.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

8.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

8.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

8.17 案例展开与验收标准

为了避免把“Tool-Using Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

8.18 常见误区与反例

学习“Tool-Using Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

8.19 小结

本章的核心结论是：掌握工具使用型架构的核心思想、适用场景、工程代价和私人订制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第八章 练习

1. 为一个论文阅读任务判断是否需要 Tool-Using Agent。
2. 写出 Tool-Using Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第九章 ReAct Agent

内容提要

- ❑ 掌握推理与行动交替的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

9.1 问题引入

通过可审计的决策摘要、动作和观察循环处理探索式任务。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 9.1 (ReAct Agent)

ReAct Agent 是一种 Agent 设计模式，其核心是：通过可审计的决策摘要、动作和观察循环处理探索式任务。

命题 9.1 (架构适配原则)

只有当任务确实需要推理与行动交替时才采用该架构。否则应优先选择更简单、更可测试的实现。

9.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

9.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def react_step(model, state, tools):
    decision = model.next_action(state)
    log = {'reason_summary': decision.reason_summary, 'tool': decision.tool}
    observation = tools[decision.tool](**decision.args)
    state['trace'].append(**log, 'observation': observation)
```

```
return state
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

9.4 深入讲解

ReAct 的价值在于把行动和观察交替起来。它适合搜索文档、浏览网页、排查错误等探索式任务。每一步行动都应有简短理由摘要和可见 observation。

实际系统不应要求或展示完整思维链。更好的做法是记录决策摘要、所选工具、参数、观察和下一步状态。这些信息足以审计，又不会把内部推理当作用户界面。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

9.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

9.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

9.7 失败模式与调试

常见失败模式如下：

- 不应暴露完整思维链，应记录动作理由和证据。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

9.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

9.9 私人定制化实践

私人 Agent 采用 ReAct Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

9.10 本章例题

例题 9.1 逐步查证 Agent 不确定某 API 的参数时，先搜索文档，观察结果，再调用示例代码检查。

解 日志中记录的是动作理由摘要，不暴露完整思维链。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

9.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

9.12 进阶练习

1. 把“逐步查证”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

9.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

9.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

9.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

9.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

9.17 案例展开与验收标准

为了避免把“ReAct Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

9.18 常见误区与反例

学习“ReAct Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

9.19 补充注记

对“ReAct Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

9.20 小结

本章的核心结论是：掌握推理与行动交替的核心思想、适用场景、工程代价和私人定制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第九章 练习

1. 为一个论文阅读任务判断是否需要 ReAct Agent。
2. 写出 ReAct Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十章 Plan-and-Execute Agent

内容提要

- ❑ 掌握先规划后执行的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

10.1 问题引入

把长任务拆成计划、执行、检查和重规划，适合讲义写作、实验部署等任务。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 10.1 (Plan-and-Execute Agent)

Plan-and-Execute Agent 是一种 Agent 设计模式，其核心是：把长任务拆成计划、执行、检查和重规划，适合讲义写作、实验部署等任务。

命题 10.1 (架构适配原则)

只有当任务确实需要先规划后执行时才采用该架构。否则应优先选择更简单、更可测试的实现。

10.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

10.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def plan_execute(goal, planner, executor, verifier):
    plan = planner(goal)
    for task in plan.tasks:
        result = executor(task)
        verdict = verifier(task, result)
```

```

if not verdict.ok:
    plan = plan.repair(task, verdict)
return plan.collect_outputs()

```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

10.4 深入讲解

Plan-and-Execute 把长任务拆成两个层级：规划器负责结构，执行器负责具体动作。这个分离使任务更容易分配、恢复和检查。

计划不是承诺，而是假设。执行过程中发现资料缺失、测试失败或权限不足时，应允许重规划。不能因为最初计划写得完整，就忽略新的观察。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

10.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

10.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

10.7 失败模式与调试

常见失败模式如下：

- 过度规划会制造虚假的确定性。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

10.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

10.9 私人订制化实践

私人 Agent 采用 Plan-and-Execute Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

10.10 本章例题

例题 10.1 写一章讲义 Planner 生成目录、定义、例题和练习四个子任务；Executor 逐项写作；Verifier 检查是否缺定义或代码。

解 计划应允许修订。若执行中发现资料不足，必须回到规划阶段。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

10.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

10.12 进阶练习

1. 把“写一章讲义”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

10.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下

面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

10.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

10.15 本章小项目

用同一个论文阅读任务做对照实验：用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

10.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

10.17 案例展开与验收标准

为了避免把“Plan-and-Execute Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。

- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

10.18 常见误区与反例

学习“Plan-and-Execute Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

10.19 小结

本章的核心结论是：掌握先规划后执行的核心思想、适用场景、工程代价和私人订制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第十章 练习

- 1. 为一个论文阅读任务判断是否需要 Plan-and-Execute Agent。

2. 写出 Plan-and-Execute Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十一章 RAG Agent

内容提要

- ❑ 掌握检索增强任务系统的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

11.1 问题引入

把检索、证据压缩、生成和引用检查纳入 Agent 循环。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 11.1 (RAG Agent)

RAG Agent 是一种 Agent 设计模式，其核心是：把检索、证据压缩、生成和引用检查纳入 Agent 循环。

命题 11.1 (架构适配原则)

只有当任务确实需要检索增强任务系统时才采用该架构。否则应优先选择更简单、更可测试的实现。

11.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

11.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def rag_agent(question, retriever, model):
    docs = retriever.search(question, filters={'project': 'current'})
    if not docs:
        return {'answer': '证据不足', 'sources': []}
    draft = model.answer(question=question, evidence=docs)
    return add_citations(draft, docs)
```


 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

11.4 深入讲解

RAG Agent 不只是先检索再回答。它还要判断证据是否足够、是否需要换 query、是否要读取更多上下文、是否能给出引用。

对于科研和工程任务，RAG Agent 的输出最好包含 answer、sources、confidence 和 missing evidence。这样用户可以知道哪些结论可靠，哪些需要继续查证。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

11.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

11.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

11.7 失败模式与调试

常见失败模式如下：

- RAG 失败常来自数据治理，而非模型生成。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

11.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

11.9 私人订制化实践

私人 Agent 采用 RAG Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

11.10 本章例题

例题 11.1 项目文档助手 用户问一个配置项含义，系统先检索 README、配置文件和 issue 记录，再生成答案。

解 答案必须带文件路径或文档来源，否则无法作为工程决策依据。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

11.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

11.12 进阶练习

1. 把“项目文档助手”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

11.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

11.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

11.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

11.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

11.17 案例展开与验收标准

为了避免把“RAG Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

11.18 常见误区与反例

学习“RAG Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

11.19 补充注记

对“RAG Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

11.20 小结

本章的核心结论是：掌握检索增强任务系统的核心思想、适用场景、工程代价和私人定制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第十一章 练习

1. 为一个论文阅读任务判断是否需要 RAG Agent。
2. 写出 RAG Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十二章 Memory-Augmented Agent

内容提要

- ❑ 掌握记忆增强架构的核心思想、适用场景、工程代价和私人订制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

12.1 问题引入

通过偏好、项目背景和历史反馈实现私人订制化。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 12.1 (Memory-Augmented Agent)

Memory-Augmented Agent 是一种 Agent 设计模式，其核心是：通过偏好、项目背景和历史反馈实现私人订制化。

命题 12.1 (架构适配原则)

只有当任务确实需要记忆增强架构时才采用该架构。否则应优先选择更简单、更可测试的实现。

12.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

12.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def memory_augmented_reply(user_id, task, memory, model):
    prefs = memory.search(user_id, kind='preference')
    project = memory.search(user_id, kind='project_context')
    answer = model.generate(task=task, preferences=prefs, context=project)
    return {'answer': answer, 'memory_used': [m.id for m in prefs + project]}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

12.4 深入讲解

记忆增强的收益来自跨任务连续性。例如用户长期使用某种论文卡片格式，Agent 可以自动沿用。但这种连续性必须服从用户控制。

记忆系统应同时支持写入、检索、冲突检测、失效和删除。没有删除能力的记忆不是私人订制，而是隐性负担。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

12.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

12.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

12.7 失败模式与调试

常见失败模式如下：

- 记忆必须可解释、可编辑、可删除。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

12.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

12.9 私人订制化实践

私人 Agent 采用 Memory-Augmented Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

12.10 本章例题

例题 12.1 长期写作偏好 用户希望所有讲义章节都有“常见误区”。Agent 在后续章节生成前自动读取该偏好。

解 偏好应可关闭。某章若不适合常见误区，Agent 应说明原因而不是机械添加。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

12.11 失败模式矩阵

症状	可能原因	修复方式
记忆投毒	错误偏好被长期保存	写入前确认并保留来源
记忆冲突	新旧偏好相互矛盾	显示冲突并让用户选择
隐私风险	保存敏感信息	默认不写入并支持删除

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

12.12 进阶练习

1. 把“长期写作偏好”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

12.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

12.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

12.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

12.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

12.17 案例展开与验收标准

为了避免把“Memory-Augmented Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

12.18 常见误区与反例

学习“Memory-Augmented Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

12.19 小结

本章的核心结论是：掌握记忆增强架构的核心思想、适用场景、工程代价和私人订制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第十二章 练习

- 1. 为一个论文阅读任务判断是否需要 Memory-Augmented Agent。
- 2. 写出 Memory-Augmented Agent 的状态字段和停止条件。
- 3. 设计一个失败样本，并说明如何从日志中定位问题。

第十三章 Reflection Agent

内容提要

- ❑ 掌握反思与修订闭环的核心思想、适用场景、工程代价和私人订制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

13.1 问题引入

让 Agent 根据测试、规则、检索证据或人工反馈修复初稿。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 13.1 (Reflection Agent)

Reflection Agent 是一种 Agent 设计模式，其核心是：让 Agent 根据测试、规则、检索证据或人工反馈修复初稿。

命题 13.1 (架构适配原则)

只有当任务确实需要反思与修订闭环时才采用该架构。否则应优先选择更简单、更可测试的实现。

13.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

13.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def reflect_then_revise(task, writer, checker, reviser):
    draft = writer(task)
    report = checker(draft)
    if report.passed:
        return draft
```

```
return reviser(draft=draft, issues=report.issues)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

13.4 深入讲解

Reflection 的可靠版本不是“模型再想一想”，而是“模型根据检查表、测试、证据或人工反馈修订”。外部信号越明确，反思越有效。

写作、代码和研究任务都适合反思闭环。写作可以检查风格和结构；代码可以运行测试；研究可以做 hostile review。每一种反思都应产出可执行修复项。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

13.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

13.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

13.7 失败模式与调试

常见失败模式如下：

- 没有外部信号的反思容易自我确认。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

13.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

13.9 私人订制化实践

私人 Agent 采用 Reflection Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

13.10 本章例题

例题 13.1 生成后检查 写作 Agent 先生成章节，再由 checker 检查是否缺代码、缺练习、缺失失败模式。

解 反思节点必须有检查表或测试结果，不能只让模型自由评价。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

13.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

13.12 进阶练习

1. 把“生成后检查”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

13.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

13.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

13.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

13.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

13.17 案例展开与验收标准

为了避免把“Reflection Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

13.18 常见误区与反例

学习“Reflection Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

13.19 补充注记

对“Reflection Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

13.20 小结

本章的核心结论是：掌握反思与修订闭环的核心思想、适用场景、工程代价和私人定制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第十三章 练习

1. 为一个论文阅读任务判断是否需要 Reflection Agent。
2. 写出 Reflection Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十四章 Workflow / Graph Agent

内容提要

- ❑ 掌握工作流与状态图的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

14.1 问题引入

用节点、边、条件和检查点（checkpoint）表示可控长流程，LangGraph 是典型实现。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 14.1 (Workflow / Graph Agent)

Workflow / Graph Agent 是一种 Agent 设计模式，其核心是：用节点、边、条件和检查点（checkpoint）表示可控长流程，LangGraph 是典型实现。

命题 14.1 (架构适配原则)

只有当任务确实需要工作流与状态图时才采用该架构。否则应优先选择更简单、更可测试的实现。

14.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

14.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def route_after_check(state):
    if state['check'].passed:
        return 'finish'
    if state['retries'] < 2:
        return 'revise'
```

```

    return 'human_review'

graph.add_node('draft', draft)
graph.add_node('check', check)
graph.add_conditional_edges('check', route_after_check)

```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

14.4 深入讲解

Graph Agent 把流程显式化。节点是可测试函数，边是控制流，状态是跨节点共享的数据，checkpoint 是恢复点。

LangGraph 这类框架适合长任务、循环任务和需要人工中断的任务。不适合只有一两个固定步骤的小任务。图的复杂度必须由任务复杂度证明，而不是由架构偏好决定。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

14.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

14.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

14.7 失败模式与调试

常见失败模式如下：

- 图不是越复杂越好；状态边界必须清楚。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

14.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

14.9 私人订制化实践

私人 Agent 采用 Workflow / Graph Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

14.10 本章例题

例题 14.1 可恢复写作图 节点包括 plan、draft、style_check、compile、revise 和 human_review。编译失败进入 revise，连续失败进入人工审查。

解 图结构让失败恢复路径显式化，checkpoint 让长任务可中断后继续。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

14.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

14.12 进阶练习

1. 把“可恢复写作图”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

14.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

14.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

14.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

14.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

14.17 案例展开与验收标准

为了避免把“Workflow / Graph Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授

权，失败如何复现，用户如何介入。

- 1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限权检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

14.18 常见误区与反例

学习“Workflow / Graph Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

14.19 小结

本章的核心结论是：掌握工作流与状态图的核心思想、适用场景、工程代价和私人订制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

 第十四章 练习

1. 为一个论文阅读任务判断是否需要 Workflow / Graph Agent。
2. 写出 Workflow / Graph Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十五章 Multi-Agent Collaboration

内容提要

- ❑ 掌握多 Agent 协作的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

15.1 问题引入

通过角色分工、共享状态和仲裁机制处理需要并行或互审的任务。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 15.1 (Multi-Agent Collaboration)

Multi-Agent Collaboration 是一种 Agent 设计模式，其核心是：通过角色分工、共享状态和仲裁机制处理需要并行或互审的任务。

命题 15.1 (架构适配原则)

只有当任务确实需要多 Agent 协作时才采用该架构。否则应优先选择更简单、更可测试的实现。

15.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

15.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def supervisor(goal, agents):  
    shared = {'goal': goal, 'claims': [], 'issues': []}  
    shared['claims'] += agents['researcher'].run(goal)  
    shared['prototype'] = agents['engineer'].run(shared)  
    shared['issues'] += agents['reviewer'].run(shared)
```



```
return agents['writer'].run(shared)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

15.4 深入讲解

多 Agent 的真正价值来自并行、专业化和互审。若几个角色看到同样上下文、使用同样工具、输出同样格式，那么它们只是更贵的单 Agent。

有效多 Agent 系统要定义通信协议、共享状态、仲裁者和停止条件。特别是科研场景，Reviewer 必须能否决 unsupported claims，Engineer 必须能报告实验失败。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

15.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

15.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

15.7 失败模式与调试

常见失败模式如下：

- 多 Agent 增加通信成本和责任不清风险。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

15.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

15.9 私人订制化实践

私人 Agent 采用 Multi-Agent Collaboration 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

15.10 本章例题

例题 15.1 研究小组 Researcher 找资料，Engineer 写实验脚本，Reviewer 质疑结论，Writer 组织文本，Supervisor 决定是否继续。

解 多 Agent 的收益来自专业分工和互审，而不是更多角色名字。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

15.11 失败模式矩阵

症状	可能原因	修复方式
角色空转	角色没有独立工具或职责	定义输入输出和仲裁规则
对话发散	无终止条件	设置预算和最大轮次
责任不清	共享状态混乱	Supervisor 记录决策

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

15.12 进阶练习

1. 把“研究小组”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

15.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下

面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

15.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

15.15 本章小项目

用同一个论文阅读任务做对照实验：用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

15.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

15.17 案例展开与验收标准

为了避免把“Multi-Agent Collaboration”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。

- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

15.18 常见误区与反例

学习“Multi-Agent Collaboration”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

15.19 小结

本章的核心结论是：掌握多 Agent 协作的核心思想、适用场景、工程代价和私人订制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第十五章 练习

- 1. 为一个论文阅读任务判断是否需要 Multi-Agent Collaboration。

2. 写出 **Multi-Agent Collaboration** 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第十六章 Human-in-the-loop Agent

内容提要

- ❑ 掌握人在回路中的核心思想、适用场景、工程代价和私人定制化落地方式。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：十大 Agent 架构。

16.1 问题引入

在高风险、低置信或价值判断节点引入人工确认。架构章节的重点不是背名词，而是回答：任务为什么需要这种结构，它带来什么可靠性收益，又增加什么复杂度。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 16.1 (Human-in-the-loop Agent)

Human-in-the-loop Agent 是一种 Agent 设计模式，其核心是：在高风险、低置信或价值判断节点引入人工确认。

命题 16.1 (架构适配原则)

只有当任务确实需要人在回路中时才采用该架构。否则应优先选择更简单、更可测试的实现。

16.2 执行流程

一个可落地的实现通常包含以下步骤：

- 明确任务特征和成功标准。
- 定义状态对象和输入输出契约。
- 选择该架构的关键节点或循环。
- 记录每一步动作、观察和错误。
- 用小型评估集判断该架构是否优于简单基线。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

16.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def guarded_execute(action, approvals):
    if action.risk in {'delete', 'send', 'pay', 'publish'}:
        ticket = approvals.request(action)
        return {'status': 'waiting_for_human', 'ticket': ticket.id}
    return action.execute()
```



笔记 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

16.4 深入讲解

人在回路中不是降低自动化水平，而是提高系统可信度。高风险动作、价值判断、低置信答案和不可逆操作都应进入人工节点。

好的审批界面应展示：**Agent** 想做什么，为什么做，依据是什么，可能风险是什么，用户可以批准、拒绝还是修改。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，**Agent** 才能从演示走向可维护系统。

16.5 工程实现要点

- 先写一个不用框架的 Python 骨架。
- 再映射到 LangGraph、LangChain 或其他框架。
- 保持每个节点职责单一。
- 把评估写入开发流程。
- 保留人工中断和回滚路径。

16.6 适用与不适用

适用场景：

- 任务结构与该架构的假设匹配。
- 需要过程可见和结果可评估。
- 私人工具和知识库已经有清晰边界。

不适用或需要谨慎的场景：

- 为了显得高级而套用架构。
- 任务只有一步却引入复杂编排。
- 无法定义失败和成功标准。

16.7 失败模式与调试

常见失败模式如下：

- 人工确认不是装饰，而是系统边界的一部分。
- 没有明确输入输出契约，导致模块之间责任不清。
- 缺少日志和评估，无法判断架构是否真的带来收益。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

16.8 评估指标

本章相关系统至少应记录：

- 相对简单基线的成功率提升。
- 额外 token 和延迟成本。
- 失败可定位性。
- 人工接管次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

16.9 私人订制化实践

私人 Agent 采用 Human-in-the-loop Agent 时，应把用户资料、领域知识、工具权限和反馈指标放入显式配置。个性化不应依赖模型临场猜测。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

16.10 本章例题

例题 16.1 发送邮件前确认 Agent 起草会议邀请后只生成草稿和收件人列表，真正发送前必须等待用户确认。

解 人在回路中把不可逆外部动作变成可审计的审批事件。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

16.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

16.12 进阶练习

1. 把“发送邮件前确认”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

16.13 设计边界与选择表

架构篇的章节要回答选择问题。每种架构都不是越高级越好，而是在任务结构、可靠性、成本和可解释性之间做出的取舍。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

16.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

16.15 本章小项目

用同一个论文阅读任务做对照实验：先用最简单的单次问答作为基线，再加入本章架构，记录成功率、工具调用次数、人工修改量和失败样本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

16.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

16.17 案例展开与验收标准

为了避免把“Human-in-the-loop Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能说明本章架构相对单次问答或简单链式调用多解决了什么问题。
- 能写出该架构的状态对象、节点或循环，并指出停止条件。
- 能给出一个不应该使用该架构的反例，说明复杂度为何不值得。
- 能用至少一个指标比较该架构和简单基线。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

16.18 常见误区与反例

学习“Human-in-the-loop Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
为架构而架构	如果一个任务用三步固定流程即可完成，就没有必要引入多 Agent 或复杂图结构。架构应由任务特征推出。
忽略简单基线	没有和单次问答、Chain 或人工流程比较，就无法证明复杂架构真的带来收益。
只看成功样例	架构章节尤其要看失败样本。一个架构在成功样例中很漂亮，不代表它在边界条件下可靠。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

16.19 补充注记

对“Human-in-the-loop Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

16.20 小结

本章的核心结论是：掌握人在回路中的核心思想、适用场景、工程代价和私人定制化落地方式。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第十六章 练习

1. 为一个论文阅读任务判断是否需要 Human-in-the-loop Agent。
2. 写出 Human-in-the-loop Agent 的状态字段和停止条件。
3. 设计一个失败样本，并说明如何从日志中定位问题。

第三部分

主流框架与工程实践

第十七章 原生 SDK：从零搭建 Agent

内容提要

- ❑ 理解底层模型接口在 Agent 工程中的位置，知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：主流框架与工程实践。

17.1 问题引入

用 Claude、OpenAI 或其他模型 SDK 直接实现消息、工具、结构化输出、流式响应、重试和成本统计。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 17.1 (底层模型接口)

底层模型接口指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：用 Claude、OpenAI 或其他模型 SDK 直接实现消息、工具、结构化输出、流式响应、重试和成本统计。

命题 17.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

17.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

17.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def call_model(client, messages, tools):
    response = client.messages.create(
        model='claude-opus-4-8',
        messages=messages,
        tools=tools,
```

```

        max_tokens=1200,
    )
    return normalize_response(response)

```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

17.4 深入讲解

原生 SDK 是理解 Agent 的最好起点。它让开发者直接面对 messages、tools、tool result、structured output、streaming、timeout、retry 和 token accounting。

即使最终使用框架，也建议保留一个原生 SDK baseline。框架升级或行为异常时，baseline 能帮助判断问题来自模型、提示词、工具还是框架抽象。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

17.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

17.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

17.7 失败模式与调试

常见失败模式如下：

- 控制力强，但状态管理、可观测性和工具编排都要自己实现。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

17.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

17.9 私人定制化实践

私人 Agent 使用原生 SDK：从零搭建 Agent 时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人工作流。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

17.10 本章例题

例题 17.1 原生工具调用 用 SDK 直接传入 messages 和 tools，模型返回 tool_use 后由本地程序执行工具。

解 原生路线最透明，适合作为所有框架章节的对照基线。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

17.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

17.12 进阶练习

1. 实现一个只支持 read_file 工具的原生 SDK Agent，并记录每次 tool_use。
2. 给模型调用增加 timeout、retry 和 token 统计。
3. 比较结构化输出和自由文本输出在解析稳定性上的差异。

17.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

17.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

17.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

17.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

17.17 案例展开与验收标准

为了避免把“原生 SDK：从零搭建 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

17.18 常见误区与反例

学习“原生 SDK：从零搭建 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

17.19 小结

本章的核心结论是：理解底层模型接口在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第十七章 练习

- 1. 说明原生 SDK：从零搭建 Agent 适合哪类私人 Agent。
- 2. 列出原生 SDK：从零搭建 Agent 隐藏的一个底层细节。
- 3. 用同一任务比较原生 SDK 和该方案的复杂度。

第十八章 LangChain：组件化 Agent 原型

内容提要

- 理解组件生态在 Agent 工程中的位置，知道何时使用、何时不用。
- 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- 所属部分：主流框架与工程实践。

18.1 问题引入

用模型、提示词、解析器、检索器和工具快速搭建原型，适合 RAG 和链式组合。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 18.1 (组件生态)

组件生态指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：用模型、提示词、解析器、检索器和工具快速搭建原型，适合 RAG 和链式组合。

命题 18.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

18.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

18.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def build_langchain_rag(prompt, model, retriever, parser):
    def fetch(inputs):
        return {'question': inputs['question'], 'docs': retriever.invoke(inputs['question'])}
    chain = fetch | prompt | model | parser
    return chain
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

18.4 深入讲解

LangChain 的优势在于组件生态和组合表达。Prompt、Model、Retriever、Parser 和 Tool 都可以作为可替换组件。

它的代价是抽象层较多。调试时不要停留在 chain 这个整体，而要逐段打印输入输出，确认每个 Runnable 接收到的数据结构。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

18.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

18.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

18.7 失败模式与调试

常见失败模式如下：

- 抽象层会隐藏细节，调试时必须回到状态和调用链。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

18.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 **Agent** 只能依赖主观感觉，很难持续改进。

18.9 私人定制化实践

私人 **Agent** 使用 **LangChain**：组件化 **Agent** 原型时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人 workflow。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

18.10 本章例题

例题 18.1 LCEL 管线 把 `prompt`、`model`、`parser` 串起来处理固定格式的论文摘要任务。

解 **LangChain** 的优势是组合生态；调试时仍要查看每个组件的输入输出。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

18.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	<code>schema</code> 或说明不清	收窄工具并加入校验
无法复盘	缺少 <code>trace</code>	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

18.12 进阶练习

1. 把“**LCEL 管线**”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

18.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 **Agent** 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

18.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

18.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

18.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

18.17 案例展开与验收标准

为了避免把“LangChain：组件化 Agent 原型”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

18.18 常见误区与反例

学习“LangChain：组件化 Agent 原型”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

18.19 补充笔记

对“LangChain：组件化 Agent 原型”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

18.20 小结

本章的核心结论是：理解组件生态在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第十八章 练习

1. 说明 LangChain：组件化 Agent 原型适合哪类私人 Agent。
2. 列出 LangChain：组件化 Agent 原型隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第十九章 LangGraph: 可控 Agent 状态机

内容提要

- ❑ 理解图执行框架在 Agent 工程中的位置, 知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分: 主流框架与工程实践。

19.1 问题引入

用 State、Node、Edge、条件边和 checkpoint 构建可恢复 Agent。框架和基础设施章节不应写成 API 罗列, 而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中, 上式的某些项会更突出。例如 RAG 强调知识与证据, LangGraph 强调状态和边, Human-in-the-loop 强调人工反馈与权限边界。

定义 19.1 (图执行框架)

图执行框架指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于: 用 State、Node、Edge、条件边和 checkpoint 构建可恢复 Agent。

命题 19.1 (框架透明原则)

引入框架后, 仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

19.2 执行流程

一个可落地的实现通常包含以下步骤:

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变, 但它们共同保证系统不是一次性问答, 而是可观察、可修正、可评估的执行过程。

19.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现, 而是用于说明状态、工具和评估如何进入同一个闭环。

```
class ResearchState(dict):
    pass

def retrieve(state):
    state['docs'] = search(state['question'])
```

```

return state

def should_continue(state):
    return 'draft' if state['docs'] else 'ask_human'

```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

19.4 深入讲解

LangGraph 的核心是状态图。State 保存任务数据，Node 修改状态，Edge 决定下一步，conditional edge 根据状态分支，checkpoint 负责恢复。

这种模型非常适合 Agent，因为 Agent 本来就是状态驱动系统。把隐含在 prompt 中的流程搬到图里，能显著提高可观察性和可恢复性。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

19.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

19.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

19.7 失败模式与调试

常见失败模式如下：

- 如果任务没有分支和循环，LangGraph 可能过重。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

补充说明：LangGraph 中的 conditional edge 是根据 State 选择下一节点的关键机制，常与 checkpoint 一起用于失败恢复。

19.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

19.9 私人订制化实践

私人 Agent 使用 LangGraph：可控 Agent 状态机时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人 workflow。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

19.10 本章例题

例题 19.1 条件边 RAG 问答图中，retrieve 后若无文档则走 ask_human，有文档则走 draft，draft 后走 citation_check。

解 State、Node、Edge 的显式化使流程比普通回调更容易审计。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

19.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

19.12 进阶练习

1. 画出一个 retrieve–draft–check–revise 的 LangGraph 状态图。
2. 设计 checkpoint 恢复场景：编译失败后从 revise 节点继续。
3. 说明条件边比 if/else 回调更容易审计的原因。

19.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

19.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

19.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

19.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

19.17 案例展开与验收标准

为了避免把“LangGraph：可控 Agent 状态机”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具

如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

19.18 常见误区与反例

学习“LangGraph：可控 Agent 状态机”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

19.19 小结

本章的核心结论是：理解图执行框架在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

 第十九章 练习

1. 说明 LangGraph: 可控 Agent 状态机适合哪类私人 Agent。
2. 列出 LangGraph: 可控 Agent 状态机隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第二十章 CrewAI：角色分工式多 Agent

内容提要

- ❑ 理解角色与任务编排在 Agent 工程中的位置，知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：主流框架与工程实践。

20.1 问题引入

用角色、任务和流程快速表达协作型 Agent，适合教学、研究原型和内容生产流程。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 20.1 (角色与任务编排)

角色与任务编排指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：用角色、任务和流程快速表达协作型 Agent，适合教学、研究原型和内容生产流程。

命题 20.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

20.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

20.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
researcher = Agent(role='Researcher', goal='collect evidence')
writer = Agent(role='Writer', goal='draft answer with citations')
reviewer = Agent(role='Reviewer', goal='find unsupported claims')
crew = Crew(agents=[researcher, writer, reviewer], tasks=tasks)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

20.4 深入讲解

CrewAI 适合把任务快速表达为角色、Task 和 Crew。它对教学和原型很友好，尤其适合研究员、写作者、审稿人这类直观分工。

但角色描述不能代替工程接口。每个 Task 仍需要输入、输出、验收标准和失败处理；否则多 Agent 只会产生更长的对话。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

20.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

20.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

20.7 失败模式与调试

常见失败模式如下：

- 角色命名不能代替真实接口和评估。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

补充说明：CrewAI 中 Task 应写清 expected output、tools 和 context，否则 Agent 角色无法稳定协作。

20.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

20.9 私人定制化实践

私人 Agent 使用 CrewAI：角色分工式多 Agent 时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人 workflow。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

20.10 本章例题

例题 20.1 角色协作 研究员收集材料，写作者生成报告，审稿人提出问题。

解 CrewAI 适合快速表达角色分工，但角色职责仍要落实到任务输入输出。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

20.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

20.12 进阶练习

1. 把“角色协作”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

20.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

20.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

20.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

20.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

20.17 案例展开与验收标准

为了避免把“CrewAI：角色分工式多 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

20.18 常见误区与反例

学习“CrewAI：角色分工式多 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

20.19 小结

本章的核心结论是：理解角色与任务编排在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第二十章 练习

- 1. 说明 CrewAI：角色分工式多 Agent 适合哪类私人 Agent。
- 2. 列出 CrewAI：角色分工式多 Agent 隐藏的一个底层细节。
- 3. 用同一任务比较原生 SDK 和该方案的复杂度。

第二十一章 AutoGen：对话式多 Agent 系统

内容提要

- ❑ 理解对话协作在 Agent 工程中的位置，知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：主流框架与工程实践。

21.1 问题引入

通过 Assistant、UserProxy、群聊、代码执行和终止条件研究多 Agent 对话。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 21.1 (对话协作)

对话协作指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：通过 Assistant、UserProxy、群聊、代码执行和终止条件研究多 Agent 对话。

命题 21.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

21.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

21.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
max_round = 8
assistant = AssistantAgent(name='coder')
proxy = UserProxyAgent(name='runner', code_execution_config={'work_dir': 'sandbox'})
proxy.initiate_chat(assistant, message='write and test a parser', max_turns=max_round)
# production systems also need max rounds and termination rules
```


 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

21.4 深入讲解

AutoGen 强调多 Agent 对话。AssistantAgent 可以提出方案，UserProxyAgent 可以运行代码或代表环境反馈结果，群聊可以组织多方协作。

生产使用时必须设置 `max_round`、终止条件、沙箱目录和工具白名单。否则对话式系统容易成本失控或执行不受控代码。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

21.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

21.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

21.7 失败模式与调试

常见失败模式如下：

- 对话越自由，越需要沙箱、预算和终止规则。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

补充说明：AutoGen 示例应显式设置 `max_round`，避免 AssistantAgent 与 UserProxyAgent 在失败时无限对话。

21.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

21.9 私人定制化实践

私人 Agent 使用 AutoGen：对话式多 Agent 系统时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人 workflow。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

21.10 本章例题

例题 21.1 代码对话实验 Assistant 写代码，UserProxy 在沙箱中运行测试并把错误返回。

解 必须设置最大轮数、工作目录和终止条件，否则对话可能无限延长。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

21.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

21.12 进阶练习

1. 把“代码对话实验”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

21.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

21.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

21.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

21.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

21.17 案例展开与验收标准

为了避免把“AutoGen：对话式多 Agent 系统”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

21.18 常见误区与反例

学习“AutoGen：对话式多 Agent 系统”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

21.19 小结

本章的核心结论是：理解对话协作在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第二十一章 练习

1. 说明 AutoGen：对话式多 Agent 系统适合哪类私人 Agent。
2. 列出 AutoGen：对话式多 Agent 系统隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第二十二章 MCP: Agent 的外部工具生态

内容提要

- ❑ 理解标准化工具协议在 Agent 工程中的位置，知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：主流框架与工程实践。

22.1 问题引入

通过 Model Context Protocol 把文件、数据库、浏览器、GitHub、飞书等能力接入模型。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 22.1 (标准化工具协议)

标准化工具协议指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：通过 Model Context Protocol 把文件、数据库、浏览器、GitHub、飞书等能力接入模型。

命题 22.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

22.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

22.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
@server.tool()
def read_project_note(path: str) -> dict:
    if not path.startswith('notes/'):
        raise PermissionError(path)
    return {'text': Path(path).read_text(encoding='utf-8')}
```

```
server.run()
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

22.4 深入讲解

MCP 的意义是把工具接入标准化。模型客户端不必为每个数据源写专用胶水，MCP server 负责暴露 tools、resources 和 prompts。

但 MCP 不负责完整任务规划。它解决的是“如何连接工具”，不是“如何决定下一步”。状态机、评估和权限仍然要在 Agent 系统中设计。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

22.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

22.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

22.7 失败模式与调试

常见失败模式如下：

- MCP 是协议层，不是完整 Agent 编排框架。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

22.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

22.9 私人定制化实践

私人 Agent 使用 MCP：Agent 的外部工具生态时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人工作流。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

22.10 本章例题

例题 22.1 本地笔记 MCP MCP server 暴露 read_note 和 search_notes，Agent 通过标准协议访问本地知识。

解 MCP 解决工具接入标准化，不替代状态机、评估和权限设计。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

22.11 失败模式矩阵

症状	可能原因	修复方式
工具过宽	MCP server 暴露危险能力	按能力拆分工具并鉴权
资源泄露	返回了无关文件	resource 层做路径白名单
协议误用	把 MCP 当编排框架	仍用状态机管理流程

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

22.12 进阶练习

1. 把“本地笔记 MCP”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

22.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

22.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

22.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

22.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

22.17 案例展开与验收标准

为了避免把“MCP: Agent 的外部工具生态”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

22.18 常见误区与反例

学习“MCP: Agent 的外部工具生态”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

22.19 补充注记

对“MCP: Agent 的外部工具生态”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

22.20 小结

本章的核心结论是：理解标准化工具协议在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第二十二章 练习

1. 说明 MCP: Agent 的外部工具生态适合哪类私人 Agent。
2. 列出 MCP: Agent 的外部工具生态隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第二十三章 向量数据库与知识库工程

内容提要

- ❑ 理解 RAG 基础设施在 Agent 工程中的位置, 知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分: 主流框架与工程实践。

23.1 问题引入

讲解 embedding、chunk、metadata、混合检索、重排、权限过滤和增量索引。框架和基础设施章节不应写成 API 罗列, 而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中, 上式的某些项会更突出。例如 RAG 强调知识与证据, LangGraph 强调状态和边, Human-in-the-loop 强调人工反馈与权限边界。

定义 23.1 (RAG 基础设施)

RAG 基础设施指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于: 讲解 embedding、chunk、metadata、混合检索、重排、权限过滤和增量索引。

命题 23.1 (框架透明原则)

引入框架后, 仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

23.2 执行流程

一个可落地的实现通常包含以下步骤:

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变, 但它们共同保证系统不是一次性问答, 而是可观察、可修正、可评估的执行过程。

23.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现, 而是用于说明状态、工具和评估如何进入同一个闭环。

```
def index_document(doc, embedder, store):
    for chunk in split_by_heading(doc):
        vector = embedder.embed(chunk.text)
        store.upsert(id=chunk.id, vector=vector, metadata=chunk.metadata)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

23.4 深入讲解

向量数据库工程包括 embedding、chunk、metadata、index、hybrid search、rerank 和权限过滤。任何一个环节薄弱，RAG 质量都会下降。

私人知识库尤其依赖 metadata。项目名、文件路径、章节标题、时间、作者和权限字段，能帮助系统避免把无关或越权资料混入答案。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

23.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

23.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

23.7 失败模式与调试

常见失败模式如下：

- 向量库不能替代数据清洗和评估。
- 版本变化导致示例失效。
- 只会调用框架 API，却不能解释 Agent 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

23.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

23.9 私人订制化实践

私人 Agent 使用向量数据库与知识库工程时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人工作流。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

23.10 本章例题

例题 23.1 章节级索引 讲义按 chapter、section、page 保存 metadata，检索时可按项目和章节过滤。

解 好的 metadata 往往比更换向量库更能提升可用性。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

23.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

23.12 进阶练习

1. 把“章节级索引”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

23.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

23.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

23.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

23.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

23.17 案例展开与验收标准

为了避免把“向量数据库与知识库工程”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

23.18 常见误区与反例

学习“向量数据库与知识库工程”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

23.19 补充注记

对“向量数据库与知识库工程”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

23.20 小结

本章的核心结论是：理解 RAG 基础设施在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第二十三章 练习

1. 说明向量数据库与知识库工程适合哪类私人 Agent。
2. 列出向量数据库与知识库工程隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第二十四章 Agent 日志、调试与可观测性

内容提要

- ❑ 理解可观测工程在 Agent 工程中的位置，知道何时使用、何时不用。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：主流框架与工程实践。

24.1 问题引入

记录 trace、span、工具输入输出、prompt 版本、成本、延迟和失败样本。框架和基础设施章节不应写成 API 罗列，而应说明它解决的工程问题、隐藏的控制细节和可替换边界。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 24.1 (可观测工程)

可观测工程指一组用于降低 Agent 工程复杂度的抽象或基础设施。它的价值在于：记录 trace、span、工具输入输出、prompt 版本、成本、延迟和失败样本。

命题 24.1 (框架透明原则)

引入框架后，仍应能说清底层状态如何流动、工具如何执行、错误如何恢复、评估如何记录。

24.2 执行流程

一个可落地的实现通常包含以下步骤：

- 先用原生接口写出最小闭环。
- 识别重复工程问题。
- 选择框架抽象替代手写部分。
- 保留日志和回退路径。
- 用同一任务比较框架前后的收益。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

24.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
with tracer.span('tool_call') as span:
    span.set('tool.name', call.name)
    span.set('tool.args', redact(call.args))
    result = execute(call)
    span.set('tool.status', 'ok')
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

24.4 深入讲解

可观测性要记录 `trace`、`span`、`prompt` 版本、工具输入输出、`latency`、`token`、成本和错误。普通 `print` 日志很难解释多步 `Agent` 的失败。

一次失败最好能被 `replay`。若开发者无法重放同一输入和同一状态，就很难判断修复是否真正有效。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，`Agent` 才能从演示走向可维护系统。

24.5 工程实现要点

- 固定依赖版本。
- 把框架对象包在自己的接口之后。
- 不要让业务逻辑散落在回调中。
- 记录每次模型和工具调用。
- 为框架升级保留测试集。

24.6 适用与不适用

适用场景：

- 需要快速原型。
- 需要复用生态组件。
- 需要图编排、检索、观测或协议接入。

不适用或需要谨慎的场景：

- 一两个函数就能完成的任务。
- 团队无法调试框架内部行为。
- 框架抽象与任务结构不匹配。

24.7 失败模式与调试

常见失败模式如下：

- 普通日志不足以调试多步 `Agent`。
- 版本变化导致示例失效。
- 只会调用框架 `API`，却不能解释 `Agent` 的状态和错误。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

补充说明：`latency` 应按端到端、模型调用、检索、工具调用四类分别记录，否则无法定位性能瓶颈。

24.8 评估指标

本章相关系统至少应记录：

- 开发时间。
- 调试时间。
- 运行成本和延迟。
- 失败回放能力。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 **Agent** 只能依赖主观感觉，很难持续改进。

24.9 私人定制化实践

私人 **Agent** 使用 **Agent** 日志、调试与可观测性时，应把用户数据路径、工具权限、模型供应商和日志位置配置化。这样更换框架时不会丢失个人工作流。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

24.10 本章例题

例题 24.1 一次失败回放 用户得到错误答案后，开发者查看 **trace**：检索命中了旧文档，**rerank** 未生效。

解 可观测性让失败从“模型胡说”变成可定位的工程问题。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

24.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

24.12 进阶练习

1. 把“一次失败回放”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

24.13 设计边界与选择表

框架篇的章节要把框架还原成底层抽象。读者学完后应能说清框架替自己管理了哪些状态、隐藏了哪些调用、增加了哪些调试成本。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 **Agent** 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

24.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

24.15 本章小项目

用同一个小型 RAG 问答任务分别写原生版本和框架版本。比较代码行数、日志可见性、错误定位难度、依赖复杂度和运行成本。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

24.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

24.17 案例展开与验收标准

为了避免把“Agent 日志、调试与可观测性”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把框架概念还原为模型调用、状态流转、工具执行和日志记录。
- 能写出最小安装或伪安装边界，并指出哪些 API 容易随版本变化。
- 能说明框架隐藏了哪些细节，以及调试时如何重新暴露这些细节。
- 能用同一任务比较原生 SDK 与该框架的成本、代码量和可观测性。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

24.18 常见误区与反例

学习“Agent 日志、调试与可观测性”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把框架当答案	框架只能降低部分工程成本，不能替代任务定义、状态设计、权限控制和评估。
忽略版本变化	Agent 框架迭代很快。讲义中的代码应服务概念，真实项目必须固定依赖并保留升级测试。
调试停在抽象层	当框架行为异常时，要回到模型输入、工具输出、状态变化和 trace，而不是只修改上层配置。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

24.19 补充注记

对“Agent 日志、调试与可观测性”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

24.20 小结

本章的核心结论是：理解可观测工程在 Agent 工程中的位置，知道何时使用、何时不用。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第二十四章 练习

1. 说明 Agent 日志、调试与可观测性适合哪类私人 Agent。
2. 列出 Agent 日志、调试与可观测性隐藏的一个底层细节。
3. 用同一任务比较原生 SDK 和该方案的复杂度。

第四部分

私人订制版 Agent 项目实战

第二十五章 私人知识库 Agent

内容提要

- ❑ 完成一个个人知识管理项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。
- ❑ 所属部分：私人订制化 Agent 项目实战。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。

25.1 问题引入

构建能读取本地资料、检索引用、沉淀偏好的私人知识库 Agent。项目章节的重点是把前文架构组合起来，而不是演示一个孤立 demo。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 25.1 (个人知识管理项目)

个人知识管理项目是围绕用户真实资料、工具和反馈构建的 Agent 项目。它要求系统知道任务边界、数据来源、权限等级和验收标准。

命题 25.1 (项目闭环原则)

每个项目必须同时交付任务流程、数据流、工具权限、评估集和失败处理，而不只是一次成功演示。

25.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义用户场景和输入输出。
- 列出数据源、工具和权限。
- 选择 RAG、Workflow、Memory 或 Multi-Agent 等组合架构。
- 实现最小可运行闭环。
- 用真实失败样本修订。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

25.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def answer_from_private_kb(question, user, retriever, model):
    docs = retriever.search(question, filters={'owner': user.id, 'allowed': True})
    answer = model.answer(question, docs)
    return {'answer': answer, 'sources': [d.source for d in docs]}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

25.4 深入讲解

私人知识库 Agent 的第一阶段应只读。它读取文档、建立索引、回答问题并给出来源。只有当检索和引用质量稳定后，才考虑自动整理和改写笔记。

数据布局要从一开始设计：原始文件、抽取文本、chunk、embedding、metadata、问答日志和用户反馈最好分开保存。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

25.5 工程实现要点

- 先做只读版本。
- 再加入写操作和人工确认。
- 保存 trace 和中间产物。
- 把用户偏好写成可编辑配置。
- 每次扩展都重新运行评估任务。

25.6 适用与不适用

适用场景：

- 用户有稳定重复的工作流。
- 数据源可整理。
- 任务结果可评价。

不适用或需要谨慎的场景：

- 用户需求尚不清楚。
- 数据没有权限边界。
- 项目只有一次性演示价值。

25.7 失败模式与调试

常见失败模式如下：

- 知识库必须处理权限、去重和来源追踪。
- 没有把用户反馈沉淀为配置或评估。
- 工具可以修改外部系统但缺少审批。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

25.8 评估指标

本章相关系统至少应记录：

- 任务交付质量。
- 引用或测试通过率。
- 人工修改量。
- 复用次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

25.9 私人订制化实践

私人知识库 Agent 的私人订制化来自用户自己的资料、术语、风格、项目结构和工具习惯。系统应允许用户查看这些配置，并随时撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

25.10 本章例题

例题 25.1 个人笔记问答 系统读取用户 Markdown、PDF 和项目记录，回答时给出来源路径和片段。

解 只读版本先验证检索质量，再考虑自动归档或改写笔记。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

25.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

25.12 进阶练习

1. 为自己的笔记目录设计 metadata schema。
2. 构造三个应拒答的问题，例如越权项目或无来源问题。
3. 写出知识库 Agent 的验收表。

25.13 设计边界与选择表

项目篇的章节要强调真实数据、真实权限和真实验收。私人订制化不是让模型换一种语气，而是把用户资料、工具、偏好和反馈闭环系统化。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

25.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

25.15 本章小项目

为自己的一个真实工作流写项目卡片：输入是什么，输出是什么，哪些工具只读，哪些工具需要审批，什么指标表示成功，失败后如何回滚。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

25.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

25.17 案例展开与验收标准

为了避免把“私人知识库 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能写出真实数据来源、文件布局、权限表和验收任务。
- 能先实现只读闭环，再说明何时加入写操作和人工确认。
- 能把用户反馈转成配置、记忆候选或评估样本，而不是只留在聊天记录中。
- 能报告项目失败样本，并给出回滚或修复路径。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

25.18 常见误区与反例

学习“私人知识库 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把 demo 当产品	一次成功演示不能说明私人 Agent 可长期使用。真实项目要面对脏数据、权限、重复任务和用户纠正。
忽略用户控制权	私人订制化必须允许用户查看、修改、删除偏好和记忆，否则个性化会变成黑箱画像。
直接开放写操作	项目第一版应尽量只读。写文件、发消息、提交代码等动作必须在验收稳定后逐步开放。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

25.19 补充注记

对“私人知识库 Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

25.20 小结

本章的核心结论是：完成一个个人知识管理项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第二十五章 练习

1. 为私人知识库 Agent 写一页项目需求。
2. 列出私人知识库 Agent 的三个只读工具和两个写工具。
3. 设计一个验收任务，并说明什么算通过。

第二十六章 论文阅读 Agent

内容提要

- ❑ 完成一个科研阅读项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人定制化。

❑ 所属部分：私人定制化 Agent 项目实战。

❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。

26.1 问题引入

实现搜索、解析、贡献提取、实验检查和阅读卡片生成。项目章节的重点是把前文架构组合起来，而不是演示一个孤立 demo。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 26.1 (科研阅读项目)

科研阅读项目是围绕用户真实资料、工具和反馈构建的 Agent 项目。它要求系统知道任务边界、数据来源、权限等级和验收标准。

命题 26.1 (项目闭环原则)

每个项目必须同时交付任务流程、数据流、工具权限、评估集和失败处理，而不只是一次成功演示。

26.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义用户场景和输入输出。
- 列出数据源、工具和权限。
- 选择 RAG、Workflow、Memory 或 Multi-Agent 等组合架构。
- 实现最小可运行闭环。
- 用真实失败样本修订。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

26.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def paper_card(pdf, extractor, model):
    sections = extractor.sections(pdf)
    return {
        'contribution': model.extract(sections['introduction']),
        'method': model.extract(sections['method']),
```

```
'experiments': model.extract(sections['experiments']),
'limitations': model.extract(sections.get('limitations', ''))
}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

26.4 深入讲解

论文阅读 Agent 应避免摘要崇拜。贡献、方法、实验和局限分别来自不同章节，不能只根据 abstract 生成完整评价。

阅读卡片应强制包含证据栏。若某篇论文没有公开代码、没有消融或没有局限讨论，Agent 应明确标注缺失，而不是补写漂亮结论。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

26.5 工程实现要点

- 先做只读版本。
- 再加入写操作和人工确认。
- 保存 trace 和中间产物。
- 把用户偏好写成可编辑配置。
- 每次扩展都重新运行评估任务。

26.6 适用与不适用

适用场景：

- 用户有稳定重复的工作流。
- 数据源可整理。
- 任务结果可评价。

不适用或需要谨慎的场景：

- 用户需求尚不清楚。
- 数据没有权限边界。
- 项目只有一次性演示价值。

26.7 失败模式与调试

常见失败模式如下：

- 论文总结不能只看摘要，实验和局限必须有证据。
- 没有把用户反馈沉淀为配置或评估。
- 工具可以修改外部系统但缺少审批。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

26.8 评估指标

本章相关系统至少应记录：

- 任务交付质量。
- 引用或测试通过率。
- 人工修改量。
- 复用次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 **Agent** 只能依赖主观感觉，很难持续改进。

26.9 私人订制化实践

论文阅读 **Agent** 的私人订制化来自用户自己的资料、术语、风格、项目结构和工具习惯。系统应允许用户查看这些配置，并随时撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

26.10 本章例题

例题 26.1 论文卡片 **Agent** 生成贡献、方法、实验、局限和可复现性五栏，并为每栏标注证据段落。

解 缺少实验证据时，不应把摘要中的 **claim** 当成实验结论。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

26.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

26.12 进阶练习

1. 把“论文卡片”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

26.13 设计边界与选择表

项目篇的章节要强调真实数据、真实权限和真实验收。私人订制化不是让模型换一种语气，而是把用户资料、工具、偏好和反馈闭环系统化。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 **Agent** 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

26.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

26.15 本章小项目

为自己的一个真实工作流写项目卡片：输入是什么，输出是什么，哪些工具只读，哪些工具需要审批，什么指标表示成功，失败后如何回滚。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

26.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

26.17 案例展开与验收标准

为了避免把“论文阅读 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能写出真实数据来源、文件布局、权限表和验收任务。
- 能先实现只读闭环，再说明何时加入写操作和人工确认。
- 能把用户反馈转成配置、记忆候选或评估样本，而不是只留在聊天记录中。
- 能报告项目失败样本，并给出回滚或修复路径。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

26.18 常见误区与反例

学习“论文阅读 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把 demo 当产品	一次成功演示不能说明私人 Agent 可长期使用。真实项目要面对脏数据、权限、重复任务和用户纠正。
忽略用户控制权	私人订制化必须允许用户查看、修改、删除偏好和记忆，否则个性化会变成黑箱画像。
直接开放写操作	项目第一版应尽量只读。写文件、发消息、提交代码等动作必须在验收稳定后逐步开放。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

26.19 补充笔记

对“论文阅读 Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

26.20 小结

本章的核心结论是：完成一个科研阅读项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第二十六章 练习

1. 为论文阅读 Agent 写一页项目需求。
2. 列出论文阅读 Agent 的三个只读工具和两个写工具。
3. 设计一个验收任务，并说明什么算通过。

第二十七章 讲义写作 Agent

内容提要

- 完成一个长文档写作项目的系统设计,从目标、数据流、工具、评估和安全边界落实私人订制的。
- 所属部分: 私人定制化 Agent 项目实战。
- 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。

27.1 问题引入

把目录规划、章节生成、风格检查和 LaTeX 编译串成 workflow。项目章节的重点是把前文架构组合起来,而不是演示一个孤立 demo。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中,上式的某些项会更突出。例如 RAG 强调知识与证据, LangGraph 强调状态和边, Human-in-the-loop 强调人工反馈与权限边界。

定义 27.1 (长文档写作项目)

长文档写作项目是围绕用户真实资料、工具和反馈构建的 Agent 项目。它要求系统知道任务边界、数据来源、权限等级和验收标准。

命题 27.1 (项目闭环原则)

每个项目必须同时交付任务流程、数据流、工具权限、评估集和失败处理,而不只是一次成功演示。

27.2 执行流程

一个可落地的实现通常包含以下步骤:

- 定义用户场景和输入输出。
- 列出数据源、工具和权限。
- 选择 RAG、Workflow、Memory 或 Multi-Agent 等组合架构。
- 实现最小可运行闭环。
- 用真实失败样本修订。

这些步骤的顺序可以因任务而变,但它们共同保证系统不是一次性问答,而是可观察、可修正、可评估的执行过程。

27.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现,而是用于说明状态、工具和评估如何进入同一个闭环。

```
def write_chapter(topic, planner, writer, checker, compiler):
    outline = planner(topic)
    tex = writer(outline)
    issues = checker(tex)
    if issues:
```

```
tex = writer.revise(tex, issues)
return compiler(tex)
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

27.4 深入讲解

讲义写作 Agent 是长文档工作流。它需要目录规划、章节草稿、风格审核、交叉引用检查、LaTeX 编译和 PDF 视觉检查。

最容易失败的是一次性生成太长内容。更好的做法是逐章写作、逐章编译、逐章审核，并把风格规范作为稳定上下文。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

27.5 工程实现要点

- 先做只读版本。
- 再加入写操作和人工确认。
- 保存 trace 和中间产物。
- 把用户偏好写成可编辑配置。
- 每次扩展都重新运行评估任务。

27.6 适用与不适用

适用场景：

- 用户有稳定重复的工作流。
- 数据源可整理。
- 任务结果可评价。

不适用或需要谨慎的场景：

- 用户需求尚不清楚。
- 数据没有权限边界。
- 项目只有一次性演示价值。

27.7 失败模式与调试

常见失败模式如下：

- 长文写作必须有质量标准和编译验证。
- 没有把用户反馈沉淀为配置或评估。
- 工具可以修改外部系统但缺少审批。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

27.8 评估指标

本章相关系统至少应记录：

- 任务交付质量。
- 引用或测试通过率。
- 人工修改量。
- 复用次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 **Agent** 只能依赖主观感觉，很难持续改进。

27.9 私人订制化实践

讲义写作 **Agent** 的私人订制化来自用户自己的资料、术语、风格、项目结构和工具习惯。系统应允许用户查看这些配置，并随时撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

27.10 本章例题

例题 27.1LaTeX 编译闭环 写作后立即运行编译，若失败则根据 log 修复；若内容缺例题则返回写作节点补充。

解 长文档 **Agent** 必须把编译和风格审核纳入流程，而不是只生成文本。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

27.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

27.12 进阶练习

1. 把“LaTeX 编译闭环”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

27.13 设计边界与选择表

项目篇的章节要强调真实数据、真实权限和真实验收。私人订制化不是让模型换一种语气，而是把用户资料、工具、偏好和反馈闭环系统化。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 **Agent** 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

27.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

27.15 本章小项目

为自己的一个真实工作流写项目卡片：输入是什么，输出是什么，哪些工具只读，哪些工具需要审批，什么指标表示成功，失败后如何回滚。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

27.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

27.17 案例展开与验收标准

为了避免把“讲义写作 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限制检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能写出真实数据来源、文件布局、权限表和验收任务。
- 能先实现只读闭环，再说明何时加入写操作和人工确认。
- 能把用户反馈转成配置、记忆候选或评估样本，而不是只留在聊天记录中。
- 能报告项目失败样本，并给出回滚或修复路径。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

27.18 常见误区与反例

学习“讲义写作 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把 demo 当产品	一次成功演示不能说明私人 Agent 可长期使用。真实项目要面对脏数据、权限、重复任务和用户纠正。
忽略用户控制权	私人订制化必须允许用户查看、修改、删除偏好和记忆，否则个性化会变成黑箱画像。
直接开放写操作	项目第一版应尽量只读。写文件、发消息、提交代码等动作必须在验收稳定后逐步开放。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

27.19 补充笔记

对“讲义写作 Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

27.20 小结

本章的核心结论是：完成一个长文档写作项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第二十七章 练习

1. 为讲义写作 Agent 写一页项目需求。
2. 列出讲义写作 Agent 的三个只读工具和两个写工具。
3. 设计一个验收任务，并说明什么算通过。

第二十八章 代码开发 Agent

内容提要

- ❑ 完成一个软件工程项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人定制化。
- ❑ 所属部分：私人定制化 Agent 项目实战。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。

28.1 问题引入

让 Agent 读项目、写计划、改代码、跑测试和审查 diff。项目章节的重点是把前文架构组合起来，而不是演示一个孤立 demo。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 28.1 (软件工程项目)

软件工程项目是围绕用户真实资料、工具和反馈构建的 Agent 项目。它要求系统知道任务边界、数据来源、权限等级和验收标准。

命题 28.1 (项目闭环原则)

每个项目必须同时交付任务流程、数据流、工具权限、评估集和失败处理，而不只是一次成功演示。

28.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义用户场景和输入输出。
- 列出数据源、工具和权限。
- 选择 RAG、Workflow、Memory 或 Multi-Agent 等组合架构。
- 实现最小可运行闭环。
- 用真实失败样本修订。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

28.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def code_agent(issue, repo):
    context = repo.inspect(issue.paths)
    plan = make_plan(issue, context)
    patch = implement(plan)
    test_result = repo.run_tests(plan.tests)
```



```
return {'patch': patch, 'tests': test_result}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

28.4 深入讲解

代码开发 Agent 必须尊重软件工程节奏：理解问题、写计划、最小修改、运行测试、审查 diff、报告风险。

如果没有测试，Agent 应先建议或补充测试。若修改涉及删除、迁移或外部发布，应请求人工确认。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

28.5 工程实现要点

- 先做只读版本。
- 再加入写操作和人工确认。
- 保存 trace 和中间产物。
- 把用户偏好写成可编辑配置。
- 每次扩展都重新运行评估任务。

28.6 适用与不适用

适用场景：

- 用户有稳定重复的工作流。
- 数据源可整理。
- 任务结果可评价。

不适用或需要谨慎的场景：

- 用户需求尚不清楚。
- 数据没有权限边界。
- 项目只有一次性演示价值。

28.7 失败模式与调试

常见失败模式如下：

- 代码 Agent 必须尊重测试、版本控制和人工确认。
- 没有把用户反馈沉淀为配置或评估。
- 工具可以修改外部系统但缺少审批。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

28.8 评估指标

本章相关系统至少应记录：

- 任务交付质量。
- 引用或测试通过率。
- 人工修改量。
- 复用次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

28.9 私人订制化实践

代码开发 Agent 的私人订制化来自用户自己的资料、术语、风格、项目结构和工具习惯。系统应允许用户查看这些配置，并随时撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

28.10 本章例题

例题 28.1 修复一个 bug Agent 先读测试和相关代码，再写计划，修改最小文件，运行测试并报告 diff。

解 没有测试的修改只能算建议；通过测试也要做代码审查。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

28.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

28.12 进阶练习

1. 把“修复一个 bug”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

28.13 设计边界与选择表

项目篇的章节要强调真实数据、真实权限和真实验收。私人订制化不是让模型换一种语气，而是把用户资料、工具、偏好和反馈闭环系统化。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。

3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

28.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

28.15 本章小项目

为自己的一个真实工作流写项目卡片：输入是什么，输出是什么，哪些工具只读，哪些工具需要审批，什么指标表示成功，失败后如何回滚。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

28.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

28.17 案例展开与验收标准

为了避免把“代码开发 Agent”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。

- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能写出真实数据来源、文件布局、权限表和验收任务。
- 能先实现只读闭环，再说明何时加入写操作和人工确认。
- 能把用户反馈转成配置、记忆候选或评估样本，而不是只留在聊天记录中。
- 能报告项目失败样本，并给出回滚或修复路径。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

28.18 常见误区与反例

学习“代码开发 Agent”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把 demo 当产品	一次成功演示不能说明私人 Agent 可长期使用。真实项目要面对脏数据、权限、重复任务和用户纠正。
忽略用户控制权	私人订制化必须允许用户查看、修改、删除偏好和记忆，否则个性化会变成黑箱画像。
直接开放写操作	项目第一版应尽量只读。写文件、发消息、提交代码等动作必须在验收稳定后逐步开放。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

28.19 补充注记

对“代码开发 Agent”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

28.20 小结

本章的核心结论是：完成一个软件工程项目的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度

是否能够通过状态、日志、评估和权限控制被管理？

第二十八章 练习

1. 为代码开发 Agent 写一页项目需求。
2. 列出代码开发 Agent 的三个只读工具和两个写工具。
3. 设计一个验收任务，并说明什么算通过。

第二十九章 多 Agent 科研工作流

内容提要

- ❑ 完成一个科研生产线的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：私人订制化 Agent 项目实战。

29.1 问题引入

组织 Planner、Researcher、Engineer、Reviewer、Writer 协作完成研究循环。项目章节的重点是把前文架构组合起来，而不是演示一个孤立 demo。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 29.1 (科研生产线)

科研生产线是围绕用户真实资料、工具和反馈构建的 Agent 项目。它要求系统知道任务边界、数据来源、权限等级和验收标准。

命题 29.1 (项目闭环原则)

每个项目必须同时交付任务流程、数据流、工具权限、评估集和失败处理，而不只是一次成功演示。

29.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义用户场景和输入输出。
- 列出数据源、工具和权限。
- 选择 RAG、Workflow、Memory 或 Multi-Agent 等组合架构。
- 实现最小可运行闭环。
- 用真实失败样本修订。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

29.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def research_loop(topic, team):
    ideas = team['ideator'].run(topic)
    related = team['librarian'].run(ideas)
    plan = team['planner'].run(ideas, related)
    review = team['reviewer'].run(plan)
```

```
return {'plan': plan, 'review': review}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

29.4 深入讲解

科研工作流 Agent 连接 idea、文献、实验、结果和论文。它可以提高效率，但不能替代真实性。实验数据、代码版本和 claim 必须可审计。

多 Agent 科研系统尤其需要反对者。Reviewer 的职责不是润色，而是找出 novelty、实验、统计和写作中的薄弱点。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

29.5 工程实现要点

- 先做只读版本。
- 再加入写操作和人工确认。
- 保存 trace 和中间产物。
- 把用户偏好写成可编辑配置。
- 每次扩展都重新运行评估任务。

29.6 适用与不适用

适用场景：

- 用户有稳定重复的工作流。
- 数据源可整理。
- 任务结果可评价。

不适用或需要谨慎的场景：

- 用户需求尚不清楚。
- 数据没有权限边界。
- 项目只有一次性演示价值。

29.7 失败模式与调试

常见失败模式如下：

- 科研 Agent 不能替代实验真实性和数据审计。
- 没有把用户反馈沉淀为配置或评估。
- 工具可以修改外部系统但缺少审批。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

29.8 评估指标

本章相关系统至少应记录：

- 任务交付质量。
- 引用或测试通过率。
- 人工修改量。
- 复用次数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

29.9 私人定制化实践

多 Agent 科研工作流的私人定制化来自用户自己的资料、术语、风格、项目结构和工具习惯。系统应允许用户查看这些配置，并随时撤销。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

29.10 本章例题

例题 29.1 从 idea 到实验 Ideator 生成想法，Librarian 查新，Planner 写实验计划，Engineer 跑初版，Reviewer 挑错。

解 每个角色输出都应可存档，不能只留对话。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

29.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

29.12 进阶练习

1. 把“从 idea 到实验”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

29.13 设计边界与选择表

项目篇的章节要强调真实数据、真实权限和真实验收。私人定制化不是让模型换一种语气，而是把用户资料、工具、偏好和反馈闭环系统化。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent

做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

29.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

29.15 本章小项目

为自己的一个真实工作流写项目卡片：输入是什么，输出是什么，哪些工具只读，哪些工具需要审批，什么指标表示成功，失败后如何回滚。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

29.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

29.17 案例展开与验收标准

为了避免把“多 Agent 科研工作流”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。

- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能写出真实数据来源、文件布局、权限表和验收任务。
- 能先实现只读闭环，再说明何时加入写操作和人工确认。
- 能把用户反馈转成配置、记忆候选或评估样本，而不是只留在聊天记录中。
- 能报告项目失败样本，并给出回滚或修复路径。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

29.18 常见误区与反例

学习“多 Agent 科研 workflow”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
把 demo 当产品	一次成功演示不能说明私人 Agent 可长期使用。真实项目要面对脏数据、权限、重复任务和用户纠正。
忽略用户控制权	私人订制化必须允许用户查看、修改、删除偏好和记忆，否则个性化会变成黑箱画像。
直接开放写操作	项目第一版应尽量只读。写文件、发消息、提交代码等动作必须在验收稳定后逐步开放。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

29.19 补充注记

对“多 Agent 科研 workflow”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

29.20 小结

本章的核心结论是：完成一个科研生产线的系统设计，从目标、数据流、工具、评估和安全边界落实私人订制化。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第二十九章 练习

1. 为多 Agent 科研 workflow 写一页项目需求。
2. 列出多 Agent 科研 workflow 的三个只读工具和两个写工具。
3. 设计一个验收任务，并说明什么算通过。

第五部分

评估、安全与前沿方向

第三十章 Agent 评估方法

内容提要

- ❑ 理解任务级评估如何决定 Agent 能否从 demo 走向长期可用系统。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：评估、安全与前沿方向。

30.1 问题引入

建立成功率、过程质量、成本延迟和稳定性的评估框架。这一篇处理 Agent 的工程现实：可靠性、可复现性、权限、成本、前沿基准和长期维护。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 30.1 (任务级评估)

任务级评估是衡量和约束 Agent 系统长期行为的一组方法。它关注：建立成功率、过程质量、成本延迟和稳定性的评估框架。

命题 30.1 (可靠性优先原则)

没有评估、日志、权限和维护机制的 Agent，即使单次 demo 成功，也不能视为可靠系统。

30.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义任务和风险模型。
- 采集运行轨迹和结果。
- 用评估集或审计规则检查。
- 修复失败并固化回归测试。
- 周期性复盘成本、权限和用户反馈。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

30.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def score_run(run, gold):
    return {
        'task_success': exact_or_human(run.output, gold.output),
        'tool_success': all(c.ok for c in run.tool_calls),
        'cost': run.tokens * run.price,
```

```
'latency': run.finished_at - run.started_at,
}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

30.4 深入讲解

Agent 评估要同时看结果和过程。结果包括答案是否正确；过程包括工具是否正确、引用是否命中、成本是否可接受、是否违反权限。

评估集应包含正常样本、边界样本、拒答样本和攻击样本。只用成功案例会让系统在真实环境中脆弱。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

30.5 工程实现要点

- 建立 golden tasks。
- 记录 prompt、模型、工具和版本。
- 区分读写执行权限。
- 报告成本和延迟。
- 维护失败样本库。

30.6 适用与不适用

适用场景：

- 任何要长期使用的 Agent。
- 高风险工具场景。
- 科研或生产系统。

不适用或需要谨慎的场景：

- 只追求演示效果而不准备维护。
- 没有日志和测试却声称可靠。
- 把 benchmark 当成唯一目标。

30.7 失败模式与调试

常见失败模式如下：

- 只让 LLM 打分是不充分的。
- 没有失败样本库，问题反复出现。
- 权限边界和用户信任没有一起设计。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

30.8 评估指标

本章相关系统至少应记录：

- 任务成功率和置信区间。
- 工具错误率。
- 人工接管率。
- 成本、延迟和回归失败数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

30.9 私人定制化实践

在私人 Agent 中，Agent 评估方法要直接服务用户控制权。用户应能知道系统做过什么、为什么做、花费多少、保存了什么、如何撤销。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

30.10 本章例题

例题 30.1 十个 golden tasks 为知识库 Agent 准备十个固定问题、标准答案、允许来源和拒答条件。

解 评估应同时记录正确率、引用命中率、成本、延迟和人工修改量。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

30.11 失败模式矩阵

症状	可能原因	修复方式
指标单一	只看 LLM judge 分数	加入任务成功率和人工标注
样本污染	测试题进入 prompt 或训练资料	隔离评估集并记录版本
忽略成本	高分但不可用	报告 token、延迟和重试

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

30.12 进阶练习

1. 为论文阅读 Agent 建立十个 golden tasks。
2. 设计一个统计表，同时记录准确率、引用命中率、成本和延迟。
3. 解释为什么不能只报告平均分。

30.13 设计边界与选择表

可靠性篇的章节要防止 demo 思维。一个 Agent 若不能被评估、调试、约束和维护，就不应被赋予持续运行或外部副作用能力。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

30.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

30.15 本章小项目

从前面任意一个 Agent 项目中抽取十个 golden tasks，记录期望输出、允许工具、禁止动作、成本上限和人工验收规则。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

30.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

30.17 案例展开与验收标准

为了避免把“Agent 评估方法”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把可靠性拆成结果正确、过程合规、成本可接受和失败可复盘四个维度。
- 能写出权限矩阵、威胁模型或 **benchmark** 报告的最小字段。
- 能说明哪些指标可以自动化，哪些指标必须人工审查。
- 能把一次失败转成回归测试、审计规则或运维检查项。

若这些标准无法满足，应降低自动化程度。例如把执行型 **Agent** 改成建议型 **Agent**，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

30.18 常见误区与反例

学习“**Agent** 评估方法”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 **Agent** 从 **demo** 走向实际使用时最容易暴露的结构性问题。

误区	说明
用平均分掩盖长尾	Agent 的危险常出现在少数边界样本中。只报告平均成功率会掩盖高风险失败。
把安全写进提示词	安全边界应在执行器、工具层和权限系统中强制执行，不能只靠模型遵守提示词。
把 benchmark 当真实能力	benchmark 是代理指标。真实私人 Agent 还要看用户数据、工具权限、成本、延迟和维护能力。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 **Agent** 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 **diff**，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 **Agent** 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 **Agent** 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

30.19 补充注记

对“**Agent** 评估方法”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 **Agent** 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

30.20 小结

本章的核心结论是：理解任务级评估如何决定 Agent 能否从 demo 走向长期可用系统。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第三十章 练习

1. 为 Agent 评估方法设计三个检查指标。
2. 说明只让 LLM 打分是不充分的。
3. 把一个失败案例转成回归测试。

第三十一章 失败模式与调试

内容提要

- ❑ 理解系统化排错如何决定 Agent 能否从 demo 走向长期可用系统。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：评估、安全与前沿方向。

31.1 问题引入

从复现、trace、状态快照、工具回放和失败样本库定位问题。这一篇处理 Agent 的工程现实：可靠性、可复现性、权限、成本、前沿基准和长期维护。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 31.1 (系统化排错)

系统化排错是衡量和约束 Agent 系统长期行为的一组方法。它关注：从复现、trace、状态快照、工具回放和失败样本库定位问题。

命题 31.1 (可靠性优先原则)

没有评估、日志、权限和维护机制的 Agent，即使单次 demo 成功，也不能视为可靠系统。

31.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义任务和风险模型。
- 采集运行轨迹和结果。
- 用评估集或审计规则检查。
- 修复失败并固化回归测试。
- 周期性复盘成本、权限和用户反馈。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

31.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def minimize_failure(trace):
    for span in trace.spans:
        replay = run_until(span.id)
        if replay.failed:
            return {'first_bad_span': span.id, 'input': span.input}
```

```
return {'first_bad_span': None}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

31.4 深入讲解

调试 Agent 要从 trace 入手。一个错误答案可能来自 prompt、retrieval、rerank、tool、memory、planner、executor 或 evaluator。

最小失败复现是关键。把失败样本缩小到第一个坏 span，再写入回归测试，才能防止同类问题反复出现。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

31.5 工程实现要点

- 建立 golden tasks。
- 记录 prompt、模型、工具和版本。
- 区分读写执行权限。
- 报告成本和延迟。
- 维护失败样本库。

31.6 适用与不适用

适用场景：

- 任何要长期使用的 Agent。
- 高风险工具场景。
- 科研或生产系统。

不适用或需要谨慎的场景：

- 只追求演示效果而不准备维护。
- 没有日志和测试却声称可靠。
- 把 benchmark 当成唯一目标。

31.7 失败模式与调试

常见失败模式如下：

- 只改 prompt 往往掩盖状态或工具问题。
- 没有失败样本库，问题反复出现。
- 权限边界和用户信任没有一起设计。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

31.8 评估指标

本章相关系统至少应记录：

- 任务成功率和置信区间。
- 工具错误率。
- 人工接管率。
- 成本、延迟和回归失败数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

31.9 私人订制化实践

在私人 Agent 中，失败模式与调试要直接服务用户控制权。用户应能知道系统做过什么、为什么做、花费多少、保存了什么、如何撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

31.10 本章例题

例题 31.1 定位首个坏 span 一次错误答案可能来自检索、重排、生成或引用格式。逐步回放 trace 可以定位第一个异常节点。

解 修复后要把失败样本加入回归集。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

31.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

31.12 进阶练习

1. 把“定位首个坏 span”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

31.13 设计边界与选择表

可靠性篇的章节要防止 demo 思维。一个 Agent 若不能被评估、调试、约束和维护，就不应被赋予持续运行或外部副作用能力。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可

按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

31.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

31.15 本章小项目

从前面任意一个 Agent 项目中抽取十个 golden tasks，记录期望输出、允许工具、禁止动作、成本上限和人工验收规则。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

31.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

31.17 案例展开与验收标准

为了避免把“失败模式与调试”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。

- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把可靠性拆成结果正确、过程合规、成本可接受和失败可复盘四个维度。
- 能写出权限矩阵、威胁模型或 **benchmark** 报告的最小字段。
- 能说明哪些指标可以自动化，哪些指标必须人工审查。
- 能把一次失败转成回归测试、审计规则或运维检查项。

若这些标准无法满足，应降低自动化程度。例如把执行型 **Agent** 改成建议型 **Agent**，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

31.18 常见误区与反例

学习“失败模式与调试”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 **Agent** 从 **demo** 走向实际使用时最容易暴露的结构性问题。

误区	说明
用平均分掩盖长尾	Agent 的危险常出现在少数边界样本中。只报告平均成功率会掩盖高风险失败。
把安全写进提示词	安全边界应在执行器、工具层和权限系统中强制执行，不能只靠模型遵守提示词。
把 benchmark 当真实能力	benchmark 是代理指标。真实私人 Agent 还要看用户数据、工具权限、成本、延迟和维护能力。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 **Agent** 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 **diff**，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 **Agent** 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 **Agent** 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

31.19 补充注记

对“失败模式与调试”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 **Agent** 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

31.20 小结

本章的核心结论是：理解系统化排错如何决定 Agent 能否从 demo 走向长期可用系统。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第三十一章 练习

1. 为失败模式与调试设计三个检查指标。
2. 说明只改 prompt 往往掩盖状态或工具问题。
3. 把一个失败案例转成回归测试。

第三十二章 安全边界与权限控制

内容提要

- ❑ 理解能力与风险治理如何决定 Agent 能否从 demo 走向长期可用系统。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：评估、安全与前沿方向。

32.1 问题引入

用最小权限、沙箱、审批、密钥隔离和审计日志约束 Agent 行为。这一篇处理 Agent 的工程现实：可靠性、可复现性、权限、成本、前沿基准和长期维护。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 32.1 (能力与风险治理)

能力与风险治理是衡量和约束 Agent 系统长期行为的一组方法。它关注：用最小权限、沙箱、审批、密钥隔离和审计日志约束 Agent 行为。

命题 32.1 (可靠性优先原则)

没有评估、日志、权限和维护机制的 Agent，即使单次 demo 成功，也不能视为可靠系统。

32.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义任务和风险模型。
- 采集运行轨迹和结果。
- 用评估集或审计规则检查。
- 修复失败并固化回归测试。
- 周期性复盘成本、权限和用户反馈。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

32.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
POLICY = {  
    'read_file': 'allow',  
    'write_file': 'approval',  
    'send_email': 'approval',  
    'delete_file': 'deny_by_default',
```

```

}

def authorize(action):
    return POLICY.get(action.name, 'deny_by_default')

```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

32.4 深入讲解

安全边界应基于能力，而不是基于模型承诺。执行器必须检查 `action` 是否被允许，不能只在 `prompt` 中告诉模型不要做坏事。

`prompt injection`、`tool injection`、`data exfiltration`、`memory poisoning` 和 `privilege escalation` 都要进入威胁模型。私人 Agent 越贴近真实工作，越需要最小权限。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

32.5 工程实现要点

- 建立 `golden tasks`。
- 记录 `prompt`、模型、工具和版本。
- 区分读写执行权限。
- 报告成本和延迟。
- 维护失败样本库。

32.6 适用与不适用

适用场景：

- 任何要长期使用的 Agent。
- 高风险工具场景。
- 科研或生产系统。

不适用或需要谨慎的场景：

- 只追求演示效果而不准备维护。
- 没有日志和测试却声称可靠。
- 把 `benchmark` 当成唯一目标。

32.7 失败模式与调试

常见失败模式如下：

- 安全不是最后加的提示词，而是架构设计。
- 没有失败样本库，问题反复出现。
- 权限边界和用户信任没有一起设计。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

32.8 评估指标

本章相关系统至少应记录：

- 任务成功率和置信区间。
- 工具错误率。
- 人工接管率。
- 成本、延迟和回归失败数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

32.9 私人订制化实践

在私人 Agent 中，安全边界与权限控制要直接服务用户控制权。用户应能知道系统做过什么、为什么做、花费多少、保存了什么、如何撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

32.10 本章例题

例题 32.1 权限矩阵 read_file 默认允许，write_file 需要确认，send_email 需要确认，delete_file 默认禁止。

解 权限不是提示词承诺，而是执行器强制检查。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

32.11 失败模式矩阵

症状	可能原因	修复方式
越权执行	执行器未检查策略	在工具层强制 authorize
提示注入	外部文档诱导工具调用	隔离文档内容和系统指令
密钥泄露	日志记录 secret	日志脱敏和密钥隔离

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

32.12 进阶练习

1. 为个人助理 Agent 写一张读、写、发送、删除权限矩阵。
2. 构造一个 prompt injection 样本并说明防护点。
3. 说明审批日志应保存哪些字段。

32.13 设计边界与选择表

可靠性篇的章节要防止 demo 思维。一个 Agent 若不能被评估、调试、约束和维护，就不应被赋予持续运行或外部副作用能力。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

32.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

32.15 本章小项目

从前面任意一个 Agent 项目中抽取十个 golden tasks，记录期望输出、允许工具、禁止动作、成本上限和人工验收规则。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

32.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

32.17 案例展开与验收标准

为了避免把“安全边界与权限控制”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授

权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把可靠性拆成结果正确、过程合规、成本可接受和失败可复盘四个维度。
- 能写出权限矩阵、威胁模型或 benchmark 报告的最小字段。
- 能说明哪些指标可以自动化，哪些指标必须人工审查。
- 能把一次失败转成回归测试、审计规则或运维检查项。

若这些标准无法满足，应降低自动化程度。例如把执行型 Agent 改成建议型 Agent，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

32.18 常见误区与反例

学习“安全边界与权限控制”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 Agent 从 demo 走向实际使用时最容易暴露的结构性问题。

误区	说明
用平均分掩盖长尾	Agent 的危险常出现在少数边界样本中。只报告平均成功率会掩盖高风险失败。
把安全写进提示词	安全边界应在执行器、工具层和权限系统中强制执行，不能只靠模型遵守提示词。
把 benchmark 当真实能力	benchmark 是代理指标。真实私人 Agent 还要看用户数据、工具权限、成本、延迟和维护能力。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 Agent 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 diff，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 Agent 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 Agent 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

32.19 补充注记

对“安全边界与权限控制”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 Agent 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

32.20 小结

本章的核心结论是：理解能力与风险治理如何决定 Agent 能否从 demo 走向长期可用系统。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能够通过状态、日志、评估和权限控制被管理？

第三十二章 练习

1. 为安全边界与权限控制设计三个检查指标。
2. 说明安全不是最后加的提示词，而是架构设计。
3. 把一个失败案例转成回归测试。

第三十三章 Agent Benchmark 与科研前沿

内容提要

- ❑ 理解基准与前沿如何决定 Agent 能否从 demo 走向长期可用系统。

❑ 所属部分：评估、安全与前沿方向。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。

33.1 问题引入

理解 SWE-bench、WebArena、OSWorld、GAIA、AgentBench、ToolBench 等基准的能力边界。这一篇处理 Agent 的工程现实：可靠性、可复现性、权限、成本、前沿基准和长期维护。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 33.1 (基准与前沿)

基准与前沿是衡量和约束 Agent 系统长期行为的一组方法。它关注：理解 SWE-bench、WebArena、OSWorld、GAIA、AgentBench、ToolBench 等基准的能力边界。

命题 33.1 (可靠性优先原则)

没有评估、日志、权限和维护机制的 Agent，即使单次 demo 成功，也不能视为可靠系统。

33.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义任务和风险模型。
- 采集运行轨迹和结果。
- 用评估集或审计规则检查。
- 修复失败并固化回归测试。
- 周期性复盘成本、权限和用户反馈。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

33.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def report_benchmark(run):
    return {
        'benchmark': run.name,
        'score': run.score,
        'cost_usd': run.cost,
```

```
'wall_clock': run.duration,
'human_interventions': run.human_steps,
}
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

33.4 深入讲解

Agent benchmark 衡量的是特定环境中的能力。SWE-bench 偏真实代码修复，WebArena 偏网页操作，OSWorld 偏桌面环境，GAIA 偏综合推理和工具使用。

阅读 benchmark 报告时要看环境、工具、成本、人工介入、数据泄漏风险和失败样例。不要把排行榜分数直接等同于自己的私人 Agent 可用性。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

33.5 工程实现要点

- 建立 golden tasks。
- 记录 prompt、模型、工具和版本。
- 区分读写执行权限。
- 报告成本和延迟。
- 维护失败样本库。

33.6 适用与不适用

适用场景：

- 任何要长期使用的 Agent。
- 高风险工具场景。
- 科研或生产系统。

不适用或需要谨慎的场景：

- 只追求演示效果而不准备维护。
- 没有日志和测试却声称可靠。
- 把 benchmark 当成唯一目标。

33.7 失败模式与调试

常见失败模式如下：

- benchmark 分数不等于真实生产能力。
- 没有失败样本库，问题反复出现。
- 权限边界和用户信任没有一起设计。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

33.8 评估指标

本章相关系统至少应记录：

- 任务成功率和置信区间。
- 工具错误率。
- 人工接管率。
- 成本、延迟和回归失败数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

33.9 私人订制化实践

在私人 Agent 中，Agent Benchmark 与科研前沿要直接服务用户控制权。用户应能知道系统做过什么、为什么做、花费多少、保存了什么、如何撤销。

私人订制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

33.10 本章例题

例题 33.1 读 benchmark 报告 同一模型在 SWE-bench 和 WebArena 上的分数不能直接比较，因为任务环境、工具和成功标准不同。

解 报告 benchmark 时必须同时报告成本、时间、环境版本和人工介入。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

33.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

33.12 进阶练习

1. 把“读 benchmark 报告”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

33.13 设计边界与选择表

可靠性篇的章节要防止 demo 思维。一个 Agent 若不能被评估、调试、约束和维护，就不应被赋予持续运行或外部副作用能力。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可

按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

33.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

33.15 本章小项目

从前面任意一个 Agent 项目中抽取十个 golden tasks，记录期望输出、允许工具、禁止动作、成本上限和人工验收规则。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

33.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

33.17 案例展开与验收标准

为了避免把“Agent Benchmark 与科研前沿”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。

- 2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。
- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把可靠性拆成结果正确、过程合规、成本可接受和失败可复盘四个维度。
- 能写出权限矩阵、威胁模型或 **benchmark** 报告的最小字段。
- 能说明哪些指标可以自动化，哪些指标必须人工审查。
- 能把一次失败转成回归测试、审计规则或运维检查项。

若这些标准无法满足，应降低自动化程度。例如把执行型 **Agent** 改成建议型 **Agent**，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

33.18 常见误区与反例

学习“**Agent Benchmark** 与科研前沿”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 **Agent** 从 **demo** 走向实际使用时最容易暴露的结构性问题。

误区	说明
用平均分掩盖长尾	Agent 的危险常出现在少数边界样本中。只报告平均成功率会掩盖高风险失败。
把安全写进提示词	安全边界应在执行器、工具层和权限系统中强制执行，不能只靠模型遵守提示词。
把 benchmark 当真实能力	benchmark 是代理指标。真实私人 Agent 还要看用户数据、工具权限、成本、延迟和维护能力。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 **Agent** 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 **diff**，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 **Agent** 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 **Agent** 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

33.19 小结

本章的核心结论是：理解基准与前沿如何决定 **Agent** 能否从 **demo** 走向长期可用系统。设计 **Agent** 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第三十三章 练习

- 1. 为 **Agent Benchmark** 与科研前沿设计三个检查指标。

2. 说明 benchmark 分数不等于真实生产能力。
3. 把一个失败案例转成回归测试。

第三十四章 构建自己的长期 Agent 系统

内容提要

- ❑ 理解长期运行架构如何决定 Agent 能否从 demo 走向长期可用系统。
- ❑ 本章统一从问题、定义、流程、代码、失败模式、评估和私人实践展开。
- ❑ 所属部分：评估、安全与前沿方向。

34.1 问题引入

把队列、记忆、权限、监控、知识更新和用户反馈整合为长期系统。这一篇处理 Agent 的工程现实：可靠性、可复现性、权限、成本、前沿基准和长期维护。

Agent = (模型, 提示, State, Memory, Tool, Planner, Executor, Evaluator).

在不同章节中，上式的某些项会更突出。例如 RAG 强调知识与证据，LangGraph 强调状态和边，Human-in-the-loop 强调人工反馈与权限边界。

定义 34.1 (长期运行架构)

长期运行架构是衡量和约束 Agent 系统长期行为的一组方法。它关注：把队列、记忆、权限、监控、知识更新和用户反馈整合为长期系统。

命题 34.1 (可靠性优先原则)

没有评估、日志、权限和维护机制的 Agent，即使单次 demo 成功，也不能视为可靠系统。

34.2 执行流程

一个可落地的实现通常包含以下步骤：

- 定义任务和风险模型。
- 采集运行轨迹和结果。
- 用评估集或审计规则检查。
- 修复失败并固化回归测试。
- 周期性复盘成本、权限和用户反馈。

这些步骤的顺序可以因任务而变，但它们共同保证系统不是一次性问答，而是可观察、可修正、可评估的执行过程。

34.3 最小 Python 骨架

下面的代码固定核心接口。它不是生产级完整实现，而是用于说明状态、工具和评估如何进入同一个闭环。

```
def nightly_maintenance(system):
    system.queue.retry_stale_jobs()
    system.memory.expire_old_candidates()
    system.permissions.rotate_tokens()
    system.metrics.detect_drift()
```

```
system.report.send_summary()
```

 **笔记** 如果示例中省略了异常处理、鉴权、重试、缓存或日志，并不表示真实系统可以省略它们。讲义中的最小代码服务于理解；生产系统必须补全工程边界。

34.4 深入讲解

长期 Agent 系统需要运维思维。任务队列、定时检查、记忆生命周期、权限续期、漂移检测和健康报告，比单次回答更重要。

私人长期系统还要尊重用户控制权。用户应能查看保存了什么记忆，哪些任务在运行，哪些工具有权限，如何暂停、撤销和导出数据。

从工程角度看，本章内容至少涉及三层边界。第一层是任务边界：系统究竟要解决什么问题，何时应该拒绝或澄清。第二层是数据边界：哪些信息可以进入上下文，哪些信息只能作为受控工具结果出现。第三层是行动边界：哪些动作可以自动执行，哪些动作必须进入人工确认。只有三层边界都清楚，Agent 才能从演示走向可维护系统。

34.5 工程实现要点

- 建立 golden tasks。
- 记录 prompt、模型、工具和版本。
- 区分读写执行权限。
- 报告成本和延迟。
- 维护失败样本库。

34.6 适用与不适用

适用场景：

- 任何要长期使用的 Agent。
- 高风险工具场景。
- 科研或生产系统。

不适用或需要谨慎的场景：

- 只追求演示效果而不准备维护。
- 没有日志和测试却声称可靠。
- 把 benchmark 当成唯一目标。

34.7 失败模式与调试

常见失败模式如下：

- 长期 Agent 的核心是维护，而不是一次性 demo。
- 没有失败样本库，问题反复出现。
- 权限边界和用户信任没有一起设计。

调试时先不要急着改提示词。更可靠的顺序是：复现失败，固定输入和模型版本，查看状态快照，检查工具输入输出，再判断是检索、记忆、规划、执行还是权限边界出了问题。

34.8 评估指标

本章相关系统至少应记录：

- 任务成功率和置信区间。
- 工具错误率。
- 人工接管率。
- 成本、延迟和回归失败数。

这些指标不一定全部自动化，但至少要能在失败复盘时被查看。没有指标的 Agent 只能依赖主观感觉，很难持续改进。

34.9 私人定制化实践

在私人 Agent 中，构建自己的长期 Agent 系统要直接服务用户控制权。用户应能知道系统做过什么、为什么做、花费多少、保存了什么、如何撤销。

私人定制化的本质不是让模型语气更亲切，而是让系统接入用户真实资料、工具和反馈，同时保留用户的控制权。任何长期保存的偏好或记忆都应能被解释、编辑和删除。

34.10 本章例题

例题 34.1 长期维护 每晚任务检查队列、清理过期记忆候选、轮换 token、统计失败率并发送摘要。

解 长期 Agent 的价值来自稳定维护，而不是一次华丽演示。处理这类任务时，应把观察写入状态，把证据或工具输出保留下来，并在最终答案中说明哪些结论来自外部证据，哪些只是模型整理。

34.11 失败模式矩阵

症状	可能原因	修复方式
结果不稳定	状态或输入没有固定	记录版本、状态和随机性
工具误用	schema 或说明不清	收窄工具并加入校验
无法复盘	缺少 trace	记录动作、观察和错误

矩阵的作用是把“模型表现不好”拆成可定位的问题。实际调试时，不应只看最终答案，而要同时检查输入、检索、工具、状态、权限和输出格式。

34.12 进阶练习

1. 把“长期维护”改造成一个可测试的小任务。
2. 为该任务写出一个失败样本和对应修复策略。
3. 给出一个不应该自动执行、必须人工确认的动作。

34.13 设计边界与选择表

可靠性篇的章节要防止 demo 思维。一个 Agent 若不能被评估、调试、约束和维护，就不应被赋予持续运行或外部副作用能力。对本章主题而言，最重要的边界不是“能不能做”，而是“是否应该让 Agent 做”。判断时可按下面四个问题展开。

1. 任务是否有明确完成标准。如果没有，系统应先帮助用户澄清目标，而不是直接执行。
2. 所需信息是否来自可靠来源。如果答案需要事实依据，就必须引入检索、引用或外部校验。
3. 工具是否会产生外部副作用。凡是写文件、发消息、提交代码、创建日程、删除数据或调用付费 API，都应有确认和审计。
4. 失败是否可以恢复。若失败不可恢复，就应降低自动化程度，把 Agent 设计为建议者而不是执行者。

注 本章主题可以和其他架构组合。例如一个私人论文阅读系统可能同时使用 RAG、工具调用、记忆、反思和人在回路中。但组合的前提是每个模块都有清晰职责；若只是把多个框架叠在一起，系统会更难调试。

34.14 实现清单

落地本章内容时，建议按下面清单逐项检查。

1. 写出任务输入、输出和拒绝条件。拒绝条件和成功条件同样重要。
2. 写出状态结构。至少包含目标、当前阶段、轨迹、错误、待确认事项和最终产物。
3. 写出工具表。每个工具都要有参数 schema、返回格式、权限等级和失败返回。
4. 写出日志格式。一次运行结束后，应能看到模型请求、工具调用、观察结果、成本和延迟。
5. 写出评估任务。至少准备三个正常样本、三个边界样本和三个应拒绝或应请求确认的样本。

34.15 本章小项目

从前面任意一个 Agent 项目中抽取十个 golden tasks，记录期望输出、允许工具、禁止动作、成本上限和人工验收规则。小项目不追求功能完整，而追求闭环完整。交付物应包含：一份状态结构说明、一份工具权限表、一段最小 Python 骨架、三条测试任务和一次失败复盘。

推荐的复盘格式如下：

- 失败输入：用户实际给了什么目标或文件。
- 期望行为：系统本应回答、检索、调用工具还是请求确认。
- 实际行为：模型选择了什么动作，工具返回了什么观察。
- 根因分类：提示词、检索、工具、状态、记忆、权限或评估中的哪一类。
- 修复方式：修改 schema、补充测试、收窄权限、增加检查节点或调整上下文组织。

34.16 与后续章节的关系

本章不是孤立知识点。它要么为后续章节提供基础概念，要么在后续项目中作为一个可替换模块出现。阅读时应特别注意两个方向：一是从简单实现走向更可控的工作流；二是从一次性任务走向带记忆、权限和评估的长期私人系统。

34.17 案例展开与验收标准

为了避免把“构建自己的长期 Agent 系统”停留在概念层面，本节给出一个更接近真实开发的验收视角。一个章节中的 Agent 设计，至少应能被写成一份小型工程说明：输入是什么，输出是什么，状态如何保存，工具如何授权，失败如何复现，用户如何介入。

1. 输入边界：列出系统允许接收的用户请求、文件类型、外部资源和任务目标。若输入缺少必要信息，Agent 应先澄清，而不是猜测。
2. 状态边界：说明哪些字段进入运行状态，例如阶段、轨迹、错误、证据、待确认动作和最终产物。状态字段越清楚，失败回放越容易。

- 3. 工具边界：区分只读、建议、写入、执行和发布五类动作。只读工具可以较宽，写入和发布工具必须有限权限检查和审计记录。
- 4. 评估边界：为本章主题准备一组固定任务。每个任务都应包含期望结果、允许工具、禁止动作、成本上限和人工验收规则。
- 5. 维护边界：说明当模型、框架、资料库或用户偏好变化时，哪些测试必须重新运行，哪些记忆或索引需要更新。

本章通过验收的最低标准如下：

- 能把可靠性拆成结果正确、过程合规、成本可接受和失败可复盘四个维度。
- 能写出权限矩阵、威胁模型或 **benchmark** 报告的最小字段。
- 能说明哪些指标可以自动化，哪些指标必须人工审查。
- 能把一次失败转成回归测试、审计规则或运维检查项。

若这些标准无法满足，应降低自动化程度。例如把执行型 **Agent** 改成建议型 **Agent**，把写操作改成草稿生成，把长期记忆改成临时状态。工程上宁可让系统少做一点，也不要让它在没有边界的情况下做得过多。

34.18 常见误区与反例

学习“构建自己的长期 **Agent** 系统”时，最容易出现的问题是把概念、框架或一次成功输出误认为完整系统。下面列出若干常见误区。它们不是细枝末节，而是 **Agent** 从 **demo** 走向实际使用时最容易暴露的结构性问题。

误区	说明
用平均分掩盖长尾	Agent 的危险常出现在少数边界样本中。只报告平均成功率会掩盖高风险失败。
把安全写进提示词	安全边界应在执行器、工具层和权限系统中强制执行，不能只靠模型遵守提示词。
把 benchmark 当真实能力	benchmark 是代理指标。真实私人 Agent 还要看用户数据、工具权限、成本、延迟和维护能力。

一个简单反例是：用户要求系统“帮我整理项目文件”。如果 **Agent** 没有先区分读取、建议、移动和删除四类动作，就可能把整理建议直接变成破坏性操作。正确做法是先生成计划和 **diff**，让用户确认后再执行。这个反例在本章主题下尤其重要，因为它说明 **Agent** 的能力越强，边界越要前置。

还要注意，私人订制化不是无限自动化。一个真正可用的私人 **Agent** 应该能解释自己使用了哪些资料，调用了哪些工具，保存了哪些偏好，哪些动作等待用户确认。若系统无法回答这些问题，就应把它降级为辅助阅读、辅助写作或辅助规划工具，而不是让它长期自主运行。

34.19 补充注记

对“构建自己的长期 **Agent** 系统”而言，最后还要检查一个实践问题：这个设计是否能被另一个人接手。若一个 **Agent** 只能在作者电脑上、只凭作者记忆运行，那么它还不是稳定系统。稳定系统至少要留下配置文件、运行日志、测试样本和权限说明。

在团队或长期项目中，建议把本章相关决策写入项目文档：为什么选择这种架构，放弃了哪些更简单的方案，哪些工具被允许，哪些操作被禁止，出现失败时先看哪份日志。这样做看似繁琐，却能显著降低后续维护成本。

34.20 小结

本章的核心结论是：理解长期运行架构如何决定 Agent 能否从 demo 走向长期可用系统。设计 Agent 时要同时问三个问题：它比简单问答多解决了什么问题？它增加了哪些复杂度？这些复杂度是否能通过状态、日志、评估和权限控制被管理？

第三十四章 练习

1. 为构建自己的长期 Agent 系统设计三个检查指标。
2. 说明长期 Agent 的核心是维护，而不是一次性 demo。
3. 把一个失败案例转成回归测试。

参考文献与延伸阅读

- [1] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: (2024). URL: <https://arxiv.org/abs/2310.06770>.
- [2] *LangChain Documentation*. 2026. URL: <https://python.langchain.com/>.
- [3] *LangGraph Documentation*. 2026. URL: <https://langchain-ai.github.io/langgraph/>.
- [4] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: (2020). URL: <https://arxiv.org/abs/2005.11401>.
- [5] *Model Context Protocol*. 2026. URL: <https://modelcontextprotocol.io/>.
- [6] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: (2023). URL: <https://arxiv.org/abs/2302.04761>.
- [7] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: (2023). URL: <https://arxiv.org/abs/2303.11366>.
- [8] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: (2023). URL: <https://arxiv.org/abs/2210.03629>.
- [9] Shuyan Zhou et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents”. In: (2023). URL: <https://arxiv.org/abs/2307.13854>.